

MT4/5 融合 EA 编程手册

[v2.1]

按照本手册技术规范编写的 MT4 程序，

放到 MT5 中不需要修改代码，反过来也可以

这套模版不能将已有的 MT4 程序转换成 MT5，反过来也不能

- 在 MT4 中编写的代码拷贝到 MT5 中无需修改，重新编译即可生成 ex5 可执行文件。
- 在 MT5 中编写的代码拷贝到 MT4 中无需修改，重新编译即可生成 ex4 可执行文件。
- 弥补了 MT4/5 语言体系中的一个重大功能项缺失：持仓单与历史单检索与统计。
- 内置多样化程序授权模式。
- 内置功能丰富的控制面板范例。
- 框架式设计，模块化编程，把精力放在策略逻辑组合上。

老易(QQ:921795 Mobil 微信:13974863868)

2023-4-19

目录

1	综述.....	1
1.1	自动化是金融交易的主流.....	1
1.2	本书的由来.....	1
1.3	本书的特点.....	1
1.4	写给新手.....	2
1.5	互助学习.....	3
1.6	关于 2.1 版.....	3
1.7	关于版权.....	3
2	模版安装与使用.....	4
2.1	程序安装.....	4
2.2	模版安装.....	4
2.3	编辑运行.....	5
2.4	范例程序.....	7
3	总体设计.....	12
3.1	总体框架.....	12
3.2	模块最小化.....	12
3.3	主程序结构.....	12
3.4	规范编码.....	13
4	基础设计.....	14
4.1	预定义变量.....	14
4.1.1	账户信息 AccountInfo(AI)	14
4.1.2	商品信息 SymbolInfo(SI)	15
4.1.3	持仓单信息 TradesOrders(TO)	16
4.1.4	持仓单统计信息 TradesStatistical(TS).....	16
4.1.5	历史单信息 HistoryOrders(HO).....	17
4.1.6	历史单统计信息 HistoryStatistical(HS)	17
4.1.7	利润保护 ProfitProtect(PP)	18
4.1.8	平仓数组 CloseTicket.....	18
4.1.9	前一个服务器时间 prv_servertime	18
4.1.10	图形对象 ObjectAttributeStruct(OAS).....	19
4.1.11	常量.....	19
4.2	账户与商品信息.....	19
4.2.1	账户信息.....	19
4.2.2	商品信息.....	20
4.3	订单统计与检索.....	20
4.3.1	持仓单.....	21
4.3.2	历史单.....	22
5	主程序 egMain	24
6	刷新数据 egDataRefresh	25
7	持仓单分类 egTOClassify	26
8	交易信号 egTradingSignal	28
9	建仓 egCreat	31

10	加仓 egAdd	33
11	止盈止损 egTPSL	36
11.1	设置初始止盈止损.....	36
11.2	设置移动止损.....	37
12	减仓平仓 egReduce.....	39
12.1	指定单号平仓.....	39
12.2	数组平仓.....	39
12.3	条件平仓.....	39
12.4	同向对冲.....	40
12.5	反向对冲.....	40
12.6	利润保护.....	41
12.7	盈损对冲.....	42
12.8	减仓平仓范例.....	43
13	EA 授权 egVerify	47
13.1	使用期限.....	49
13.2	交易服务器.....	49
13.3	账户余额.....	50
13.4	帐号类型.....	50
14	EA 封面	51
15	控制面板设计.....	52
15.1	标准程序库简介	52
15.2	面向对象编程.....	52
15.3	创建个性化类和实体.....	53
15.4	创建面板元件.....	55
15.5	面板初始化 PanelInit()	56
15.6	面板行为 PanelAction()	57
15.7	刷新面板数据 PanelRefresh()	60
15.8	回测模式 PanelTestMode();	61
16	命令集.....	64
16.1	账户与商品信息.....	64
16.1.1	刷新账户信息.....	64
16.1.2	商品信息.....	64
16.1.3	持仓商品.....	65
16.2	持仓单统计与检索.....	66
16.2.1	复制持仓单数组.....	66
16.2.2	持仓单数组按类型排序.....	66
16.2.3	刷新持仓单数组.....	67
16.2.4	刷新持仓单统计信息.....	68
16.2.5	持仓单检索.....	68
16.2.6	持仓单定位.....	70
16.2.7	持仓单数组数组合并.....	70
16.3	历史单统计与检索.....	72
16.3.1	复制历史单数组.....	72
16.3.2	历史单数组按类型排序.....	72

16.3.3	刷新历史单数组.....	73
16.3.4	刷新历史单统计信息.....	73
16.3.5	历史单检索.....	74
16.3.6	历史单定位.....	75
16.3.7	历史单总数.....	76
16.4	订单操作.....	76
16.4.1	建仓.....	76
16.4.2	建仓量整形.....	77
16.4.3	同类单间距.....	78
16.4.4	指定 k 线持仓单数量.....	78
16.4.5	指定 k 线历史单平仓数量.....	79
16.4.6	持仓单设置止盈止损.....	80
16.4.7	订单平仓.....	80
16.4.8	数组平仓.....	81
16.4.9	条件平仓.....	81
16.4.10	同向对冲.....	82
16.4.11	反向对冲.....	83
16.4.12	利润保护.....	84
16.4.13	盈损对冲.....	85
16.5	常规函数.....	87
16.5.1	交易状态.....	88
16.5.2	k 线平均高度(点).....	89
16.5.3	查找订单注释中是否有指定关键字的订单.....	89
16.5.4	查找是否有指定程序识别码的订单.....	90
16.5.5	金额换算建仓量.....	91
16.5.6	在字符串中获取特征码.....	91
16.5.7	有效时间段.....	92
16.5.8	双线交叉.....	92
16.5.9	文件写入.....	93
16.5.10	读 csv 文件数据到二维数组.....	94
16.6	技术指标.....	94
16.6.1	k 线数据.....	95
16.6.2	egldc_AC.....	95
16.6.3	egldc_ADX.....	96
16.6.4	egldc_Alligator.....	96
16.6.5	egldc_AO.....	98
16.6.6	egldc_ATR.....	98
16.6.7	egldc_Bands.....	99
16.6.8	egldc_BearsPower.....	100
16.6.9	egldc_BullsPower.....	101
16.6.10	egldc_CCI.....	102
16.6.11	egldc_DeMarker.....	103
16.6.12	egldc_Envelopes.....	103
16.6.13	egldc_iFractals.....	105

16.6.14	egIdc_Gator.....	105
16.6.15	egIdc_Ichimoku	107
16.6.16	egIdc_MA	108
16.6.17	egIdc_MACD.....	109
16.6.18	egIdc_Momentum.....	110
16.6.19	egIdc_OsMA	110
16.6.20	egIdc_RSI	112
16.6.21	egIdc_RVI.....	112
16.6.22	egIdc_SAR.....	113
16.6.23	egIdc_StdDev.....	114
16.6.24	egIdc_Stochastic.....	115
16.6.25	egIdc_WPR (%R)	116
16.6.26	egIdc_ZigZag.....	117
16.7	图形对象.....	118
16.7.1	输出信息到图表.....	118
16.7.2	输出信息到日志.....	118
16.7.3	输出向下箭头.....	118
16.7.4	输出向上箭头.....	119
16.7.5	以像素坐标输出 BMP 图形	120
16.7.6	输出按钮.....	122
16.7.7	输出可编辑文字框.....	123
16.7.8	输出水平线.....	124
16.7.9	输出矩形标签.....	124
16.7.10	输出趋势线.....	126
16.7.11	输出垂直线.....	126
16.7.12	点击鼠标获取对象名	127
16.7.13	图表环境初始化.....	127
16.7.14	移动鼠标获取对象名	128
16.7.15	鼠标移动到对象显示备注	128
16.7.16	鼠标移动对象.....	129
16.7.17	对象跟随坐标移动.....	129
16.7.18	刷新对象显示内容	130
16.7.19	删除关键字对象.....	130
16.7.20	组合对象:步进取值.....	130
16.7.21	组合对象:单选.....	132
16.7.22	关于面板制作.....	133

【文档更新记录】

日期	版本	更新内容
2021/3/15	1.0	开始编写。
2021/4/1	1.1	第一稿完成。
2021/5/1	1.2	第二稿完成。
2021/5/15	1.3	添加 MT4/5 全部公用技术指标共计 25 个。
2021/6/2	1.4	1. 增加自定义指标 ZigZag 范例。 2. MT5 更新后语法调整, OC5.mqh 相应语句做了调整。
2021/7/13	1.6	1. 文本增加“交易状态”函数 egTradingStatus()说明。 2. 持仓单统计信息(TS)中增加“最大浮盈、最大浮亏”预定义变量和说明, 该值在程序中自动刷新。 3. 主程序改名为“MT45FUSE[1.6]”。 4. 修复上一个版本按钮失效的 bug。
2021/7/15	1.7	1. 函数库 egOC.mqh 文件添加持仓单数组合并(egTOMerge)函数。 2. 主程序添加持仓单分类(egTOClassify)函数(范例)。
2021/8/22	1.8	1. 函数库 egUI.mqh 添加“组合对象:单选”(egUI_ObjGrp_CheckBox)函数。 2. 函数库 egUI.mqh 添加“组合对象:步进取值”(egUI_ObjGrp_StepBox)函数。 3. 函数库 egUI.mqh 添加“输出水平线”(egObj_HLine)函数。 4. 主程序修正授权失效后不能正常退出 EA。
2023/3/22	2.0	1. 提供简洁编程模版: MT45FUSE[2.0]。 2. 提供一个带有详细中文注释的范例 EA: MT45_Demo[2.0]。 3. 更新 egOC5.mq5 中的建仓、平仓函数。 4. 修复 egOC.mqh 中的 egFileRW()错误逻辑。 5. 修正 egOC.mqh 中的 egTimeValid()错误逻辑。 6. 在 egUI.mqh 中添加画趋势线函数 egObj_Trend()。 7. 在 egOC.mqh 中添加推送信息函数 egSendInfo()。 8. 将平仓数组变量 CloseArray 原类型 int 修改为 ulong。 9. 共计 25 个常用指标, 分别在 egIdc4.mqh 和 egIdc5.mqh 中。 10 在 egGV.mqh 中添加了一个历史平仓单号数组 ticket_hst[]变量, 用于 egOrdersInShiftHst 函数返回指定 k 线上的所有平仓单的单号。 11 在 egGV.mqh 中添加了一个持仓单单号数组 ticket_cur[]变量, 用于 egOrdersInShift 函数可返回指定 k 线上的所有持仓单的单号。 12 修正并简化 egUI.mqh 中创建对象函数的代码。 13 egUI.mqh 中添加三个创建对象函数, 分别是: 在指定时间价位标注向下箭头(egObj_ArrowDown)、在指定时间价位标注向上箭头(egObj_ArrowUp)、垂直线(egObj_VLine)输出趋势线(egObj_Trend)。 14 更正有效时间段函数 egTimeValid 算法。
2023/4/19	2.1	1. 修复 egOC.mqh 中盈亏对冲(egTSHR())函数的一个错误逻辑。 2. 全新打造“MT45 融合模版范例”, 控制面板按钮在 MT4/5 历史数据测试中可用。

日期	版本	更新内容
		3. 加固个别函数逻辑 4. 重写本手册“控制面板”一章内容。

1 综述

1.1 自动化是金融交易的主流

外汇交易是金融市场重要的组成部分。遍布全球的外汇交易所形成了一个不间断的大型交易场所。银行间的兑换全部在交易所场内进行，场外则由做市商为更大规模的投资人提供交易平台，完成外汇交易。

一周五天，每天 24 小时不间断的行情、高杠杆、快进快出的交易特点正在吸引更多的人参与其中。

有统计数据表明 2017 年全球每日外汇交易额为 10 万亿美元，2020 年增长为每日 20 万亿美元。经济发展自然是如此大幅度增长的根本原因，而在如此大量的交易面前，自动化交易所做的贡献也毋庸置疑。

金融市场还包括了债券、股票、期货、现货、金融衍生品的交易。它们都已经进入到了自动化时代，学会编程成为大多数从业人员的充电内容。

1.2 本书的由来

金融交易发展至今，几乎所有交易软件都支持自动化交易。要实现自动化交易就需要编写程序，而现实情况是，每个交易软件都有属于自己的一套语言语法体系，相互之间不通用。这就意味着在 A 平台编写好的程序，想要到 B 平台执行，必须重写代码。

“有趣”的是，用户最多的外汇交易软件公司迈达克公司目前的 MT4 和 MT5 两套交易软件居然也会“语言不通”。

在工作中我经常面对这种尴尬局面，在为客户编写好 MT4 程序之后，又被要求“转换”为 MT5。语言不通，不是简单转换就能完成的，而是意味着要重写代码，各种逻辑也要重新测试。

深入了解 MT4/5 语言语法之后，我决定编写一套“MT4/5 融合编程模版”。希望能实现：在 MT4 中编写好的程序，拷贝到 MT5 中，不需要改一个字母就能使用，反过来也是一样。今天，这个目标实现了。

1.3 本书的特点

这不是一本 MT4/5 操作手册，上手之前需要熟悉 MT4/5 软件的功能，要熟悉 MT4/5 代码编辑器的使用。这些资料可以在相应的软件中按一下“F1”键就能获得。

本书沿着“信号->建仓->加仓->止盈止损->减仓平仓->控制面板->授权许可”的主线，逐一介绍“MT4/5 融合编程模版”中各种函数的使用，常见逻辑模块的搭建，文中穿插各种交易方法的描述。

1.4 写给新手

从 2009 年开始接触迈达克公司的 MT3 到现在的 MT4/5，我经历了自动化交易的技术变革，接触过各种包括股票、期货、外汇的程序语言，见证了迈达克公司从小到大的变化。我认为在学习编写交易程序时，MT4/5 是首选，因为 MT4/5 语言遵循 C、C++ 语言规范，可以很好地应对各种交易策略的逻辑。

我把学习程序的人群分为程序员和新手两类：

第一类，程序员。无论你擅长哪种计算机语言，用“MT4/5 融合编程模版”都可以快速上手。

第二类，新手。在这里要多说几句。

虽然这本书的目的是让读者能够高级别快速入门，但实际上不可能对每个人都有效。多年的工作经验告诉我，编程这门技能并不适合所有人学习，它要求学习者具备较强的逻辑思维能力和逻辑表达能力，如果要成为高手，更需要一些天生禀赋。

“不试试，怎么知道我不行？”——没错，我鼓励所有想学编程的人投入时间和精力来尝试。但请记住，时间和精力是人生的成本，是金钱买不到的资源，是不可再生的资源。所以，如果你意识到自己“不够行”，我希望你能尽快完成“从入门到放弃”的过程，尽快找到适合自己的角色。因为，不会编程并不能阻碍你成长为金融界翘楚。

我建议，不妨限定一个不长的时间做尝试。在尝试期间向身边有经验的程序员咨询讨教，以尽快确认自己是否具备学好编程的各种基础条件，比如上面提到的，逻辑思维及表达能力。

根据以往的经验，推荐做一个连续 20 天，每天 1 小时的学习计划。

不会编程的新手是否有高起点入门的途径？答案是肯定的。卖油翁曾经说过：“我亦无他，唯手熟尔。”学习编程就是一个勤动手、多思考的过程。所有人都会历经，从照葫芦画瓢开始，通过反复练手，反复思考，最终到能够独立创作。

本书提供了若干个“高级葫芦”，无论你对语句是否理解，都可以先从一个具体的案例出发，在反复编写练习的过程中，逐步延伸阅读例如 C 语言规则之类的基础性知识。需要强调的是，只有在学习实践中出现了不懂的或者需要进一步理解的问题的时候，才特别需要去找资料学习，而不需要入门就先学那些枯燥的基础编程知识，“苦练基本功”完全可以在照葫芦画瓢阶段中完成。

真正的高起点，第一条是“兴趣”，能够一直保持兴致勃勃的状态，才能保持成长的高效率，这是快速成为精通编程高手的必要条件。

高起点第二条则是“坚持”，既然选择了就不要放弃。作为新手自然有很多不解、疑惑、甚至摸不着头脑的时候，这很正常。“读书百遍，其义自见”说的就是这个道理，反复动手做，做着做着，有些东西就会在无数的亲身体验和思考中豁然开朗了。反而，不停地查书找资料，死磕一个知识点，并不见得有很好的效果。

在完成了练手任务后，你就可以开始尝试着编写与范例不一样的程序，进入到创作阶段，独立实现一些小功能模块。

如果你是一名操盘手，学习编程的目的也只是为了能够简单编写一些自己想要的功能，那么就请切记不要忘记初心，避免在学习编程过程中走火入魔，避免变成一名蹩脚的程序员，还把交易给荒废了。

如果你是一名程序员，并且想在自动化交易编程方面有所作为，那么务必要熟悉甚至精通交易技术和交易规则，因为一个不懂交易的程序员不是好程序员。然而，交易显然存在巨大的诱惑，所以切记不要忘记初心，避免在研究交易方法与策略时走火入魔，避免变成一名蹩脚的操盘手，还把编程给荒废了。

无论是操盘手还是程序员，想要做好自己的角色，都需要多年的积累和沉淀。成功绝没有捷径可走，成为双料冠军更是需要无比坚韧的耐力和绝对清晰的头脑。

1.5 互助学习

独自闷头学不如寻伴一起学，大家互为师生，在提问中成长，在解答中升华。

凡是购买“MT4/5 融合编程模版”完整版的朋友均可免费加入互助学习 QQ 群。

以后的升级更新版（包括文档、代码等）也会及时共享到群文件中，免费下载。

1.6 关于 2.1 版

自从 2021 年发布 1.9 版至今，有许多朋友在学习和实战过程中反馈了大量的宝贵经验，这让 MT4/5 融合编程系统更加完善，各项函数模块也更加牢固。在此期间，我也用模版编写了大量 EA，进一步验证了这套系统的可靠性。

按照本手册技术规范编写 EA 程序后可以发布成一个单独的可执行 EA 文件，这个文件可以在其他电脑中运行，不需要附带函数库。

MT4/5 融合 EA 编程[2.0]包含了一个只有框架的名为“MT45FUSE[2.0]”的模版程序，一个含有图形化控制面板的可自动交易的名为“MT45 融合模版范例”的范例程序，一套 include 库文件。

这个版本非常稳定，重大升级应该在很久以后了。

我决定，今天，正式发布《MT4/5 融合 EA 编程手册[2.1]》，同时配套一个 chm 文档，方便在编码时查阅资料。

1.7 关于版权

自从 1.9 版发布以来，有“热心”网友将这套系统在一些网站上收费发布，并自称是原创作者。

为我做宣传，我表示感谢！为自己创收，我表示理解！

还请给予我最基本的尊重，请务必注明原创作者是“老易”。

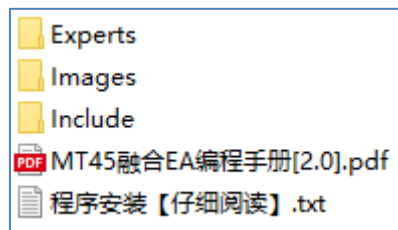
2 模版安装与使用

2.1 程序安装

到做市商提供的网站下载电脑版的 MT4 或者 MT5，按提示安装并运行相关程序。

2.2 模版安装

获取的程序包解压后有以下文件：



其中“Experts”、“Include”文件夹中是程序源码，“Images”是图片。另外有一份操作手册和程序安装说明。

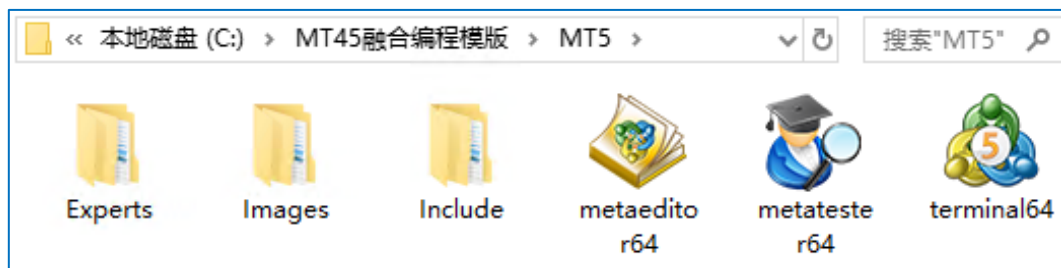
- 以 MT5 为例，MT4 操作步骤一样。MT5 终端运行后，打开数据文件夹：



- 双击进入 MQL5（MQL4）文件夹：

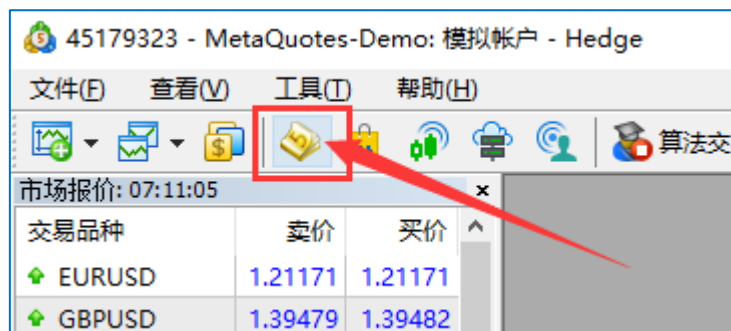


- 拷贝 Experts 文件夹、Include 文件夹和 Images 文件夹到数据文件夹中的 MQL5 (MQL4) 中:

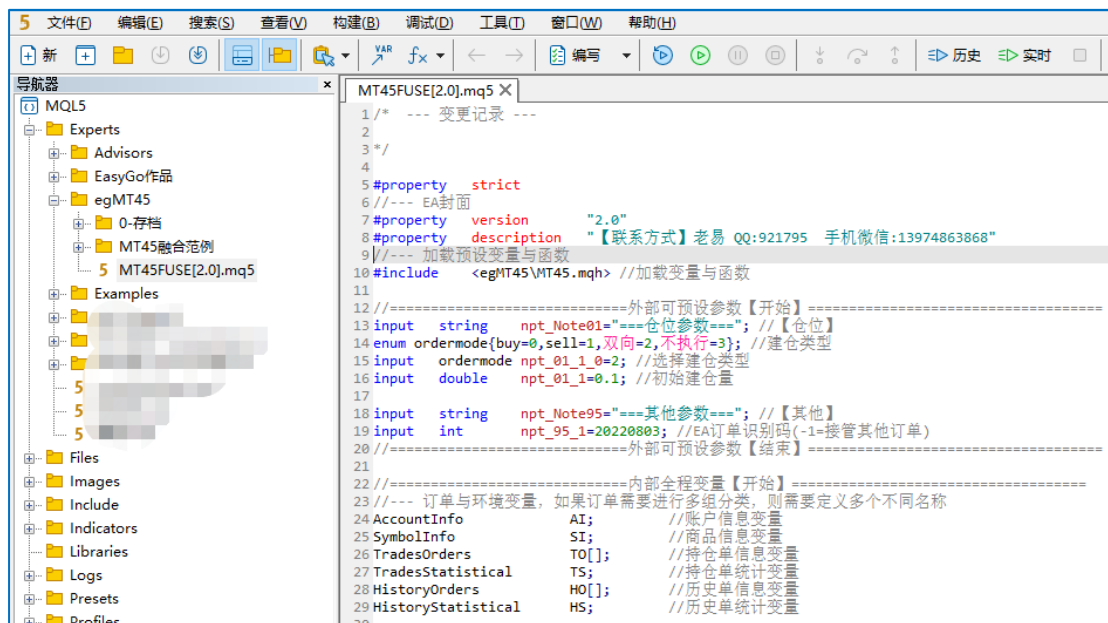


2.3 编辑运行

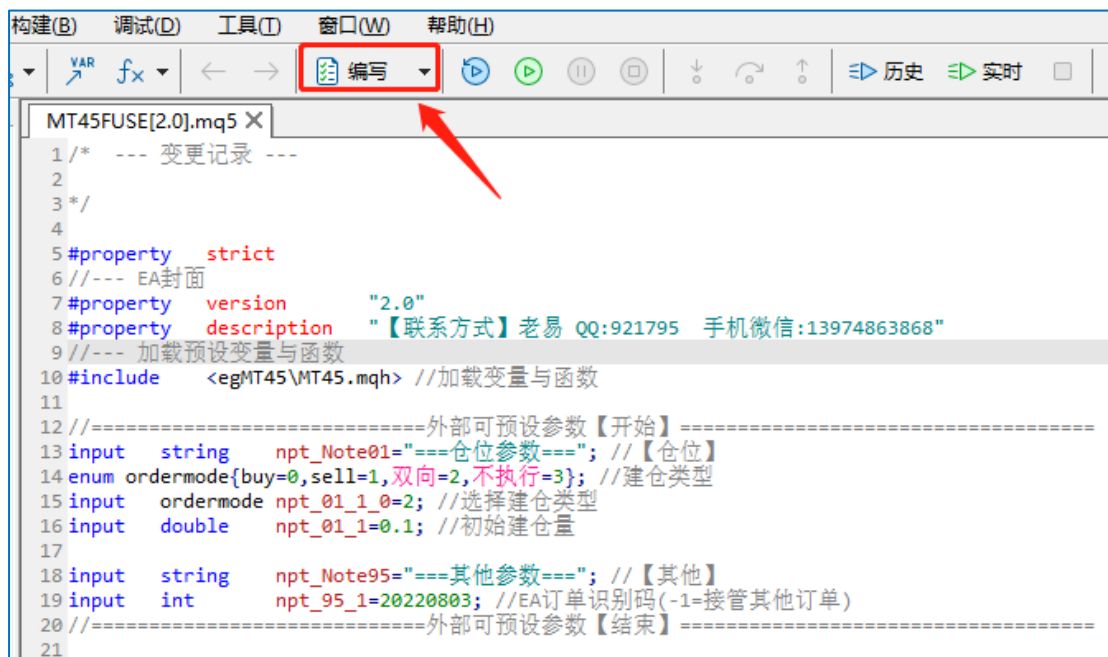
- 打开 MT5 代码编辑器:



- 双击打开 MT4/5FUSE[2.0].mq5 文件



- 点击“编写”按钮，生成可运行程序 MT4/5FUSE[2.0].ex5:

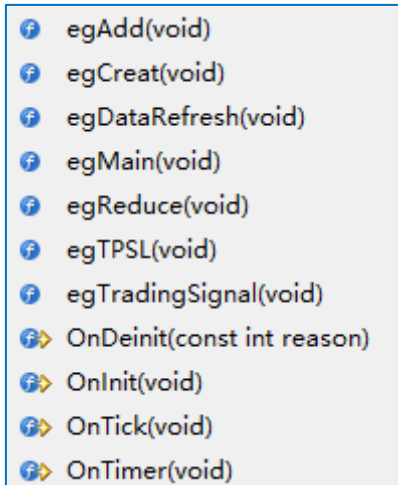


- 加载 EA。在 MT5 终端程序中打开一个图表，在“导航”中的“EA 交易”列表中找到并双击“MT4/5FUSE[2.0]”，这个 EA 就加载到图表中了：



2.4 范例程序

在 egMT4/5 文件夹中的“MT45FUSE[2.0]”是一个基本模版程序，包含了系统默认的 4 个模块：OnDeinit（程序退出处理）、OnInit（加载初始化）、OnTick（每个报价时间处理）、OnTimer（时间周期事件处理）。还包含了 7 个自定义模块：egAdd（加仓）、egCreat（建仓）、egDataRefresh（刷新数据）、egMain（主程序）、egReduce（平仓减仓）、egTPSL（止盈止损）、egTradingSignal（交易信号）。



在“egMT45\MT45 融合范例”文件夹中的“MT45_Demo[2.0]”是一个范例程序，除了基本模版程序有的模块之外，添加了 4 个模块：egReduce_bnt（按钮平仓）、egTOClassify（持仓单分类）、egVerify（授权验证）、OnChartEvent（图表事件处理），同时附带了一个控制面板程序：egUIBoard.mqh。

```

f egAdd(void)
f egCreat(void)
f egDataRefresh(void)
f egMain(void)
f egReduce(void)
f egReduce bnt(void)
f egTOClassify(void)
f egTPSL(void)
f egTradingSignal(string &mySingle[])
f egVerify(void)
f OnChartEvent(const int id,const long &param,...)
f OnDeinit(const int reason)
f OnInit(void)
f OnTick(void)
f OnTimer(void)

```

范例程序加载到图表后会显示一个控制面板：



该程序采用快慢均线计算入场信号：


```

/*
函 数:交易信号
输出参数:信号代码,可能包含多个
算 法:
双均线确定开仓方向:
当前快线在慢线之上,且双线连续3根k线均线上行,发出buy建仓信号。
当前快线在慢线之下,且双线连续3根k线均线下行,发出sell建仓信号。
*/
void egTradingSignal(string &mySingle[])
{
//--- 定义1维数组,输出三个信号,分别是: buy(买入)、sell(卖出)、null(无)。
    ArrayResize(mySingle,1);
    mySingle[0]="null"; //初始化信号代码
//--- 分别计算k1的快慢均线数值
    double myMA1_1=egIdc_MA(SI.symbol,npt_80_1,npt_80_2,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7); //快线,采样数量13
    double myMA1_2=egIdc_MA(SI.symbol,npt_80_1,npt_80_2,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7+1); //快线,采样数量13
    double myMA1_3=egIdc_MA(SI.symbol,npt_80_1,npt_80_2,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7+2); //快线,采样数量13
    double myMA2_1=egIdc_MA(SI.symbol,npt_80_1,npt_80_3,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7); //慢线,采样数量34
    double myMA2_2=egIdc_MA(SI.symbol,npt_80_1,npt_80_3,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7+1); //慢线,采样数量34
    double myMA2_3=egIdc_MA(SI.symbol,npt_80_1,npt_80_3,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7+2); //慢线,采样数量34
//--- 计算建仓信号

```

```

/*
函 数:建仓
输出参数:
算 法:
*/
bool egCreat()
{
//--- buy建仓
    if ( true
        && SignalCode[0]=="buy"
        && TS.buy_orders==0
        && ( false
            || npt_01_1_0==0 //选择建仓类型 buy
            || npt_01_1_0==2 //选择建仓类型 双向
        )
        && egOrderCreat(SI.symbol,0,npt_01_1,MyOrderComment,MagicNumber,0,0)
        && egPrint(PrintToLog,"【建仓】buy单市价建仓")
    )
    {
        return(true);
    }
//--- sell建仓
    if ( true
        && SignalCode[0]=="sell"
        && TS.sell_orders==0
        && ( false
            || npt_01_1_0==1 //选择建仓类型 sell
            || npt_01_1_0==2 //选择建仓类型 双向
        )
        && egOrderCreat(SI.symbol,1,npt_01_1,MyOrderComment,MagicNumber,0,0)
        && egPrint(PrintToLog,"【建仓】sell单市价建仓")
    )
    {
        return(true);
    }
}

```

保持最小间距加仓:


```

/*
函数: 加仓
输出参数:
算法:
*/
bool egAdd()
{
//--- buy加仓
    if ( true
        && npt_01_2>0
        && TS.buy_orders>0 //有buy持仓单
        && SignalCode[0]=="buy" //有信号
        && egOrderInsert(TO,0,SI.ask,npt_01_2*SI.point) //同类单之间保持100点间距
        && egOrdersInShift(TO,SI.symbol,0,0,-1,0,ticket_cur)==0 //当前k线(k0)无buy建仓单
        && egOrdersInShiftHst(HO,SI.symbol,0,0,-1,0,ticket_hst)==0 //当前k线(k0)无buy建仓单
        && egOrderCreat(SI.symbol,0,npt_01_1,MyOrderComment,MagicNumber,0,0) //执行加仓
        && egPrint(PrintToLog,"【加仓】buy单市价加仓")
    )
    {
        return(true);
    }
}

```

根据预设参数执行初始止损和移动止损:

```

/*
函数: 止盈止损
输出参数:
算法:
*/
bool egTPSL()
{
    int i=0;
    int myArraySize=ArraySize(TO);
//--- 初始止盈止损
    for (i=0;i<myArraySize;i++)
    {
        //止盈
        if ( true
            && npt_20_1!=0
            && TO[i].takeprofit==0 //没有止盈价
            && ( false
                //buy单
                || ( true
                    && TO[i].type==0
                    && egOrderTPSL(TO[i].ticket,0,TO[i].openprice+npt_20_1*SI.point) //设置npt_20_1点止盈
                )
                //sell单
                || ( true
                    && TO[i].type==1
                    && egOrderTPSL(TO[i].ticket,0,TO[i].openprice-npt_20_1*SI.point) //设置npt_20_1点止盈
                )
            )
        )
    {
        //止损
        if ( true
            && npt_20_2!=0
            && TO[i].stoploss==0 //没有止损价
            && ( false

```

采用利润保护、反向信号获利、同向对冲、反向对冲方式平仓出场:

```

/*
函数: 平仓减仓
输出参数:
算法:
*/
bool egReduce()
{
    //--- 利润保护
    PP[0].name="当前商品";
    PP[0].current_profit=TS.buy_grp_profit+TS.sell_grp_profit+TS.buy_grp_cost+TS.sell_grp_cost; //当前利润 浮动利润+成本
    PP[0].profit_start=npt_60_1; //启动金额
    PP[0].profit_backrate=npt_60_2; //利润回撤
    if ( true
        {
            egProfitProtect(TO,CloseTicket,PP[0])==true
            {
                egPrint(PrintToLog,"【利润保护】当前商品利润保护达标, 市价平仓")
                egPrint(PrintToLog,"最高利润:"+DoubleToString(PP[0].profit_max,2)+" 当前利润:"+DoubleToString(PP[0].current_profit,2))
            }
        }
        return(true);
    }

    //--- 有反向信号, 所有浮盈单市价平仓
    if ( false
        {
            {
                TS.buy_orders>0 //有buy单
                SignalCode[0]=="sell"
                egOrdersCloseByCondition(TO,CloseTicket,0,0)
                egPrint(PrintToLog,"【信号平仓】有反向信号sell, 当前商品浮盈buy单市价平仓")
            }
            {
                TS.sell_orders>0 //有sell单
                SignalCode[0]=="buy"
            }
        }
    )
}

```

程序关键部位都有详尽的中文注释说明。

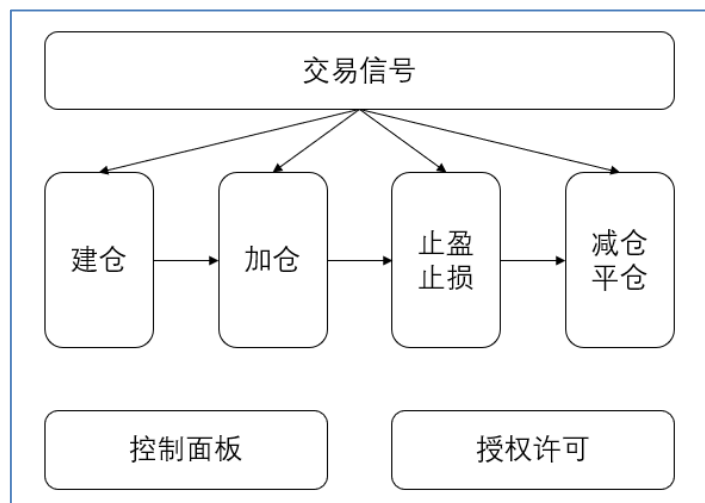
这是一个完整的自动化交易程序，在此基础上加以修改，就可以为自己定制一套 EA。

3 总体设计

3.1 总体框架

交易是一轮接着一轮进行的，一轮交易是指从空仓到有持仓再到空仓的过程。

一轮交易包含了信号、建仓、加仓、止盈止损、减仓平仓五个模块，一套完整的策略框架还包含控制面板、授权许可两个模块，如下图所示。



信号、建仓、加仓、止盈止损、减仓平仓等五个模块为持仓单操作，每项操作都从交易信号中获取许可。

交易信号一般通过技术指标计算而来。值得一提的是，交易信号还要依据持仓单情况，结合保证金占用、浮亏等情况提交操作信号。例如上涨信号比较强，保证金和浮亏都不大的时候，buy 单加仓量可以适当提高，反之就要适当减少。

3.2 模块最小化

任何一个交易策略都可以分解为若干个单一行为模块，从而形成标准函数库。利用功能丰富的函数库可以快速搭建复杂、稳定的交易策略。

例如使用持仓单检索函数命令 `egOrderSearch`，就可以得到如“最早建仓的 buy 单单号”。

再例如使用有效时间段命令 `egTimeValid`，就可以获取“当前是否处在策略允许执行的时间段当中”的值。

这套融合编程系统弥补了 MT4/5 语言体系中的一个重大功能项缺失：持仓单与历史单检索。

3.3 主程序结构

主程序主要有 9 个模块构成：

预定义变量 egGV.mqh	刷新数据 egDataRefresh	交易信号 egTradingSignal
建仓 egCreat	加仓 egAdd	止盈止损 egTPSL
减仓平仓 egReduce	控制面板 egUIBoard	授权许可 egVerify

- 预定义变量 **egGV.mqh**: 独立文件, 定义了包括账户信息、商品信息、持仓单信息、历史单信息等等程序中必须调用的数据类型。
- 刷新数据 **egDataRefresh**: 主程序模块, 实时刷新预定义变量数据和交易信号。
- 交易信号 **egTradingSignal**: 主程序模块, 实时计算信号。
- 建仓 **egCreat**: 主程序模块, 空仓 (含单边空仓) 建仓操作。
- 加仓 **egAdd**: 主程序模块, 有持仓单时加仓操作。
- 止盈止损 **egTPSL**: 主程序模块, 对持仓单执行止盈止损操作。
- 减仓平仓 **egReduce**: 主程序模块, 对持仓单执行平仓减仓操作。
- 控制面板 **egUIBoard**: 独立文件, 在图表中显示控制面板。
- 授权许可 **egVerify**: 主程序模块, 对程序设置授权许可。

3.4 规范编码

编程前必须针对交易策略编写一个需求分析报告, 按照“信号、建仓、加仓、止盈止损、减仓平仓”等五个章节用文字描述出来, 并反复斟酌后定稿。没有经过这个设计流程的, 往往导致最终将程序推翻重来。

MT4/5 模版已经将交易策略分成了 9 个模块, 编码时务必将相关逻辑写在对应模块中。如果在订单操作模块中夹杂信号计算, 会导致逻辑混乱, 难以查找程序问题。

规范代码书写格式, 关键部位要有详细的注释说明, 既有利于日后修改代码时快速理解模块功能, 也方便其他程序员接手进行后期维护。

4 基础设计

为了实现 MT4/5 兼容，为了交易策略顺利执行，需要预定义一些通用变量，需要打造一系列针对持仓单和历史单检索、统计的函数。

4.1 预定义变量

MT4 和 MT5 对于账户信息、商品信息、持仓单信息、历史单信息等内容，预定义方式不同，两种语言差异较大。为了实现语言融合，挑选其共有的常用的变量，外加自定义变量，制订出一套偏重实用性预定义变量方案。

持仓单、历史单统计信息变量的预定义，是为了后续的订单控制做铺垫。这是本套模板的亮点之一，也是最核心的技术，填补了 MT4、MT5 不具备订单检索功能和订单统计功能的空白。

在“\Include\egMT45\egGV.mqh”(General Variable, 通用变量)文件中，用结构体形式，预定义了以下通用变量。

4.1.1 账户信息 AccountInfo(AI)

变量类型	变量名	说明
bool	trade_allowed	是否允许交易
bool	trade_expert	是否允许智能交易
int	trade_mode	账户类型(0=模拟账户,1=比赛账户,2=实盘账户)
int	margin_so_mode	强平模式(0-百分比,1-金额)
long	login	账号
long	leverage	杠杆
long	limit_orders	最大持仓单数量
double	balance	余额
double	credit	信用额度
double	profit	利润
double	equity	净值
double	margin	已用保证金
double	margin_free	可用保证金
double	margin_level	保证金水平
double	margin_so_call	追加保证金水平
double	margin_so_so	保证金强平水平
string	name	账户名称
string	server	服务器名称
string	currency	结算货币
string	company	服务商名称
string	trade_symbols[]	*持仓商品名称列表(数组) 自定义

*列表中“trade_symbols”为自定义数组，这是 MT4 和 MT5 都没有预定义的，但在多货

币应用中又非常有用的变量，MT4/5 模板会将持仓商品名称赋值到该数组中。

4.1.2 商品信息 SymbolInfo(SI)

变量类型	变量名	说明
bool	select	可选商品
bool	spread_float	浮动点差
int	symbols_total	商品总数
int	digits	报价小数位数
int	spread	点差
int	trade_stop_level	停止水平
int	trade_calc_mode	合同价计算模式
int	trade_mode	订单执行类型(0-禁止,1-Buy,2-Sell,3-只允许平仓,4-无限制)
int	trade_freeze_level	冻结水平
int	swap_mode	掉期计算模式
long	swap_rollover3days	掉期执行日(0-周日,1-周一,2-周二,3-周三,4-周四,5-周五,6-周六)
datetime	time	最后报价时间
double	bid	卖出报价
double	ask	买入报价
double	mid	*中间报价=(bid+ask/2) 自定义
double	unit_lots	*单位仓量 自定义
double	unit_high	*单位高度 自定义
double	unit_profit	*单位利润 自定义
double	point	报价点值
double	trade_tick_value	单点价值
double	trade_tick_size	最小变动价位
double	trade_contract_size	合约大小
double	volume_min	最小建仓量
double	volume_max	最大建仓量
double	volum_step	建仓递增量
double	swap_long	Buy 单掉期
double	swap_short	Sell 单掉期
double	margin_initial	初始保证金
double	margin_maintenance	维持保证金
double	margin_required_buy	1 标准手 buy 单保证金
double	margin_required_sell	1 标准手 sell 单保证金
string	currency_base	基本货币名称
string	currency_profit	利润货币名称
string	currency_margin	保证金货币名称
string	descript	商品描述

string	path	商品分类
string	symbol	商品名称

*列表中“mid”为中间报价、“unit_lots、unit_high、unit_profit”为自定义变量，分别是单位仓量、单位高度和单位利润，用于持仓风险计算。这些是 MT4 和 MT5 都没有预定义，但在应用中又非常有用的变量，模板程序会将相关数据赋值到变量中。

列表中“margin_required_buy”和“margin_required_sell”是 1 标准手需要占用的保证金，MT5 作了区分，MT4 没有区分，为了统一标准，按 MT5 规则预定义。

4.1.3 持仓单信息 TradesOrders(TO)

变量类型	变量名	说明
ulong	ticket	订单号
string	symbol	商品名称
int	type	订单类型(0-Buy,1-Sell,2-BuyLimit,3-SellLimit,4-BuyStop,5-SellStop)
datetime	opentime	建仓时间
double	lots	建仓量
double	openprice	建仓价
double	stoploss	止损价
double	takeprofit	止盈价
double	profit	利润
double	commission	佣金
double	swap	掉期
string	comment	注释
int	magicnumber	程序识别码
double	cost	*成本 自定义

*列表中“cost”为每一单的成本，属于自定义变量，是佣金和掉期总和。

4.1.4 持仓单统计信息 TradesStatistical(TS)

变量类型	变量名	说明
int	buy_orders	Buy 单数量总计
int	buylimit_orders	BuyLimit 单数量总计
int	buystop_orders	BuyStop 单数量总计
double	buy_grp_lots	Buy 组成交持仓单持仓量总计
double	buy_grp_profit	Buy 组成交持仓单利润总计
double	buy_grp_avg	Buy 组成交持仓单均价
double	buy_grp_margin	Buy 组成交持仓单保证金占用
double	buy_grp_cost	Buy 组成交持仓单成本(含掉期、手续费、税金等)
int	sell_orders	Sell 单数量总计
int	selllimit_orders	SellLimit 单数量总计
int	sellstop_orders	SellStop 单数量总计

double	sell_grp_lots	Sell 组成成交持仓单持仓量总计
double	sell_grp_profit	Sell 组成成交持仓单利润总计
double	sell_grp_avg	Sell 组成成交持仓单均价
double	sell_grp_margin	Sell 组成成交持仓单保证金占用
double	sell_grp_cost	Sell 组成成交持仓单成本(含掉期、手续费等)
double	max_lots	最大可建仓量
double	max_floating_profit	*最大浮动盈利 自定义
double	max_floating_loss	*最大浮动亏损 自定义

列表中最大浮盈和最大浮亏变量为每次 EA 加载后连续统计数据，EA 重新加载后重新开始刷新。如果需要统计“每一轮交易”的最大浮盈浮亏，需要在主程序中“每一轮开始”环节初始化一次该变量。

4.1.5 历史单信息 HistoryOrders(HO)

变量类型	变量名	说明
ulong	ticket	订单号
string	symbol	商品名称
int	type	订单类型(0-Buy,1-Sell,2-BuyLimit,3-SellLimit,4-BuyStop,5-SellStop)
double	lots	建仓量
datetime	opentime	建仓时间
double	openprice	建仓价
datetime	closetime	平仓时间
double	closeprice	平仓价
double	stoploss	止损价
double	takeprofit	止盈价
double	profit	利润
double	commission	佣金
double	swap	掉期
string	comment	注释
int	magicnumber	程序识别码
double	cost	*成本 自定义

*列表中“cost”为每一单的成本，属于自定义变量，是佣金和掉期总和。

4.1.6 历史单统计信息 HistoryStatistical(HS)

变量类型	变量名	说明
int	buy_orders	Buy 单数量总计
int	buy_limit_orders	BuyLimit 单数量总计
int	buy_stop_orders	BuyStop 单数量总计
int	buy_win_orders	Buy 盈利单数量总计
int	buy_loss_orders	Buy 亏损单数量总计

double	buy_win_profit	Buy 盈利单利润总计
double	buy_loss_profit	Buy 亏损单利润总计
double	buy_profit	Buy 单成交利润总计(含成本)
double	buy_lots	Buy 单成交量总计
int	sell_orders	Sell 单数量总计
int	sell_limit_orders	SellLimit 单数量总计
int	sell_stop_orders	SellStop 单数量总计
int	sell_win_orders	Sell 盈利单数量总计
int	sell_loss_orders	Sell 亏损单数量总计
double	sell_win_profit	Sell 盈利单利润总计
double	sell_loss_profit	Sell 亏损单利润总计
double	sell_profit	Sell 单成交利润总计(含成本)
double	sell_lots	Sell 单成交量总计
double	win_ratio	胜率
double	loss_ratio	败率
double	odds	赔率
double	kelly	凯利

上述变量中 buy_win_profit、buy_loss_profit、sell_win_profit、sell_loss_profit 不包含成本（掉期、手续费等），buy_profit、sell_profit 包含了成本。

4.1.7 利润保护 ProfitProtect(PP)

变量类型	变量名	说明
string	name	名称
double	current_profit	当前利润
double	profit_start	启动利润
double	profit_max	最高利润
double	profit_backrate	利润回撤率
bool	protection_state	利保状态

4.1.8 平仓数组 CloseTicket

变量类型	变量名	说明
int	CloseTicket[]	批量平仓时，将持仓单号放置其中

4.1.9 前一个服务器时间 prv_servvertime

变量类型	变量名	说明
datetime	prv_servvertime	前一个服务器时间，用于检测市场是否休市

4.1.10 图形对象 ObjectAttributeStruct(OAS)

变量类型	变量名	说明
string	name	名称
string	text	文字内容
bool	select	是否点击
bool	ctrl_state	前一个状态(按钮有效)
int	width	宽度
int	height	高度
int	x	锚点 x 坐标
int	y	锚点 y 坐标
int	dx	锚点 x 偏移量
int	dy	锚点 y 偏移量
color	ft_color	前景色
color	bg_color	背景色
color	bd_color	边框色

用于描绘图形界面，包括矩形框、文字框、按钮等图形对象。

4.1.11 常量

变量类型	变量名	说明
const string	ProgramName	无后缀程序名
const string	ProgramPath	程序所在路径

注意：主图显示对象的名称规则中包含了无后缀程序名，由于 MT4/5 对象名称过长会导致对象显示错乱，故程序名不宜过长。

4.2 账户与商品信息

4.2.1 账户信息

MT4/5 编程模版为账户信息预定义了一个结构体类型的变量，选取了常用属性，详情参见[账户信息 AccountInfo\(AI\)](#)。

在主程序头部预定义一个账户信息变量 AI：

```
1. AccountInfo AI; //账户信息变量
```

在主程序 egDataRefresh() 模块中输入以下命令，将相关信息赋值给变量 AI：

```
1. egRefreshAI(AI); //账户信息
```

这条命令的意思是刷新一个名为“AI”（参数 1：AI）的账户信息变量。

参考[刷新账户信息](#)。

4.2.2 商品信息

MT4/5 编程模版为商品信息预定义了一个结构体类型的变量，选取了常用属性，详情参见[商品信息 SymbolInfo\(SI\)](#)。

在主程序头部预定义一个商品信息变量 SI:

```
1. SymbolInfo SI; //商品信息变量
```

在主程序 egDataRefresh() 模块中输入以下命令，将相关信息赋值给变量 SI:

```
1. egRefreshSI(_Symbol,SI); //当前商品信息
```

这条命令的意思是刷新一个名为“SI”（参数 1: SI）的商品信息变量。

参考[刷新商品信息](#)。

4.3 订单统计与检索

制定与执行交易策略，离不开订单统计和检索，本章重点介绍这项基础功能。

交易订单分为两类，一类是正在持仓还没有平仓的订单，以及挂单，叫做“持仓单”，另一类是已经平仓或者取消了的挂单，叫做“历史单”。

无论是 MT4 还是 MT5，内部命令集里均没有对订单的统计和检索函数，不能快速应对稍复杂策略的诉求，例如查找最早一张建仓的持仓单、XAUUSD 的最后一张平仓、buy 单组中浮盈最大的持仓单等等。

持仓单和历史单的属性有区别，如历史单有平仓时间、平仓价，而持仓单没有。

要想对持仓单或者历史单进行统计、检索，必须先打造一个“订单数组”，然后在此基础上进行统计和检索。

订单通常包括 buy、sell、buylimit、selllimit、buystop 和 sellstop 六种类型。在 MT4、MT5 中，六类对应的代码如下：

订单类型	对应代码(int)
buy	0
sell	1
buylimit	2
selllimit	3
buystop	4
sellstop	5

由于 MT4、MT5 预定义常量名称不同，而对应的代码相同。所以，编写程序时，请使用代码，以确保二者在编译时相互兼容。

4.3.1 持仓单

MT4/5 编程模版为持仓单预定义了一个结构体类型的变量，选取常用属性，详情参见[持仓单信息 TradesOrders\(TO\)](#)。

4.3.1.1 订单数组

在主程序头部预定义一个持仓单变量 TO:

```
1. TradesOrders TO[]; //持仓单信息变量
```

在主程序 egDataRefresh()模块中输入以下命令，将生成一个符合条件的持仓单变量数组:

```
1. egRefreshTO(TO,"EURUSD",-1,101); //持仓单信息
```

这条命令的意思是刷新一个名为“TO”（参数 1: TO）的持仓单数组，包括所有符合指定的商品名称（参数 2: EURUSD）、任意类型（参数 3: -1）、指定订单识别码（参数 4:101）条件的持仓单。

参考[刷新持仓单数组](#)。

4.3.1.2 统计

在主程序头部预定义一个持仓单统计变量 TS:

```
1. TradesStatistical TS; //持仓单统计变量
```

在主程序 egDataRefresh()模块中输入以下命令，将生成一个符合条件的持仓单统计变量:

```
1. egRefreshTS(SI,TO,TS); //持仓单统计信息
```

这条命令的意思是刷新一个基于 SI（指定商品）、TO（指定持仓单集合）的名为“TS”的持仓单统计变量。

参考[刷新持仓单统计](#)，[持仓单统计信息 TradesStatistical\(TS\)](#)。

4.3.1.3 检索

在主程序中输入以下命令:

```
1. egOrderSearch(TO,"EURUSD",0,1,123,1);
```

返回符合组合条件的订单号。

查询 TO（参数 1）数组中符合以下条件组合的订单号：商品名称为 EURUSD（参数 2）、按建仓时间排序（参数 3）、持仓单为 Sell 类型（参数 4）、程序识别码为 123（参数 5）、最近建仓（参数 6）。

简单说就是返回最后建仓的那张持仓单单号。

6 个参数任意组合，就能完成复杂条件的订单检索。

参考[持仓单检索](#)。

4.3.2 历史单

MT4/5 编程模版为历史单预定义了一个结构体类型的变量，选取常用属性，详情参见[历史单信息 HistoryOrders\(HO\)](#)。

4.3.2.1 订单数组

在主程序头部预定义一个历史单变量 HO：

```
1. HistoryOrders HO[]; //历史单信息变量
```

在主程序 egDataRefresh() 模块中输入以下命令，将生成一个符合条件的持仓单变量数组：

```
1. egRefreshHO(HO,"EURUSD",-1,101,false,0,0); //历史单信息
```

这条命令的意思是刷新一个名为“HO”（参数 1：HO）的历史单数组，包括所有符合指定的商品名称（参数 2：EURUSD）、任意类型（参数 3：-1）、指定订单识别码（参数 4:101）、按建仓价（参数 5 false）、最早开始（参数 6 0）、到当前时间（参数 7 0）条件的历史单。简单说就是构造一个包括所有历史单的 HO 数组。

参考[刷新历史单数组](#)。

4.3.2.2 统计

在主程序头部预定义一个持仓单统计变量 HS：

```
1. HistoryStatistical TS; //历史单统计变量
```

在主程序 egDataRefresh() 模块中输入以下命令，将生成一个符合条件的持仓单统计变量：

```
1. egRefreshHS(HO,HS); //刷新历史单统计
```

运行结果将生成一个基于 HO（指定历史单集合）的实时更新的 HS 变量。

参考[刷新历史单统计信息](#)，[历史单统计信息 HistoryStatistical\(HS\)](#)。

4.3.2.3 检索

在主程序中输入以下命令：

```
1. egOrderSearchHst(HO,"*",0,-1,-1,-1);
```

返回在 HO（参数 1）数组中符合以下条件组合的订单号：任意商品（参数 2）、任意持仓单类型（参数 4）、所有持仓单（参数 5）、建仓时间（参数 3）、最大的（参数 6）。

简单说就是返回最后建仓的那张持仓单单号。

6 个参数任意组合，就能完成复杂条件的订单检索。

参考[历史单检索](#)。

5 主程序 egMain

MT4/5 主程序模块通常放在 OnTick 模块里，我的习惯是单独打造一个 main 模块，然后将该模块放置在 OnTick 中：

```
1. void OnTick()
2. {
3.     egMain();
4. }
```

以下是主程序逻辑代码：

```
1. /*
2. 函 数:总流程
3. 输出参数:true-可执行, false-不可执行
4. 算 法:
5. */
6. bool egMain()
7. {
8.     ///--- 如果交易停止, 不执行后续动作
9.     if (egTradingStatus()==false) return(false);
10.    ///--- 平仓优先
11.    if (egOrdersCloseByArray(CloseTicket)) return(true); //批量平仓
12.    ///--- 刷新数据
13.    egDataRefresh();
14.    ///--- 控制逻辑
15.    if ( false
16.        || egReduce() //减仓平仓
17.        || egCreat() //建仓
18.        || egAdd() //加仓
19.        || egTPSL() //止盈止损
20.    )
21.    {
22.        return(true);
23.    }
24.    ///---结束
25.    return(true);
26. }
```

主程序在每个报价到达（OnTick）或者时间间隔（OnTimer）成立时，都会执行一次。

所有交易策略都适合这种框架，设计者只需在控制逻辑模块（14 行开始）中组织调试相关交易逻辑即可。

6 刷新数据 egDataRefresh

```

1.  /*
2.  函    数:刷新数据
3.  输出参数:
4.  算    法:刷新所有数据
5.  */
6.  void egDataRefresh()
7.  {
8.  //--- 更新程序识别码
9.      MagicNumber=npt_95_1;
10.
11. //--- 刷新账户与商品信息
12.     egRefreshAI(AI); //账户信息
13.     egRefreshSI(_Symbol,SI); //当前商品信息
14.
15. //--- 刷新持仓单基本信息
16.     egRefreshTO(TO,SI.symbol,-1,MagicNumber); //持仓单信息
17.     egRefreshTS(SI,TO,TS); //持仓单统计信息
18.
19. //--- 刷新历史单基本信息
20.     if (   true
21.         && npt_99_1==true //启用历史数据(false=不启用)
22.         && prvHistoryOrders!=egHstTotal() //历史单数量发生变化才执行一次
23.     )
24.     {
25.         egRefreshHO(HO,_Symbol,-1,MagicNumber,npt_99_2,
26.             StringToTime(npt_99_3),StringToTime(npt_99_4)); //刷新历史单数组
27.         egRefreshHS(HO,HS); //刷新历史单统计
28.         prvHistoryOrders=egHstTotal();
29.     }
30. //--- 刷新信号
31.     egTradingSignal();
32.
33. //--- 结束
34.     return;
35. }

```

主程序中的数据刷新模块负责实时提供持仓单、历史单、交易信号数据。

7 持仓单分类 egTOClassify

实际操盘中，经常需要将持仓单进行分类处理，例如多空单分别控制，再例如多商品分组处理等。

配套的范例程序（MT45_Demo[2.0]）中提供了一个持仓单分类范例，将 TO 中的订单分为 Buy 和 Sell 两个组。

首先在主程序预定义三个数组变量：

```
1.  //--- 订单与环境变量，如果订单需要进行多组分类，则需要定义多个不同名称
2.  AccountInfo          AI;           //账户信息变量
3.  SymbolInfo           SI;           //商品信息变量
4.  TradesOrders         TO[];         //持仓单信息变量
5.  TradesStatistical    TS;           //持仓单统计变量
6.  HistoryOrders        HO[];         //历史单信息变量
7.  HistoryStatistical   HS;           //历史单统计变量
```

然后在 egTOClassify 模块中编写分类代码：

```
1.  /*
2.  函 数:持仓单分类
3.  输出参数:
4.  算 法:
5.  */
6.  void egTOClassify()
7.  {
8.  //--- 初始化持仓单数组
9.      ArrayResize(TO_Buy,0);ArrayResize(TO_Buy,ArraySize(TO));
10.     ArrayResize(TO_Sell,0);ArrayResize(TO_Sell,ArraySize(TO));
11. //--- 开始分类
12.     int i=0,j=0,k=0;
13.     for (i=0;i<ArraySize(TO);++i)
14.     {
15.         if ( true
16.             && TO[i].ticket>0
17.             && TO[i].type==0
18.         )
19.         {
20.             egTOCopy(TO[i],TO_Buy[j]);
21.             j++;
22.         }
23.         if ( true
24.             && TO[i].ticket>0
25.             && TO[i].type==1
```

```
26.         )
27.     {
28.         egTOCopy(TO[i],TO_Sell[k]);
29.         k++;
30.     }
31. }
32. //--- 规范、统计分类组
33.     ArrayResize(TO_Buy,j);
34.     egRefreshTS(SI,TO_Buy,TS_Buy);
35.     ArrayResize(TO_Sell,k);
36.     egRefreshTS(SI,TO_Sell,TS_Sell);
37.     return;
38. }
```

最后在 egDataRefresh 模块中添加一条命令：

```
1. //--- 持仓单分类
2.     egTOClassify();
```

处理后，在订单控制逻辑中可以随时调取 TO_Buy、TS_Buy、TO_Sell、TS_Sell 各种数据。

8 交易信号 egTradingSignal

交易信号通常都是从技术指标计算而来的，MT4 和 MT5 获取技术指标数据的命令完全不同，以均线为例：

MT4 命令：

```
1. double   iMA(string      symbol,          // 交易品种名称
2.          int      timeframe,          // 周期
3.          int      ma_period,          // 平均周期
4.          int      ma_shift,          // 平移
5.          int      ma_method,          // 平滑类型
6.          int      applied_price,      // 价格或者处理程序类型
7.          int      shift              // k 线序号
8.          );
```

MT5 命令：

```
1. int   iMA(string      symbol,          // 交易品种名称
2.      ENUM_TIMEFRAMES  period,          // 周期
3.      int      ma_period,          // 平均周期
4.      int      ma_shift,          // 平移
5.      ENUM_MA_METHOD   ma_method,      // 平滑类型
6.      ENUM_APPLIED_PRICE applied_price  // 价格或者处理程序类型
7.      );
```

MT4 命令获取指定 k 线（shift）序列的 MA 值，MT5 命令将所有 k 线的 MA 值赋值到指定数组中。

有些指标有多个输出项，例如布林线（Bands）有上线、中线、下线三个输出项。还有来自第三方的技术指标，输入和输出参数并不确定。

为了实现 MT4/5 技术指标的融合，只能将 MT4/5 参数相同的技术指标实现兼容，而且要自定义一系列指标，工作量十分大。本模版目前完成了 MT4/5 共有的默认指标融合，以后逐步完善，补充到文档和代码中。完善的代码将随时共享在 QQ 的互助学习群中。

值得提醒大家注意的是，MT4/5 获取 k 线数据的方式有所不同。MT4 中可以用 `Open[0]` 获取当前 k 线开盘价，而 MT5 没有这样的命令。但 MT4/5 都可以用 `iOpen(NULL,PERIOD_CURRENT,0)` 来获取当前 k 线开盘价，所以在编写程序时，请务必使用 MT4/5 通用的内部命令，不要使用 MT4 特有的等命令。

参考[技术指标](#)。

【范例】

算法描述：利用双均线（快慢线）相对位置计算交易信号。快线采样数量为 13，慢线采样数量为 34，如果 k1 快线在 k1 慢线之上，发出 buy 建仓信号，反之发出 sell 建仓信号。

- 在主程序头部预定义信号变量。

```
1. string      SignalCode[]; //信号代码数组变量,有几类信号就定义几维,最少 1 维
```

这里定义 1 维数组,输出三个信号,分别是: buy (买入)、sell (卖出)、null (无)。也可以不用预定义, SignalCode 维度将在信号模块 (egTradingSignal) 模块中给出定义。

- 在主程序[刷新数据 egDataRefresh](#)模块中输入刷新信号代码。

```
1. //--- 刷新信号
2. egTradingSignal(SignalCode);
```

每一次价格变化都会执行一次信号计算模块,确保 SignalCode 变量得到实时更新。

- 在主程序 egTradingSignal 信号函数模块中编写相关代码。

注意: 所有操盘信号都要写在这个模块中,避免日后调整逻辑的时候发生混乱。

```
1. /*
2. 函 数:交易信号
3. 输出参数:信号代码,可能包含多个
4. 算 法:
5. 双均线确定开仓方向:
6. 当前快线在慢线之上,且双线连续 3 根 k 线均线上行,发出 buy 建仓信号。
7. 当前快线在慢线之下,且双线连续 3 根 k 线均线下行,发出 sell 建仓信号。
8. */
9. void egTradingSignal(string &mySingle[])
10. {
11. //--- 定义 1 维数组,输出三个信号,分别是: buy (买入)、sell (卖出)、null
    (无)。
12.     ArrayResize(mySingle,1);
13.     mySingle[0]="null"; //初始化信号代码
14. //--- 分别计算 k1 的快慢均线数值
15.     double myMA1_1=egIdc_MA(SI.symbol,npt_80_1,npt_80_2,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7); //快线,采样数量 13
16.     double myMA1_2=egIdc_MA(SI.symbol,npt_80_1,npt_80_2,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7+1); //快线,采样数量 13
17.     double myMA1_3=egIdc_MA(SI.symbol,npt_80_1,npt_80_2,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7+2); //快线,采样数量 13
18.     double myMA2_1=egIdc_MA(SI.symbol,npt_80_1,npt_80_3,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7); //慢线,采样数量 34
19.     double myMA2_2=egIdc_MA(SI.symbol,npt_80_1,npt_80_3,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7+1); //慢线,采样数量 34
20.     double myMA2_3=egIdc_MA(SI.symbol,npt_80_1,npt_80_3,npt_80_4,npt_80_5,npt_80_6,0,npt_80_7+2); //慢线,采样数量 34
21. //--- 计算建仓信号
22.     if ( true
23.         //快线在慢线之上
24.         && myMA1_1>myMA2_1
25.         //快线上行
```

```

26.      && myMA1_1>myMA1_2
27.      && myMA1_2>myMA1_3
28.      //慢线上行
29.      && myMA2_1>myMA2_2
30.      && myMA2_2>myMA2_3
31.    )
32.    {
33.      mySingle[0]="buy";
34.    }
35.    if (   true
36.      //快线在慢线之下
37.      && myMA1_1<myMA2_1
38.      //快线下行
39.      && myMA1_1<myMA1_2
40.      && myMA1_2<myMA1_3
41.      //慢线下行
42.      && myMA2_1<myMA2_2
43.      && myMA2_2<myMA2_3
44.    )
45.    {
46.      mySingle[0]="sell";
47.    }
48.    //---结束
49.    return;
50. }

```

信号维度可以针对建仓、加仓、平仓等操盘行为分别定义，以便精准提交信号。在多种信号并存的时候，需要注意的是不能出现类似 buy 建仓信号和 buy 平仓信号同时存在的逻辑冲突。

以上逻辑代码在配套的 MT45_Demo[2.0]中。

9 建仓 egCreat

交易策略中，第一单入场非常重要，这张入场单将成为之后所有操盘行为的锚定单。由于大部分交易允许多空单同时持仓，因此要注意区分对待多空持仓类型。

如果每轮交易只允许单边持仓，那么建仓的条件就是“指定商品空仓”。

如果每轮交易允许双向持仓，那么建仓条件就是“指定商品单边空仓”。

建仓单可以是 Buy、Sell、BuyLimit、SellLimit、BuyStop、SellStop 中的任意一种类型。

在主程序 egCreat() 模块中编写以下命令完成建仓任务。

```

1.  /*
2.  函    数: 建仓
3.  输出参数:
4.  算    法:
5.  */
6.  bool egCreat()
7.  {
8.  //--- buy 建仓
9.      if (   true
10.         && SignalCode[0]=="buy"
11.         && TS.buy_orders==0
12.         && (   false
13.             || npt_01_1_0==0 //选择建仓类型 buy
14.             || npt_01_1_0==2 //选择建仓类型 双向
15.             )
16.         && egOrderCreat(SI.symbol,0,npt_01_1,MyOrderComment,MagicNumber,0,0)
17.         && egPrint(PrintToLog,"【建仓】buy 单市价建仓")
18.         )
19.      {
20.          return(true);
21.      }
22. //--- sell 建仓
23.      if (   true
24.         && SignalCode[0]=="sell"
25.         && TS.sell_orders==0
26.         && (   false
27.             || npt_01_1_0==1 //选择建仓类型 sell
28.             || npt_01_1_0==2 //选择建仓类型 双向
29.             )
30.         && egOrderCreat(SI.symbol,1,npt_01_1,MyOrderComment,MagicNumber,0,0)
31.         && egPrint(PrintToLog,"【建仓】sell 单市价建仓")
32.         )

```

```
33.    {  
34.        return(true);  
35.    }  
36. }
```

在配套的 MT45_Demo[2.0]中还有创建 buylimit、selllimit、buystop、sellstop 的范例代码。

参考[建仓 egOrderCreat](#)。

10 加仓 egAdd

加仓与建仓概念不同，建仓的前提是没有持仓单或者单边没有持仓单，加仓的前提是有持仓单或者单边有持仓单。

加仓是调度仓位实现风险控制的方法之一。先约定 3 个加仓类型名词，如图所示：



根据当前报价所在位置，所有持仓单都浮盈时的加仓叫做“正向加仓”。所有持仓单中有浮盈单和浮亏单时的加仓叫做“插入加仓”。所有持仓单都浮亏时的加仓叫做“反向加仓”。

加仓不能随意，必须有以下条件之一。

- 同类单之间保持间距。
- 当前 k 线 (k0) 禁止二次建仓。
- 当前 k 线 (k0) 有同类单平仓的，禁止加仓。
- 有加仓信号。

在主程序 egAdd() 模块中输入以下代码，共有 5 个条件同时满足后执行 buy 单加仓。

```

1.  /*
2.  函 数: 加仓
3.  输出参数:
4.  算 法:
5.  */
6.  bool egAdd()
7.  {
8.  //--- buy 加仓
9.      if ( true
10.         && npt_01_2>0
11.         && TS.buy_orders>0 //有 buy 持仓单
12.         && SignalCode[0]=="buy" //有信号
13.         && egOrderInsert(T0,0,SI.ask,npt_01_2*SI.point) //同类单之间保持 100 点
           间距
14.         && egOrdersInShift(T0,SI.symbol,0,0,-1,0,ticket_cur)==0 //当前 k 线
           (k0) 无 buy 建仓单
15.         && egOrdersInShiftHst(H0,SI.symbol,0,0,-1,0,ticket_hst)==0 //当前 k 线
           (k0) 无 buy 建仓单

```



```

16.      && egOrderCreat(SI.symbol,0,npt_01_1,MyOrderComment,MagicNumber,0,0)
      //执行加仓
17.      && egPrint(PrintToLog,"【加仓】buy 单市价加仓")
18.      )
19.      {
20.          return(true);
21.      }
22. //--- sell 加仓
23.      if (      true
24.          && npt_01_2>0
25.          && TS.sell_orders>0 //有 sell 持仓单
26.          && SignalCode[0]=="sell" //有信号
27.          && egOrderInsert(T0,1,SI.bid,npt_01_2*SI.point) //同类单之间保持 100 点
      间距
28.          && egOrdersInShift(T0,SI.symbol,0,1,-1,0,ticket_cur)==0 //当前 k 线
      (k0) 无 sell 建仓单
29.          && egOrdersInShiftHst(H0,SI.symbol,0,1,-1,0,ticket_hst)==0 //当前 k 线
      (k0) 无 sell 建仓单
30.          && egOrderCreat(SI.symbol,1,npt_01_1,MyOrderComment,MagicNumber,0,0)
      //执行加仓
31.          && egPrint(PrintToLog,"【加仓】sell 单市价加仓")
32.      )
33.      {
34.          return(true);
35.      }
36. //---结束
37.      return(false);
38. }

```

注意:如果要使用历史平仓单作为建仓、加仓条件,必须先在主程序中启用历史单数组。

```

1. input  string  npt_Note99=="==历史单参数=="; //【历史单】
2. input  bool    npt_99_1=true; //启用历史数据(false=不启用)
3. input  bool    npt_99_2=true; //选用建仓价(true=平仓价)
4. input  string  npt_99_3="2019.8.8 00:00:00"; //截取历史数据开始时间(空值=最
      早)
5. input  string  npt_99_4=""; //截取历史数据结束时间(空值=当前时间)

```

正向加仓的判断条件为指定类型持仓单组没有浮亏单,例如:

```
1. if (egOrderSearch(T0,SI.symbol,3,0,-1,1)==0) //buy 组没有浮亏单
```

反向加仓的判断条件为指定类型持仓单组没有浮盈单,例如:

```
1. if (egOrderSearch(T0,SI.symbol,2,1,-1,1)==0) //sell 组没有浮盈单
```

插入加仓的判断条件为指定类型持仓单组既有浮盈单也有浮亏单，例如：

```
1. if ( true
2.     && egOrderSearch(T0,SI.symbol,2,1,-1,1)>0) //sell 组有浮盈单
3.     && egOrderSearch(T0,SI.symbol,3,1,-1,1)>0) //sell 组有浮亏单
4. )
```

参见配套的 MT45_Demo[2.0]。

参考[同类单间隔](#)、[指定 k 线持仓单数量](#)、[指定 k 线历史单平仓数量](#)。

11 止盈止损 egTPSL

给持仓单设置止盈止损价，可以确保行情波动过程中，报价一旦触碰止盈止损价，会自动平仓。

止损方式有两种，分别是初始止损和移动止损，其中移动止损又分为点位移动和阶梯移动两种。

初始止损是指持仓单入场后，设置一定间距的认亏价，报价一旦触碰，持仓单自动认亏平仓。

移动止损是指报价向持仓单有利方向运行（对于 buy 单，报价越来越高；对于 sell 单，报价越来越低），当浮盈达到预设值后，开始朝着有利方向保持一定间距修改止损价。

点位移动止损是指报价向有利方向每移动一次，止损价跟着修改一次，操作频率高。

阶梯移动止损是指报价向有利方向每移动一个固定间隔，止损价跟着修改一次，操作频率低。

在主程序 egTPSL() 模块中输入以下代码，实现持仓单初始止损和点位移动止损。

11.1 设置初始止盈止损

```

1.  //--- 初始止盈止损
2.  for (i=0;i<myArraySize;i++)
3.  {
4.      //止盈
5.      if ( true
6.          && npt_20_1!=0
7.          && T0[i].takeprofit==0 //没有止盈价
8.          && ( false
9.              //buy 单
10.             || ( true
11.                 && T0[i].type==0
12.                 && egOrderTPSL(T0[i].ticket,0,T0[i].openprice+npt_20_1*S
13.                 I.point) //设置 npt_20_1 点止盈
14.             )
15.             //sell 单
16.             || ( true
17.                 && T0[i].type==1
18.                 && egOrderTPSL(T0[i].ticket,0,T0[i].openprice-
19.                 npt_20_1*SI.point) //设置 npt_20_1 点止盈
20.             )
21.         )
22.     }
23.     //止损
24.     if ( true

```

```

24.         && npt_20_2!=0
25.         && T0[i].stoploss==0 //没有止损价
26.         && ( false
27.             //buy 单
28.             || ( true
29.                 && T0[i].type==0
30.                 && egOrderTPSL(T0[i].ticket,1,T0[i].openprice-
npt_20_2*SI.point) //设置 npt_20_2 点止损
31.             )
32.             //sell 单
33.             || ( true
34.                 && T0[i].type==1
35.                 && egOrderTPSL(T0[i].ticket,1,T0[i].openprice+npt_20_2*S
I.point) //设置 npt_20_2 点止损
36.             )
37.         )
38.     )
39. }
40. }

```

11.2 设置移动止损

```

1. //--- 点位移动止损
2.     double ts_start=NormalizeDouble(npt_20_3*SI.point,SI.digits);//浮盈间距达
到 npt_20_3 点以上，启动移损
3.     double ts_intvl=NormalizeDouble(npt_20_4*SI.point,SI.digits);//止损价保持
npt_20_4 点间距
4.     for (i=0;i<myArraySize;i++)
5.     {
6.         if ( false
7.             //buy 单
8.             || ( true
9.                 && npt_20_3>0
10.                && npt_20_4>0
11.                && T0[i].type==0
12.                && SI.bid-T0[i].openprice>ts_start //浮盈达标
13.                && ( false
14.                    || T0[i].stoploss==0 //无止损价
15.                    || NormalizeDouble(SI.bid-
ts_intvl,SI.digits)>T0[i].stoploss //有止损价
16.                )
17.                && egOrderTPSL(T0[i].ticket,1,NormalizeDouble(SI.bid-
ts_intvl,SI.digits)) //设置 npt_20_4 点止损
18.            )

```

```

19.          //sell 单
20.          || (   true
21.              && npt_20_3>0
22.              && npt_20_4>0
23.              && T0[i].type==1
24.              && T0[i].openprice-SI.ask>ts_start //浮盈达标
25.              && (   false
26.                  || T0[i].stoploss==0 //无止损价
27.                  || NormalizeDouble(SI.ask+ts_intv1,SI.digits)<T0[i].stop
    loss //有止损价
28.              )
29.              && egOrderTPSL(T0[i].ticket,1,NormalizeDouble(SI.ask+ts_intv
    1,SI.digits)) //设置 npt_20_4 点止损
30.          )
31.      )
32.  {}
33.  }

```

阶梯止损代码略。

注意：MT4 和 MT5 系统计算价格时，小数点通常有 8 位，为了使程序逻辑正常运行，需要用系统命令 `NormalizeDouble`，将精度调整到与报价精度一致。

参见配套的 MT45_Demo[2.0]。

参考[持仓单设置止盈止损](#)。

12 减仓平仓 egReduce

减仓是调度仓位实现风险控制的方法之一，这一章介绍的平仓函数涵盖了目前已知的所有平仓方法，请认真消化，必定受益匪浅。

所有与减仓平仓的代码都必须写在主程序的 egReduce 模块当中。

12.1 指定单号平仓

分为整单平仓和部分仓位平仓。例如：

```
1. egOrderClose(123456,0); //订单号为 123456 市价平仓，如果是挂单，则移除。
2. egOrderClose(123456,0.01); //订单号为 123456 市价减仓 0.01 手
```

参见[订单平仓 egOrderClose](#)。

12.2 数组平仓

平仓是交易策略中优先级别最高的一个环节。

从终端向交易服务器发送平仓指令、获取服务器返回的平仓结果存在延时，预定的平仓任务没有完成之前，必须禁止其他的订单操作，因此需要采取数组平仓的办法来解决。

MT4/5 模版预定义了一个[平仓数组 CloseTicket](#)，所有需要平仓的持仓单号都存放在这个数组中，本条命令对数组中的持仓单执行平仓。

这是一条针对批量平仓的基础命令。

点击配套的 MT45_Demo[2.0]程序呈现的控制面板中的“一键清仓”、“Buy 组平仓”、“Sell 组平仓”按钮会将相关的单号赋值到 CloseTicket 数组中，再由数组平仓命令统一执行。

所有属于整单平仓的逻辑都采取“将相关的单号赋值到 CloseTicket 数组中，再由数组平仓命令统一执行”的方案。

参见[数组平仓 egOrdersCloseByArray](#)。

12.3 条件平仓

按组合条件平仓。组合条件分为持仓单类型（Buy、Sell、BuyLimit、SellLimit、BuyStop、SellStop 以及所有类型等 7 种）和平仓模式（浮赢、浮亏以及任意等 3 种）两大类型，子项可以进行任意匹配。例如以下命令表示将 TO 持仓单数组中所有浮盈的 buy 单执行平仓：

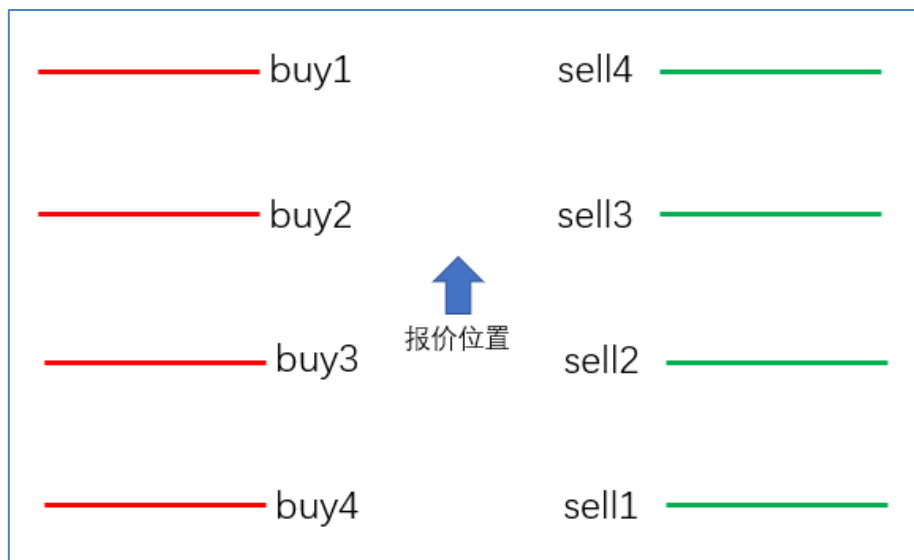
```
1. egOrdersCloseByCondition(TO,CloseTicket,0,0);
```

参见[条件平仓 egOrdersCloseByCondition](#)。

12.4 同向对冲

在包含了浮盈和浮亏的 buy 单组中，如果浮盈单浮盈金额大于等于从最远浮亏单开始累计的浮亏金额，那么所有浮盈单加上最远浮亏单可以平仓，这叫做同向对冲。

同向对冲的目的是为了减少持仓量，避免未来市场出现不可控风险。



上图中 buy4、buy3 和 buy1 可以执行同向对冲，sell4、sell3 和 sell1 可以执行同向对冲。

```

1.  //--- 同向对冲
2.      if (   true
3.          && TS.buy_orders>1 //有 2 张以上 buy 单
4.          && egSDHR(T0,0,CloseTicket,100*SI.point,1.1)==true
5.          && egPrint(true,"【同向对冲】buy 单组执行同向对冲")
6.      )
7.      {
8.          return(true);
9.      }

```

这段代码表示当最低价 buy 单浮盈 100 点时，开始执行 buy 单组同向对冲。

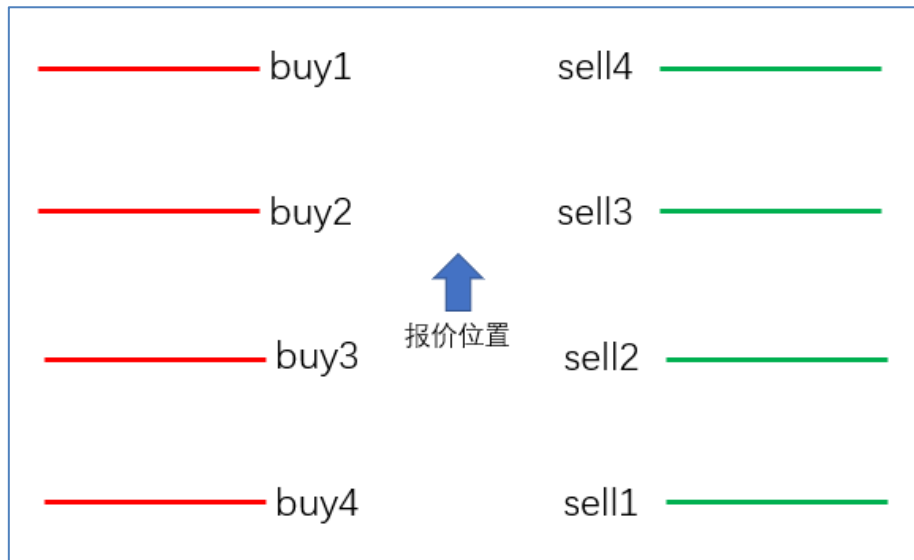
实际应用中，可以设置各种前置条件来执行不同需求的同向对冲。

参见[同向对冲 egSDHR](#)。

12.5 反向对冲

双向持仓，buy 组有浮盈单，sell 组有浮亏单，如果 buy 组浮盈单浮盈金额大于等于 sell 组从最远浮亏单开始累计的浮亏金额，那么所有浮盈单加上最远浮亏单可以平仓，这叫做反向对冲。

反向对冲的目的是为了减少持仓量，避免未来市场出现不可控风险。



上图中 buy4、buy3 和 sell1 可以执行反向对冲，sell4、sell3 和 buy1 可以执行反向对冲。

```

1.  //--- 反向对冲
2.      if (   true
3.          && TS.buy_orders>1 //有 2 张以上 buy 单
4.          && egODHR(T0,1,CloseTicket,100*SI.point,1.1)==true
5.          && egPrint(true,"【反向对冲】浮亏 sell 单组执行反向对冲")
6.      )
7.      {
8.          return(true);
9.      }

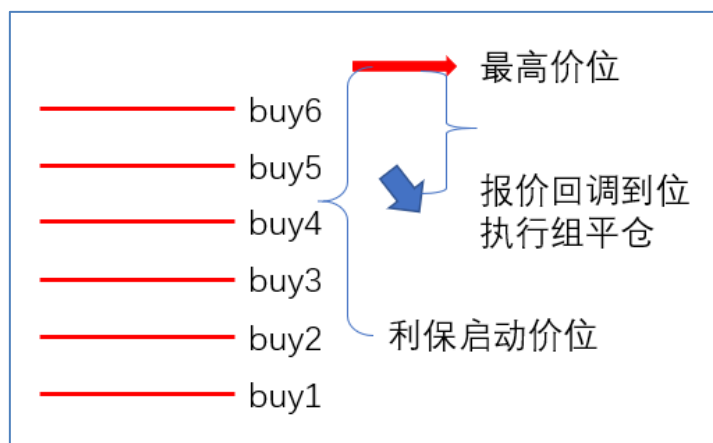
```

这段代码表示当最低价 buy 单浮盈 100 点时，开始执行 sell 单组浮亏单反向对冲。实际应用中，可以设置各种前置条件来执行不同需求的同向对冲。

参见[反向对冲 egODHR](#)。

12.6 利润保护

监控一组浮盈持仓单，实时刷新最高浮盈金额，如果当前浮盈金额相比最高浮盈回撤了一定比例，该组持仓单全体市价平仓。这叫做利润保护。



上图为一个浮盈的 buy 单组，当浮盈达到预设金额时，启动利润保护规则，在报价上升过程中，实时刷新最高浮盈金额，并将其与当前浮盈金额对比，随着报价回调，浮盈利润出现回撤，当回撤到预设比例后，执行组平仓。

这个方法只与目标持仓单组利润关联，不涉及任何技术指标，目标就是为了入袋为安。

```

1.  //--- 利润保护
2.      //当前货币
3.      PP[0].name="当前货币";
4.      PP[0].current_profit=TS.buy_grp_profit+TS.sell_grp_profit+TS.buy_grp_cost
      +TS.sell_grp_cost; //当前利润 浮动利润+成本
5.      PP[0].ProfitStart=20; //启动金额
6.      PP[0].ProfitBackRate=0.3; //利润回撤
7.      if ( egProfitProtect(TO,CloseTicket,PP[0])==true
8.          && egPrint(PrintOut,"【利润保护】当前商品利润保护达标，市价平仓")
9.          && egPrint(PrintOut,"最高利润:"+
      DoubleToString(PP[0].ProfitMax,2)+" 当前利润:"+
      DoubleToString(PP[0].current_profit,2))
10.         )
11.     {
12.         return(true);
13.     }

```

这段代码表示当前货币所有持仓单 TO 持仓单组浮盈达到 20 元启动利润保护，当 TO 组从最高利润回撤 30%时，执行全体平仓。

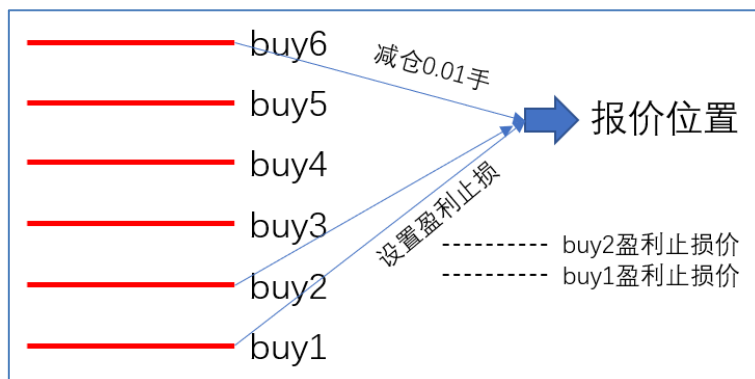
参见[利润保护](#)。

12.7 盈损对冲

消除最近浮亏单始终都是仓位调度的第一目标。虽然同向对冲和反向对冲可以实现减仓，但是这两种方法都是针对整单操作的。在实际操盘过程中，若浮亏单距离现价很远，则很难实现对冲，将导致浮亏越来越大。

如果将最近浮亏单按最小建仓量的方式执行减仓对冲，难度就会减少，0.01 手浮亏仅仅

是 0.1 手的十分之一。在双向持仓时，为了迎合一个单边行情，扩大浮盈额度，浮盈单采取盈利止损的办法，而不是平仓，这叫做盈损对冲。



如上图，有一组 buy 单，行情到达报价位置，buy6 单减仓 0.01 手的同时 buy1 和 buy2 两张单执行盈利止损。如果行情继续上行，buy6 单会继续减仓，buy1 和 buy2 单止损价会上移，buy3 单会加入进来，加快最远浮亏单的减仓速度。如果行情下行，触碰止损线，持仓单平仓，结果还是无损平仓。

盈损对冲既可以在单一品种也可以在多品种之间进行，如果执行多品种盈损对冲，关键是预设持仓单数组（TO）要将目标品种填写进去。

```
1. egTSHR(TS,TO,0.001,0.003,1.1,0.001,0,1);
```

这段代码的意思是：TO 数组中的 buy 单组，有浮赢浮亏单，当最低价浮盈单达到 100 点（参数 3，0.001 元，下同），预计盈利单止损价距离现价 300 点（参数 4），对冲系数 1.1（参数 5），最高价 buy 单达到浮亏 100 点（参数 6），按照最小建仓量 0.01 手（参数 7）时，开始执行浮亏 buy 单减仓，浮盈 buy 单设置盈利止损操作。

参见[盈损对冲 egTSHR](#)。

12.8 减仓平仓范例

配套的 MT45_Demo[2.0]程序代码：

```
1. /*
2. 函 数:平仓减仓
3. 输出参数:
4. 算 法:
5. */
6. bool egReduce()
7. {
8. 利润保护
9. PP[0].name="当前商品";
10. PP[0].current_profit=TS.buy_grp_profit+TS.sell_grp_profit+TS.buy_grp_cost+TS.sell_grp_cost; //当前利润 浮动利润+成本
```

```

11.    PP[0].profit_start=npt_60_1; //启动金额
12.    PP[0].profit_backrate=npt_60_2; //利润回撤
13.    if (    true
14.        && egProfitProtect(T0,CloseTicket,PP[0])==true
15.        && egPrint(PrintToLog,"【利润保护】当前商品利润保护达标，市价平仓")
16.        && egPrint(PrintToLog,"最高利
    润:"+DoubleToString(PP[0].profit_max,2)+" 当前利
    润:"+DoubleToString(PP[0].current_profit,2))
17.    )
18.    {
19.        return(true);
20.    }
21.
22. //--- 有反向信号，所有浮盈单市价平仓
23.    if (    false
24.        || (    true
25.            && TS.buy_orders>0 //有 buy 单
26.            && SignalCode[0]=="sell"
27.            && egOrdersCloseByCondition(T0,CloseTicket,0,0)
28.            && egPrint(PrintToLog,"【信号平仓】有反向信号 sell，当前商品浮盈 buy
    单市价平仓")
29.        )
30.        || (    true
31.            && TS.sell_orders>0 //有 sell 单
32.            && SignalCode[0]=="buy"
33.            && egOrdersCloseByCondition(T0,CloseTicket,1,0)
34.            && egPrint(PrintToLog,"【信号平仓】有反向信号 buy，当前商品浮盈 sell
    单市价平仓")
35.        )
36.    )
37.    {
38.        return(true);
39.    }
40.
41. //--- 同向对冲
42.    //buy 组
43.    if (    true
44.        && (    false
45.            || npt_40_1_1==0 //选择同向对冲模式 buy
46.            || npt_40_1_1==2 //选择同向对冲模式 双向
47.        )
48.        && TS.buy_orders>1 //有 2 张以上 buy 单
49.        && egSDHR(T0,0,CloseTicket,npt_40_1_2*SI.point,npt_40_1_3)==true
50.        && egPrint(PrintToLog,"【同向对冲】buy 单组执行同向对冲")

```

```

51.     )
52.     {
53.         return(true);
54.     }
55.     //sell 组
56.     if ( true
57.         && ( false
58.             || npt_40_1_1==1 //选择同向对冲模式 sell
59.             || npt_40_1_1==2 //选择同向对冲模式 双向
60.         )
61.         && TS.sell_orders>1 //有 2 张以上 sell 单
62.         && egSDHR(T0,1,CloseTicket,npt_40_1_2*SI.point,npt_40_1_3)==true
63.         && egPrint(PrintToLog,"【同向对冲】sell 单组执行同向对冲")
64.     )
65.     {
66.         return(true);
67.     }
68.
69. //--- 反向对冲
70. //buy 组
71. if ( true
72.     && ( false
73.         || npt_40_2_1==0 //选择同向对冲模式 buy
74.         || npt_40_2_1==2 //选择同向对冲模式 双向
75.     )
76.     && TS.sell_orders>0 //有 1 张以上 sell 单
77.     && egODHR(T0,0,CloseTicket,npt_40_2_2*SI.point,npt_40_2_3)==true
78.     && egPrint(PrintToLog,"【反向对冲】浮亏 buy 单组执行反向对冲")
79. )
80. {
81.     return(true);
82. }
83. //sell 组
84. if ( true
85.     && ( false
86.         || npt_40_2_1==1 //选择同向对冲模式 sell
87.         || npt_40_2_1==2 //选择同向对冲模式 双向
88.     )
89.     && TS.buy_orders>0 //有 1 张以上 buy 单
90.     && egODHR(T0,1,CloseTicket,npt_40_2_2*SI.point,npt_40_2_3)==true
91.     && egPrint(PrintToLog,"【反向对冲】浮亏 sell 单组执行反向对冲")
92. )
93. {
94.     return(true);

```

```
95.     }  
96. //--- 结束  
97.     return(false);  
98. }
```

13 EA 授权 egVerify

配套的 MT45_Demo[2.0]呈现了这个模块。

EA 授权共有 4 种方式，包括使用期限、交易服务器、账户余额、账号类型（又分为 4 种方式实盘账号、模拟账号、指定账号、任意账号）等。这 4 种方式重叠使用可以实现多条件授权。例如可以设置指定交易账号“123456”的使用到期日。

在主程序头部输入以下代码，预定义授权变量：

```
1.  //--- 主程序授权信息，空信息表示无限制
2.      //使用期限 格式：“2028.1.1 00:00”，空值=不限制，以服务器时间为准
3.      string main_auz_date="";
4.      //授权服务器 范例：“MetaQuotes-Demo” 空值=不限制
5.      string main_auz_sever="";
6.      //账户余额 范例：“10000.00”，空值=不限制
7.      string main_auz_balance="";
8.      //账号类型（分为 4 种方式实盘账号、模拟账号、指定账号、任意账号）
9.      const string main_auz_real="20101221"; //所有实盘账户 可自定义
10.     const string main_auz_demo="19970409"; //所有模拟账户 可自定义
11.     string main_auz_account=""; //制定账户 空值=任意账号，main_auz_real=所有实
    盘账户，main_auz_demo=所有模拟账户
```

主程序中的 egVerify()为授权模块，返回 0 表示有效，其他返回值表示无效：

1-帐号无效，2-期限已过，3=余额超标，4=未授权服务器。

```
1.  /*
2.      函    数:授权验证
3.      输出参数:
4.      0-合法, 1-帐号无效, 2-期限已过, 3=余额超标, 4=未授权服务器
5.      算    法:
6.      */
7.  int egVerify()
8.  {
9.      //使用期限 格式：“2028.1.1 00:00”，空值=不限制，以服务器时间为准
10.     if ( main_auz_date!="")
11.         && StringToTime(main_auz_date)<TimeCurrent()
12.         && egPrint(PrintToLog,"本 EA 有效期已过")
13.     )
14.     {
15.         return(2);
16.     }
17.     //授权服务器 范例：“MetaQuotes-Demo” 空值=不限制
18.     if ( main_auz_sever!="")
```

```

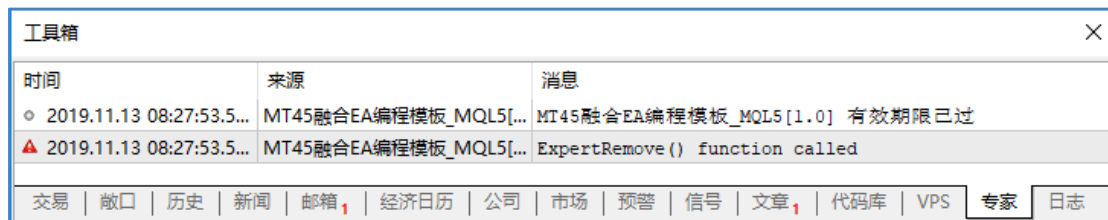
19.      && main_auz_sever!=AI.server
20.      && egPrint(PrintToLog,"本 EA 仅限指定平台使用")
21.    )
22.  {
23.      return(4);
24.  }
25. //账户余额 范例: "10000.00", 空值=不限制
26.  if (   main_auz_balance!="
27.      && (double)main_auz_balance>AI.balance
28.      && egPrint(PrintToLog,"账户余额超过本 EA 有效管控范围")
29.    )
30.  {
31.      return(3);
32.  }
33. //账号类型 (分为 4 种方式: 实盘账号、模拟账号、指定账号、任意账号)
34.  if (   main_auz_account!="
35.      && (   false
36.          || (   true
37.              && main_auz_account==main_auz_demo
38.              && AI.trade_mode!=0
39.              && egPrint(PrintToLog,"本 EA 仅限模拟账户使用")
40.            )
41.          || (   true
42.              && main_auz_account==main_auz_real
43.              && AI.trade_mode==0
44.              && egPrint(PrintToLog,"本 EA 仅限真实账户使用")
45.            )
46.          || (   true
47.              && (main_auz_account!=main_auz_demo && main_auz_account!=mai
n_auz_real)
48.              && main_auz_account!=(string)AI.login
49.              && egPrint(PrintToLog,"本 EA 仅限指定账户使用")
50.            )
51.          )
52.    )
53.  {
54.      return(1);
55.  }
56. //---结束
57.  return(0);
58. }

```

13.1 使用期限

在“授权到期”双引号中输入程序到期日期，注意用英文字符，格式为“yyyy.mm.dd hh.mm”。输入空值表示没有限制。

程序到期后，会自动从图表中卸载，并且在 EA 日志中有提示：



MT5 界面



MT4 界面

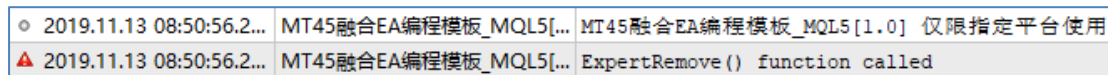
到期前 10 天开始，图表左上角会有到期显示：



13.2 交易服务器

在“授权服务器”双引号中输入交易服务器全称，表示程序仅限于指定服务器使用。输入空值表示没有限制。

程序识别到非指定交易服务器，会自动从图表中卸载，并且在 EA 日志中有提示：



交易服务器可以从 MT5(4)终端界面上“工具-选项”窗口中查到：

13.3 账户余额

在“授权账户余额”双引号中输入金额数，表示程序仅在账户余额小于指定金额时可以使用。输入空值表示没有限制。

程序识别到账户金额超过指定金额，会自动从图表中卸载，并且在 EA 日志中有提示：

2019.11.13 08:59:26.9...	MT45融合EA编程模板_MQL5[...	MT45融合EA编程模板_MQL5[1.0] 账户余额超标
2019.11.13 08:59:26.9...	MT45融合EA编程模板_MQL5[...	ExpertRemove() function called

13.4 帐号类型

在“授权账户”双引号中输入账户 ID（一串数字）表示程序只能在指定账户中使用。

输入“real”表示程序只能在非模拟账户中使用。

输入“demo”表示程序只能在模拟账户中使用。

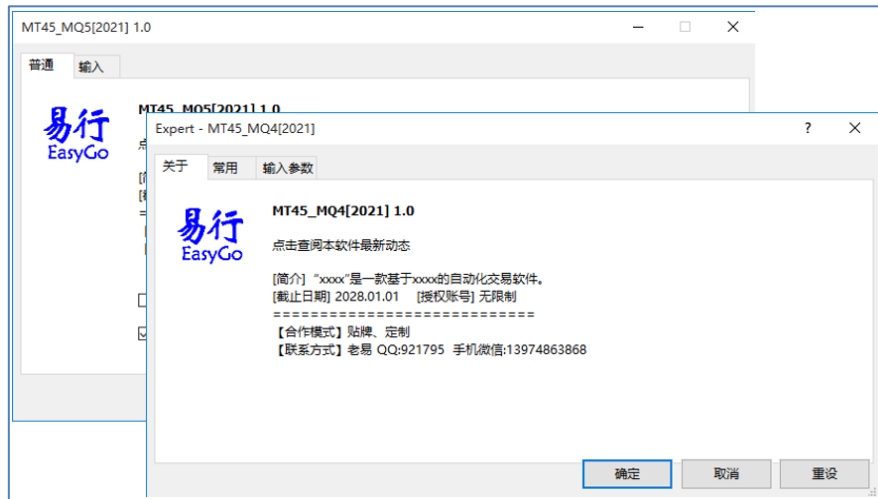
输入空值表示没有限制。

程序识别到账户金额超过指定金额，会自动从图表中卸载，并且在 EA 日志中有提示：

2019.11.13 09:09:04.9...	MT45融合EA编程模板_MQL5[...	MT45融合EA编程模板_MQL5[1.0] 仅限指定账户使用
2019.11.13 09:09:04.9...	MT45融合EA编程模板_MQL5[...	ExpertRemove() function called
2019.11.13 09:10:32.1...	MT45融合EA编程模板_MQL5[...	MT45融合EA编程模板_MQL5[1.0] 仅限实盘/竞赛账户使用
2019.11.13 09:10:32.1...	MT45融合EA编程模板_MQL5[...	ExpertRemove() function called

14 EA 封面

EA 加载后，图表会弹出一个对话框，“关于”标签就是程序封面：



在主程序开头输入以下代码：

```
1. //--- EA 封面
2. #property version "2.0"
3. #property copyright "点击查阅本软件最新动态"
4. #property link "http://blog.sina.com.cn/yiwen"
5. #property description "[简介] "xxxx"是一款基于 xxxx 的自动化交易软件。"
6. #property description "[截止日期] 2028.01.01 [授权账号] 无限制"
7. #property description "===== "
8. #property description "【合作模式】贴牌、定制"
9. #property description "【联系方式】老易 QQ:921795 手机微信:13974863868"
10. #property icon "\\Images\\EasyGo\\yixingLogo.ico" //logo 必须是
    ico 格式
```

注意：图标 logo 文件格式必须是 ico 格式，制作好图片后可以在百度搜索“ico 转换”随便找个在线转换的网站，按照指引操作就可以得到 ico 格式的文件了。

15 控制面板设计

配套的 MT45 融合模版范例呈现了这个模块。

EA 操盘需要一个友好的人机交互界面，这就是本章要讲述的“控制面板程序设计”。先来看看下图，具备了一个控制面板应有的基本功能。：

- 可用鼠标拖动，摆放在主图中任意位置。
- 可以收放，收起后能显示更多的主图内容，方便做技术分析。
- 动态显示数据，如报价、持仓单统计、交易时间等数据。
- 按钮可以执行一些 MT45 主程序不能便捷操作的功能，如暂停 EA、一键清仓等。
- 鼠标指向特定元素，弹出文字、图片说明。



15.1 标准程序库简介

MT4/MT5 都提供了一整套标准程序库，在“Include”文件夹中，这些程序采用了面向对象的方法编制，二者是相互兼容，是通用的。

MT5 帮助文档提供了这套程序库完整的使用说明，MT4 则没有。

控制面板编程需要用到“Include\Controls”中的 Dialog.mqh（对话框）、Button.mqh（按钮）和 Label.mqh（标签）三个程序库。

我们不需要知道标准程序库的内部逻辑，会用就行。

为了使主程序（MT45 融合模版范例.mq5）更加简洁，我们将所有与控制面板相关的代码写到“MT45Panel.mqh”文件中，接下来的编码会在两个文件中进行。

本章通过具体案例介绍“面向对象编程”的概念。

15.2 面向对象编程

面向对象编程中“类”和“实体”可以简单地这么来理解：某小学每个年级设置 4 个班，每个班 50 人，这相当于“类”，现在我们给每个班编号 1 班（实体 1），2 班（实体 2），以此类推。现在给“类”增加一个属性，例如男女生各占 25 人，那么每个实体自然也就具备了这个属性，而不需要逐个去增加。

修改“类”属性，每个“实体”的属性会自动修改，这在专业上叫做“继承”。面向对象编程的好处就在于：对于同一“类”的“实体”整体性更强，避免重复编码，代码也更健壮。

下面，通过实际编码来快速理解这个概念。

15.3 创建个性化类和实体

在程序头声明程序库：

```
1. #include <Controls\Dialog.mqh> // 加载对话框
2. #include <Controls\Button.mqh> // 加载按钮
3. #include <Controls\Label.mqh> // 加载标签
4. #include <egMT45\MT45.mqh> // 加载变量与函数
```

其中第 4 行是 MT45 融合系统的变量与函数。

先定义一个名为“CegPanel”控制面板的类：

```
1. /*
2. 函数：定义 CegPanel 类
3. 返回：
4. 算法：包含预定义元件变量及方法、初始化方法、刷新数据方法
5. */
6. class CegPanel
7. {
8.     private: //私有变量
9.         //==== 新增元件 开始 ====
10.         CLabel m_label1; //标签 1 其他信息
11.         //==== 新增元件 结束 ====
12.
13.         string obj_text; // 对象文字
14.         string edit1_usual; // 状态栏常规信息变量
15.
16.         CEdit m_edit1; // 文本 1 状态栏
17.
18.         CEdit m_edit2; // 文本 2 持仓单 Sell 组信息框
19.         CEdit m_edit2_1; // 文本 2_1 Sell 组浮动利润
20.         CEdit m_edit2_2; // 文本 2_2 Sell 组持仓单数
21.         CEdit m_edit2_3; // 文本 2_3 Sell 组持仓量/保证金
22.
23.         CEdit m_edit3; // 文本 3 持仓单 Buy 组信息框
24.         CEdit m_edit3_1; // 文本 3_1 Buy 组浮动利润
25.         CEdit m_edit3_2; // 文本 2_2 Buy 组持仓单数
26.         CEdit m_edit3_3; // 文本 2_3 Buy 组持仓量/保证金
27. }
```

```

28.      CEdit  m_edit4_1;  // 文本 4_1 账户余额
29.      CEdit  m_edit4_2;  // 文本 4_2 账户净值
30.      CEdit  m_edit4_3;  // 文本 4_3 已用保证金
31.      CEdit  m_edit4_4;  // 文本 4_4 可用保证金
32.      CEdit  m_edit4_5;  // 文本 4_5 保证金比例
33.      CEdit  m_edit4_6;  // 文本 4_6 账户利润
34.
35.      CButton m_button1;  // 按钮 1 一键清仓
36.      CButton m_button2;  // 按钮 2 Sell 组平仓
37.      CButton m_button3;  // 按钮 3 Buy 组平仓
38.      CButton m_button4;  // 按钮 4 暂停
39.
40.      public: //公开变量与方法
41.          CAppDialog m_Panal;          // 主窗口
42.          bool      PanelInit();        // 面板初始化
43.          bool      PanelRefresh();     // 刷新面板数据
44.          int       PanelAction();     // 面板行为
45.          void      PanelTestMode();    // 面板回测模式
46.
47.      protected: //保护方法
48.          //==== 新增方法 开始 ====
49.          bool CreateLabel1();          //标签 1 其他信息
50.          //==== 新增方法 结束 ====
51.
52.          bool CreateEdit1();           // 状态栏
53.          bool CreateButton1();         // 按钮-一键清仓
54.          bool CreateButton2();         // 按钮-Sell 组平仓
55.          bool CreateButton3();         // 按钮-Buy 组平仓
56.          bool CreateButton4();         // 按钮-暂停
57.
58.          bool CreateEdit2();           // 持仓单 Sell 组信息框
59.          bool CreateEdit3();           // 持仓单 Buy 组信息框
60.          bool CreateEdit2_1();         // Sell 组浮动利润
61.          bool CreateEdit3_1();         // Buy 组浮动利润
62.          bool CreateEdit2_2();         // Sell 组持仓单数
63.          bool CreateEdit2_3();         // Sell 组持仓量/保证金
64.          bool CreateEdit3_2();         // Buy 组浮动利润
65.          bool CreateEdit3_3();         // Buy 组持仓量/保证金
66.
67.          bool CreateEdit4_1();         // 账户余额
68.          bool CreateEdit4_2();         // 账户净值
69.          bool CreateEdit4_3();         // 已用保证金
70.          bool CreateEdit4_4();         // 可用保证金
71.          bool CreateEdit4_5();         // 保证金比例

```

```

72.         bool CreateEdit4_6();    // 账户利润
73.
74. };
75.
76. CegPanel egPanel; // 定义面板变量

```

第 76 行定义一个名为“egPanel”的实体。接下来都是围绕这个实体（egPanel）展开编程。

名为“CegPanel”的类包含了变量和方法。其中：

第 8 行开始到第 38 行用 **private** 定义私有变量；

第 40 行到 45 行用 **public** 定义公开变量和方法；

第 47 行到 72 行用 **protected** 定义受保护的方法。

用 **public** 定义的变量和方法可以在主程序中读取、调用，用 **private** 和 **protected** 定义的变量和方法只能在 CegPanel 内部读取调用，这样设计的目的是让程序逻辑不容易混乱。

15.4 创建面板元件

用 **protected** 定义受保护的方法都是面板上各种元件，在程序中输入以下代码：

```

1.  /*
2.  函数：按钮-一键清仓
3.  返回：true=成功  false=失败
4.  算法：x 坐标根据主面板宽度均分
5.  */
6.  bool CegPanel::CreateButton1()
7.  {
8.      int x0,x1,y1,x2,y2; // 位置坐标
9.      x0=(m_Panal.Width()-BORDERDEV_X)/8;
10.     x1=1*x0-BUTTON_WIDTH/2; // x1 坐标
11.     y1=INDENT_TOP; // y1 坐标
12.     x2=x1+BUTTON_WIDTH; // x2 坐标
13.     y2=y1+BUTTON_HEIGHT; // y2 坐标
14.     obj_text="一键清仓"; // 文字
15.
16.     if ( false
17.         || !m_button1.Create(0,obj_text,0,x1,y1,x2,y2) // 创建按钮
18.         || !m_button1.Text(obj_text) // 设置文字
19.         || !m_Panal.Add(m_button1) // 绑定到面板
20.     )
21.     {
22.         return(false);
23.     }
24.     ObjectSetInteger(0,m_button1.Name(),OBJPROP_ZORDER,1); //设置鼠标优先，在
    历史测试中按钮生效

```

```

25.
26. //--- 结束
27.     return(true);
28. }

```

第 6 行“bool CegPanel::CreateButton1()”表示 CreateButton1 行为函数属于 CegPanel 类。
第 24 行设置“一键清仓”按钮鼠标属性，可以在 MT45 的历史数据测试中生效。

15.5 面板初始化 PanelInit()

将编写好的元件组合到控制面板中：

```

1.  /*
2.  函数： 面板初始化
3.  返回： true=成功  false=失败
4.  */
5.  bool CegPanel::PanelInit()
6.  {
7.  //--- 创建主窗口
8.      if (!m_Panal.Create(0,ProgramName+" "+_Symbol,0,BOARD_X,BOARD_Y,BOARD_W
          IDTH,BOARD_HEIGHT)) return(false);
9.
10.     //==== 新增元件 开始 ====
11.     CreateLabel1();    // 标签 1 其他信息
12.     //==== 新增元件 结束 ====
13.
14. //--- 添加元件
15.     CreateEdit1();    // 状态栏
16.     CreateButton1();  // 按钮-一键清仓
17.     CreateButton2();  // 按钮-Sell 组平仓
18.     CreateButton3();  // 按钮-Buy 组平仓
19.     CreateButton4();  // 按钮-暂停
20.
21.     CreateEdit2();    // 持仓单 Sell 组信息框
22.     CreateEdit2_1();  // Sell 组浮动利润
23.     CreateEdit2_2();  // Sell 组浮动利润
24.     CreateEdit2_3();  // Sell 组持仓量/保证金
25.
26.     CreateEdit3();    // 持仓单 Buy 组信息框
27.     CreateEdit3_1();  // Buy 组浮动利润
28.     CreateEdit3_2();  // Buy1 组浮动利润
29.     CreateEdit3_3();  // Buy 组持仓量/保证金
30.
31.     CreateEdit4_1();  // 账户余额

```

```

32. CreateEdit4_2(); // 账户净值
33. CreateEdit4_3(); // 已用保证金
34. CreateEdit4_4(); // 可用保证金
35. CreateEdit4_5(); // 保证金比例
36. CreateEdit4_6(); // 账户利润
37.
38. //--- 运行面板
39. m_Panal.Run();
40.
41. //--- 结束
42. return(true);
43. }

```

这个函数在主程序（MT45 融合模版范例）的 OnInit()中调用一次（以下代码第 11、12 行）：

```

1. /*
2. 函数：EA 加载初始化
3. 返回：
4. 算法：
5. */
6. int OnInit()
7. {
8. //--- 刷新数据
9. egDataRefresh();
10.
11. //--- 初始化面板 npt_99_5=true 启用控制面板(true=启用,false=禁用)
12. if (npt_99_5==true && !egPanel.PanelInit()) return(INIT_FAILED);
13.
14. //--- 创建时间触发器,设置秒（毫秒）为周期运行一次时间触发器 OnTimer()
15. EventSetMillisecondTimer(10); // 10 毫秒(0.01 秒)刷新
16.
17. return(INIT_SUCCEEDED);
18. }

```

15.6 面板行为 PanelAction()

这个函数用来识别鼠标点击按钮、鼠标滑入等行为，以下为部分代码：

```

1. /*
2. 函数：面板行为
3. 返回：-1=无行为 0=暂停 1=一键清仓 2=Sell 组平仓 3=Buy 组平仓
4. */
5. int CegPanel::PanelAction()

```



```

6. {
7.     int myreturn=-1; //返回值变量
8.
9.     ///--- “暂停”按钮 m_button4, 按钮文字变色
10.    if (m_button4.Pressed()==true) myreturn=0;
11.
12.    ///--- 点击“Buy 组平仓”按钮 m_button3, 执行平仓
13.    if (    true
14.        && TS.buy_orders>0
15.        && m_button3.Pressed()==true
16.        && egOrdersCloseByCondition(T0,CloseTicket,0,-1)
17.        && egPrint(true,"点击“Buy 组平仓”按钮, Buy 组执行平仓")
18.        && m_button3.Pressed(false)==true
19.    )
20.    {
21.        myreturn=3;
22.    }
23. ....
24. ///==== 以下输出模版默认元件提示信息 ====
25.
26. ///--- 鼠标进入/移出“Buy 组平仓”按钮 m_button3, 状态栏提示说明文字
27.    if (    true
28.        && m_button3.StateFlags()==7 // 鼠标进入
29.        && m_edit1.Color(clrBlue) // 状态栏文字变色
30.        && m_edit1.Text("点击【Buy 组平仓】按钮, 所有 Buy 单市价平仓。注意: 平仓有
        延时") // 状态栏文字
31.    )
32.    {}
33. ....
34.
35. ///--- 结束
36.    return(myreturn);
37. }

```

这个函数在主程序（MT45 融合模版范例）的 OnChartEvent()中调用处理鼠标事件（以下代码第 12 行）：

```

1. /*
2. 图表事件
3. */
4. void OnChartEvent(const int id,          // event ID 事件代码
5.                  const long& lparam,    // event parameter of the long type
6.                  const double& dparam, // event parameter of the double typ
7.                  int x,                 // 鼠标 x 坐标
8.                  int y)                // 鼠标 y 坐标

```

```

7.          const string& sparam // event parameter of the string typ
   e  对象名称
8.          )
9.  {
10. //--- 面板事件
11.     egPanel.m_Panal.ChartEvent(id,lparam,dparam,sparam);
12.     egPanel.PanelAction();
13.
14. }

```

第 11 行处理鼠标常规行为，例如最小化、最大化、拖动面板、关闭面板等。

这个函数在主程序（MT45 融合模版范例）的 egMain() 中调用获取“暂停”按钮状态，当处于暂停状态时，不执行后续动作（以下代码第 17、18 行）：

```

1.  /*
2.  函 数:总流程
3.  输出参数:true-可执行, false-不可执行
4.  算 法:
5.  */
6.  bool egMain()
7.  {
8.  //--- 如果交易停止, 不执行后续动作
9.      if (egTradingStatus()==false) return(true);
10.
11. //--- 平仓优先 数组平仓
12.     if (egOrdersCloseByArray(CloseTicket)) return(true);
13.
14. //--- 刷新数据
15.     egDataRefresh();
16.
17. //--- 暂停 EA
18.     if (egPanel.PanelAction()==0) return(true);
19.
20. //--- 控制逻辑
21.     if ( false
22.         || egReduce() // 减仓
23.         || egCreat() // 建仓
24.         || egAdd() // 加仓
25.         || egTPSL() // 止盈止损
26.     )
27.     {
28.         return(true);
29.     }
30.
31. //--- 结束

```

```

32.     return(true);
33. }

```

15.7 刷新面板数据 PanelRefresh()

这个函数用来刷新面板数据，以下是部分代码：

```

1.  /*
2.  函数：刷新面板数据
3.  返回：true=成功 false=失败
4.  */
5.  bool CegPanel::PanelRefresh()
6.  {
7.  //==== 新增元件刷新 开始 ====
8.
9.  //--- m_label1; //标签 1 其他信息
10.     obj_text="建仓信号： "+SignalCode[0];
11.     m_label1.Text(obj_text);
12.
13. //==== 新增元件刷新 结束 ====
14.
15.
16. //--- 空仓时“Buy 组平仓”按钮 m_button3 文字显示灰色
17.     if (TS.buy_orders==0) m_button3.Color(clrSilver); // 文字显示灰色
18.     else m_button3.Color(clrBlack); // 文字显示黑色
19.     .....
20. //--- 鼠标进入/移出“m_edit4_6 账户利润”，状态栏提示说明文字
21.     obj_text=DoubleToString(AI.profit,2);
22.     m_edit4_6.Text(obj_text);
23.
24. //--- 结束
25.     return(true);
26. }

```

这个函数在控制面板程序（MT45Panel.mqh）的 PanelTestMode()测试模式函数中调用一次（以下代码第 11、12 行）：

```

1.  /*
2.  函 数：面板回测模式
3.  输出参数：ture
4.  算 法：按钮在 MT4 中有效
5.  */
6.  void CegPanel::PanelTestMode() // 面板回测模式
7.  {

```

```

8.  //--- 不是回测模式，退出
9.      if (MQLInfoInteger(MQL_TESTER)==false) return;
10.
11. //--- 进入回测模式
12.     PanelRefresh(); // 刷新面板数据
13.
14. //--- 结束
15.     return;
16. }

```

这个函数在主程序（MT45 融合模版范例）的 OnTimer()中调用一次（以下代码第 11、12 行）：

```

1.  /*
2.  函数：时间触发器
3.  返回：
4.  算法：每 10 毫秒刷新一次
5.  */
6.  void OnTimer()
7.  {
8.  //---刷新系统数据
9.      egDataRefresh();
10.
11. //---刷新面板数据
12.     egPanel.PanelRefresh();
13.
14. }

```

注意：这里启用了时间触发器，程序每 0.01 秒执行一次数据刷新。时间触发器在历史测试中无效。

15.8 回测模式 PanelTestMode();

这个函数专门为 MT45 历史测试编写，实时刷新历史数据：

```

1.  /*
2.  函    数：面板回测模式
3.  输出参数：ture
4.  算    法：按钮在 MT4 中有效
5.  */
6.  void CegPanel::PanelTestMode() // 面板回测模式
7.  {
8.  //--- 不是回测模式，退出
9.      if (MQLInfoInteger(MQL_TESTER)==false) return;

```

```

10.
11. //--- 进入回测模式
12.     PanelRefresh(); // 刷新面板数据
13.
14. //--- 结束
15.     return;
16. }

```

在主程序（MT45 融合模版范例）的 egDataRefresh()中调用（以下代码第 33、34 行）：

```

1.  /*
2.  函    数:刷新数据
3.  输出参数:
4.  算    法:刷新所有数据
5.  */
6.  void egDataRefresh()
7.  {
8.  //--- 更新程序识别码
9.      MagicNumber=npt_95_1;
10.
11. //--- 刷新账户与商品信息
12.     egRefreshAI(AI); // 账户信息
13.     egRefreshSI(_Symbol,SI); // 当前商品信息
14.
15. //--- 刷新持仓单基本信息
16.     egRefreshTO(TO,SI.symbol,-1,MagicNumber); // 持仓单信息
17.     egRefreshTS(SI,TO,TS); // 持仓单统计信息
18.
19. //--- 刷新历史单基本信息
20.     if (    true
21.         && npt_99_1==true // 启用历史数据(false=不启用)
22.         && prvHistoryOrders!=egHstTotal() // 历史单数量发生变化才执行一次
23.     )
24.     {
25.         egRefreshHO(HO,_Symbol,-
26.             1,MagicNumber,npt_99_2,StringToTime(npt_99_3),StringToTime(npt_99_4)); // 刷
27.             新历史单数组
28.         egRefreshHS(HO,HS); // 刷新历史单统计
29.         prvHistoryOrders=egHstTotal();
30.     }
31.
32. //--- 刷新信号
33.     egTradingSignal(SignalCode);
34.
35. //--- 回测模式

```

```
34.     egPanel.PanelTestMode();  
35.  
36. //--- 结束  
37.     return;  
38. }
```

16 命令集

16.1 账户与商品信息

16.1.1 刷新账户信息

```

1.  /*
2.  函    数：刷新账户信息
3.  输出参数：
4.  算法说明：给常用账户信息赋值
5.  */
6.  void egRefreshAI(AccountInfo &myAI)

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myAI：预定义账户信息变量。[账户信息 AccountInfo\(AI\)](#)。

【范例】

1. 在主程序头部预定义一个账户信息变量 AI：

```
1. AccountInfo AI; //账户信息变量
```

2. 在主程序 egDataRefresh() 模块中输入以下命令，将生成一个符合条件的账户信息变量：

```
1. egRefreshAI(AI); //账户信息
```

这条命令的意思是刷新一个名为 “AI” 的账户信息变量。

在程序中输入 AI.balance 即获取当前账户的余额。

在程序中输入 AI.company 即获取当前账户的服务商名称。

其他信息以此类推。

多货币操作时，在程序中输入 egTradingSymbols(AI.trade_symbols) 获取当前持仓商品数量，输入 AI.trade_symbols[0] 获取当前持仓商品中第一个商品名称。参考[持仓商品](#)。

正文参考：[订单数组](#)。

16.1.2 商品信息

```

1.  /*
2.  函    数：刷新商品信息
3.  输出参数：
4.  算法说明：
5.  */

```

```

6. void egRefreshSI(string mySymbol, //商品名称
7.                SymbolInfo &mySI //商品信息
8.                )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egOC5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egOC4.mqh 文件中。

【参数说明】

Symbol: 指定商品名称，_Symbol 表示当前图表商品，指定非当前图表商品时，必须输入完整的名称，例如 XAUUSD 在有些平台名称是 GOLD，或者 XAUUSDpro。

mySI: 预定义商品信息变量。多货币操作可以定义多个变量，例如 SI1、SI2，也可以定义为数组，灵活度更高。[商品信息 SymbolInfo\(SI\)](#)。

【范例】

1. 在主程序头部预定义一个商品信息变量：

```
1. SymbolInfo SI; //商品信息变量
```

2. 在主程序 egDataRefresh() 模块中输入以下命令，将生成一个符合条件的商品信息变量：

```
1. egRefreshSI(_Symbol,SI); //当前商品信息
```

这条命令的意思是刷新一个名为 “SI” 的当前图表商品信息变量，参数 “_Symbol” 可以指定平台任意商品名称，例如：“EURUSD”。

在程序中输入 SI.bid 即获取指定商品的 bid 报价。

在程序中输入 SI.spread 即获取指定商品的点差。

其他信息以此类推。

正文参考：[商品信息](#)。

16.1.3 持仓商品

```

1. /*
2. 函 数：持仓商品
3. 输出参数：持仓商品数量、商品名称数组
4. 算法说明：
5. */
6. int egTradingSymbols(string &mySymbol[]) //商品名称数组

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

mySymbol[]: 预定义商品名称数组，在模版中可直接使用 AI.trade_symbols。

返回: 该命令返回两个参数，一是持仓商品数量，二是商品名称数组 mySymbol[]。

【范例】

1. 输入 egTradingSymbols(AI.trade_symbols)，返回当前持仓商品数量，参数 AI.trade_symbols 为预定义账户信息字符型数组变量。

2. 输入 `AI.trade_symbols[0]`，返回当前持仓商品中第一个商品名称，如果没有持仓商品，程序会报错，使用这条命令前，要注意商品数量必须大于 0，例如：

```
1. if (egTradingSymbols(AI.trade_symbols)>0) Print(AI.trade_symbols[0]);
```

注意：如果 `AI.trade_symbols[n]` 中的 `n` 大于等于当前持仓商品数量，程序也会报错。

16.2 持仓单统计与检索

16.2.1 复制持仓单数组

```
1. /*
2. 函 数：复制持仓单变量
3. 输出参数：true=成功，false=失败
4. 算法说明：
5. */
6. bool egT0Copy(TradesOrders &myT01, //源变量
7.               TradesOrders &myT02 //目标变量
8.               )
```

命令源码在 “\Include\egMT45\” 中的 `egOC.mqh` 文件中，MT4/5 通用。

【参数说明】

myT01：源持仓单变量。[持仓单信息 TradesOrders\(TO\)](#)。

myT02：目标持仓单变量。[持仓单信息 TradesOrders\(TO\)](#)。

返回：true 表示执行成功，false 表示不成功。

16.2.2 持仓单数组按类型排序

```
1. /*
2. 函 数：持仓单数组按类型排序
3. 输出参数：
4. 算法说明：冒泡算法，针对指定的订单数组按条件执行降序排序。
5. */
6. void egT0Sort(TradesOrders &myT0[], //持仓单数组
7.               int mySeekMode //排序类型 0=按建仓时间,1=按建仓
8.               价,2=按浮盈,3=按浮亏)
```

命令源码在 “\Include\egMT45\” 中的 `egOC.mqh` 文件中，MT4/5 通用。

【参数说明】

myT0：持仓单数组。[持仓单信息 TradesOrders\(TO\)](#)。

mySeekMode：排序类型 0=按建仓时间,1=按建仓价,2=按浮盈,3=按浮亏。

返回：无返回。

说明：按浮盈和按浮亏排序结果是一样的，都是按降序排列。

16.2.3 刷新持仓单数组

```

1.  /*
2.  函    数：刷新持仓单数组
3.  输出参数：持仓单数量
4.  算法说明：
5.  */
6.  int egRefreshTO(TradesOrders    &myTO[],    //持仓单数组
7.                  string          mySymbol,    //指定商品, "*"表示所有持仓单
8.                  int             myType,       //指定订单类型:
9.                  //0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有
10.                 int             myMagicNum    //程序识别码, -1=所有持仓单
11.                 )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egOC5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egOC4.mqh 文件中。

【参数说明】

myTO[]: 预定义持仓单变量。持仓单信息 TradesOrders(TO)。

mySymbol: 指定商品名称, "*"表示所有持仓单。多货币操作可预定义若干 myTO 变量, 实现快速分类。

myType: 指定订单类型, 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有。注意不要用 MT4/5 常量, 直接用数字, 避免不兼容。

myMagicNum: 程序识别码, -1=所有持仓单。约定程序识别码必须是一个大于 0 的正整数。

【范例】

1. 在主程序头部预定义一个持仓单变量:

```
1. TradesOrders TO_buy[]; //持仓单信息变量
```

2. 在主程序 egDataRefresh() 模块中输入以下命令:

```
1. egRefreshTO(TO_buy,"EURUSD",0,-1); //持仓单信息
```

运行结果将生成一个实时更新的 TO_buy 数组, 这个数组汇集了所有符合以下条件的持仓单: 商品名称为 EURUSD, 订单类型为 buy, 所有持仓单。

在程序中输入变量“TO_buy[0].ticket”就能够获取在数组中序号为 0 的持仓单的订单号。

较为复杂的操盘策略中, 经常需要将持仓单按不同条件分类处理, 可以有针对性地预定义不同名称的持仓单变量。

正文参考: [订单数组](#)。

16.2.4 刷新持仓单统计信息

```

1.  /*
2.  函    数:刷新持仓单统计信息
3.  输出参数:false-未统计, true-已统计
4.  算    法:
5.  单边组分别统计累计持仓单数量、持仓量、利润、均价、保证金、成本
6.  */
7.  bool egRefreshTS(SymbolInfo      &mySI,          //商品信息
8.                  TradesOrders    &myTO[],         //持仓单数组
9.                  TradesStatistical &myTS           //统计结果
10.                 )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

mySI: 指定商品的相关信息, 计算保证金时要取数。[商品信息 SymbolInfo\(SI\)](#)。

myTO: 指定持仓单数组。[持仓单信息 TradesOrders\(TO\)](#)。

myTS: 生成持仓单统计信息。[持仓单统计信息 TradesStatistical\(TS\)](#)。

返回: true 表示执行成功, false 表示不成功。

【范例】

1. 在主程序头部预定义一个持仓单统计变量 TS:

```
1. TradesStatistical TS; //持仓单统计变量
```

2. 在主程序 egDataRefresh() 模块中输入以下命令, 将生成一个符合条件的持仓单统计变量:

```
1. egRefreshTS(SI, TO, TS); //持仓单统计信息
```

运行结果将生成一个基于 SI (指定商品)、TO (指定持仓单集合) 的实时更新的 TS 变量。

在程序中输入变量 “TS.buy_orders” 就能够获取 TO 持仓单集合中 buy 单的数量。

其他以此类推。

注意:

- 多货币操作时, TO 数组中只能有一个商品入列, 以确保准确的统计结果。
- buy 组利润 (TS.buy_profit) 和 sell 组利润 (TS.sell_profit) 包含了掉期和手续费。

正文参考: [统计](#)。

16.2.5 持仓单检索

```

1.  /*
2.  函    数: 持仓单检索
3.  输出参数: 订单号, 0=无订单

```

```

4.  算法说明: 冒泡
5.  针对指定的订单数组执行按组合条件定位搜索。
6.  */
7.  ulong  egOrderSearch(TradesOrders  &myTO[],          //持仓单数组
8.                          string      mySymbol,          //商品名称, Symbol()为当前
   图表商品名, "*"为所有商品
9.                          int         mySeekMode,         //排序类型 0=按建仓时间,1=
   按建仓价,2=按浮盈,3=按浮亏
10.                         int         myOrderType,        //订单类
   型 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有
11.                         int         myMagicNum,         //程序识别码, -1=所有订单
12.                         int         mySerialNumber      //排序 1 为最大, 2 为次大, 以
   此类推, -1 为最小, -2 为次小, 以此类推
13. )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

myTO[]: 持仓单数组。 [持仓单信息 TradesOrders\(TO\)](#)。

mySymbol: 商品名称, "*"为所有商品。如果 myTO 中有多个商品名称, 则可以指定其中一个。

mySeekMode: 排序类型, 0=按建仓时间,1=按建仓价,2=按浮盈,3=按浮亏。

myOrderType: 订单类型, 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有。

myMagicNum: 程序识别码, -1=所有订单。

mySerialNumber: 排序 1 为最大, 2 为次大, 以此类推, -1 为最小, -2 为次小, 以此类推。

返回: 订单号, 0=查无订单。

【范例】

这条命令可以设置非常丰富的条件组合来查找相应的持仓单。例如:

- 查找所有持仓单中浮亏最小的单号, 没有浮亏单则返回 0:

```
1. egOrderSearch(TO, "*", 3, -1, -1, -1);
```

- 查找 buy 单组中建仓时间最近的单号:

```
1. egOrderSearch(TO, "*", 0, 0, -1, 1);
```

- 查找 sell 单组中建仓价最低的单号:

```
1. egOrderSearch(TO, "*", 1, 1, -1, -1);
```

- 如果 TO 包括了多个商品, 可以这样查找 EURUSD 的 sell 单组中建仓价最低的单号:

```
1. egOrderSearch(TO, "EURUSD", 1, 1, -1, -1);
```

- 如果 TO 中有多个程序控制的持仓单, 可以这样查找程序识别码为 123456 的 sell 单组中建仓价最低的单号:

```
1. egOrderSearch(T0, "*", 1, 1, 123456, -1);
```

注意:

按浮盈查找持仓单时, 只返回浮盈单单号, 没有浮盈单的, 返回 0。

按浮亏查找持仓单时, 只返回浮亏单单号, 没有浮亏单的, 返回 0。

正文参考: [检索](#)。

16.2.6 持仓单定位

```
1. /*
2. 函 数: 持仓单定位
3. 输出参数: 持仓单在 myTO 数组中的位置, -1 表示找不到订单
4. 算法说明:
5. */
6. int egOrderPos(TradesOrders &myTO[], //持仓单数组
7.                ulong myTicket //订单号
8.                )
```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

myTO[]: 持仓单数组。 [持仓单信息 TradesOrders\(TO\)](#)。

myTicket: 订单号。指定一个持仓单单号。

返回: 指定订单在 myTO 数组中的序号。

【范例】

myTO 是一个数组, 持仓单保存在每一个 myTO[i]里, 用 for 命令遍历所有持仓单时, 不能通过订单号实现循环, 只能通过订单所在数组的序号来实现。这条命令返回指定订单在 myTO 数组中的序号。例如:

```
1. int i=egOrderPos(T0, 123456);
```

返回单号为 “123456” 的持仓单在 myTO 数组中的序号。

TO[i].openprice 表示序号为 i 的持仓单建仓价。其他属性以此类推。

16.2.7 持仓单数组数组合并

```
1. /*
2. 函 数: 持仓单数组数组合并
3. 输出参数:
4. 算法说明: 将参数表中两个持仓单数组数组合并为一个。
5. */
6. void egTOMerge(TradesOrders &myT01[], //源 T01
7.                TradesOrders &myT02[], //源 T02
```

```

8.         TradesOrders      &myTO[]      //目标 TO
9.     )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myTO1[]: 源持仓单数组。[持仓单信息 TradesOrders\(TO\)](#)。

myTO2[]: 源持仓单数组。[持仓单信息 TradesOrders\(TO\)](#)。

myTO[]: 目标持仓单数组。[持仓单信息 TradesOrders\(TO\)](#)。

返回: 无。

【范例】

在一个交易策略中经常会将各种分类处理的持仓单组汇总起来进行操作。这条命令就是将分类持仓单组合并为一个汇总持仓单组。常用于多货币策略、同一商品多类别策略。

在主程序中预定义三组变量，TO、TS 为持仓单汇总变量，TO_1、TS_1 为根据信号 1 入场的持仓单组变量，TO_2、TS_2 为根据信号 2 入场的持仓单组变量。

```

1. //汇总单
2. TradesOrders      TO[];      //持仓单信息变量
3. TradesStatistical TS;        //持仓单统计变量
4. //信号 1
5. TradesOrders      TO_1[];    //信号 1 组持仓单信息变量
6. TradesStatistical TS_1;      //信号 1 组持仓单统计变量
7. int magic1=1001; //EA 识别码
8. //信号 2
9. TradesOrders      TO_2[];    //信号 2 组持仓单信息变量
10. TradesStatistical TS_2;      //信号 2 组持仓单统计变量
11. int magic2=1002; //EA 识别码

```

在 “egDataRefresh” 模块中刷新信号 1、信号 2 持仓单数组变量，然后合并到 TO:

```

1. //--- 刷新信号 1 持仓单基本信息
2. egRefreshTO(TO_1,SI.symbol,-1,magic1); //持仓单信息
3. egRefreshTS(SI,TO_1,TS_1); //持仓单统计信息
4. //--- 刷新信号 2 持仓单基本信息
5. egRefreshTO(TO_2,SI.symbol,-1,magic2); //持仓单信息
6. egRefreshTS(SI,TO_2,TS_2); //持仓单统计信息
7. //--- 刷新持仓单基本信息 合并持仓单组
8. egTOMerge(TO_1,TO_2,TO); //持仓单信息
9. egRefreshTS(SI,TO,TS); //持仓单统计信息

```

持仓单数组 TO 包含了 TO_1 和 TO_2 数组全部记录。

注意: 在多货币环境中，持仓单统计变量 TS 中的组均价、保证金数据无效。

16.3 历史单统计与检索

16.3.1 复制历史单数组

```

1.  /*
2.  函    数: 复制历史单变量
3.  输出参数: true=成功, false=失败
4.  算法说明:
5.  */
6.  bool egHOCopy(HistoryOrders &myHO1, //源变量
7.               HistoryOrders &myHO2  //目标变量
8.               )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myHO1: 源历史单变量。 [历史单信息 HistoryOrders\(HO\)](#)。

myHO2: 目标历史单变量。 [历史单信息 HistoryOrders\(HO\)](#)。

返回: true 表示执行成功，false 表示不成功。

16.3.2 历史单数组按类型排序

```

1.  /*
2.  函    数: 历史单数组按类型排序
3.  输出参数:
4.  算法说明: 冒泡，针对指定的订单数组按条件执行降序排序。
5.  */
6.  void egHOSort(HistoryOrders    &myHO[],    //持仓单数组
7.               int               mySeekMode  //排序类型 0=按建仓时间,1=按建仓
               价,2=按盈利,3=按亏损,4-按平仓时间,5-按平仓价
8.               )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myHO[]: 历史单数组。 [历史单信息 HistoryOrders\(HO\)](#)。

mySeekMode: 排序类型 0=按建仓时间,1=按建仓价,2=按盈利,3=按亏损,4-按平仓时间

返回: 无返回。

说明: 按盈利和按亏损排序结果是一样的，都是按降序排列。

16.3.3 刷新历史单数组

```

1.  /*
2.  函    数: 刷新部分历史单数组
3.  输出参数: 历史单数量
4.  算法说明: 刷新指定时间之后的历史单记录
5.  */
6.  int egRefreshHO(HistoryOrders  &myHO[],          //历史单数组
7.                  string          mySymbol,         //指定商品, "*"表示所有持仓
8.                  int              myType,           //指定订单类型,
9.                  int              myMagicNum,        //程序识别码, -1 表示所有持仓
10.                 bool             myOCType,          //建平仓类型, false=建仓,true=
11.                 datetime          myTimeStart,      //历史单开始时间, 0=最早
12.                 datetime          myTimeEnd,        //历史单截止时间, 0=全部
13.                 )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egOC5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egOC4.mqh 文件中。

【参数说明】

myHO[]: 历史单数组。 [历史单信息 HistoryOrders\(HO\)](#)。

mySymbol: 指定商品, "*"表示所有持仓单。

myType: 指定订单类型, 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有。

myMagicNum: 程序识别码, -1 表示所有持仓单。

myOCType: 建平仓类型, false=建仓,true=平仓。建仓平仓时间不同, 可分别定义。

myTimeStart: 历史单开始时间, 0=最早。

myTimeEnd: 历史单截止时间, 0=全部。

注意: 开始结束时间采用了系统时间格式, 是一串正整数, 这个参数需要用系统命令做个转换, 例如:

```
1. StringToTime("2021.4.1 6:00:00")
```

表示将时间"2021.4.1 6:00:00"转换为系统时间。

开始结束时间可以精确到秒。

16.3.4 刷新历史单统计信息

```

1.  /*
2.  函    数: 刷新历史单统计信息
3.  输出参数: false-未统计, true-已统计

```



```

4. 算法:统计 myHO 数组, 给 myHS 赋值。
5. */
6. bool egRefreshHS(HistoryOrders    &myHO[],    //历史单数组
7.                  HistoryStatistical &myHS      //统计结果
8.                  )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

myHO: 指定历史单数组。[历史单信息 HistoryOrders\(HO\)](#)。

myHS : 生成历史单统计信息。[历史单统计信息 HistoryStatistical\(HS\)](#)。

返回: true 表示执行成功, false 表示不成功。

【范例】

1. 在主程序头部预定义一个持仓单统计变量 HS:

```
1. HistoryStatistical TS; //历史单统计变量
```

2. 在主程序 egDataRefresh() 模块中输入以下命令, 将生成一个符合条件的持仓单统计变量:

```
1. egRefreshHS(HO,HS); //刷新历史单统计
```

运行结果将生成一个基于 HO (指定历史单集合) 的实时更新的 HS 变量。

在程序中输入变量 “HS.buy_orders” 就能够获取 HO 持仓单集合中 buy 单的数量。

其他以此类推。

注意:

- 多货币操作时, HO 数组中只能有一个商品入列, 以确保准确的统计结果。
- buy 组利润 (HS.buy_profit) 和 sell 组利润 (HS.sell_profit) 包含了掉期和手续费。

正文参考: [统计](#)。

16.3.5 历史单检索

```

1. /*
2. 函数: 历史单检索
3. 输出参数: 订单号, 0=无订单
4. 算法说明: 冒泡
5. 针对指定的订单数组执行按组合条件定位搜索。
6. */
7. ulong egOrderSearchHst(HistoryOrders    &myHO[],    //历史单数组
8.                        string            mySymbol,    //商品名称, Symbol()为
   当前图表商品名, "*"为所有商品
9.                        int               mySeekMode,    //排序类型 0=按建仓时
   间,1=按建仓价,2=按盈利,3=按亏损,4-按平仓时间,5-按平仓价
10.                       int               myOrderType,    //订单类
   型 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有

```

```

11.          int          myMagicNum,      //程序识别码, -1-所有订
    单
12.          int          mySerialNumber //排序 1 为最大, 2 为次
    大, 以此类推, -1 为最小, -2 为次小, 以此类推
13.          )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

myHO[]: 历史单数组。 [历史单信息 HistoryOrders\(HO\)](#)。

mySymbol: 商品名称, Symbol()为当前图表商品名, "*"为所有商品。

mySeekMode: 排序类型, 0=按建仓时间,1=按建仓价,2=按盈利,3=按亏损,4=按平仓时间,5=按平仓价。

myOrderType: 订单类型, 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有。

myMagicNum: 程序识别码, -1-所有订单。

mySerialNumber: 排序, 1 为最大, 2 为次大, 以此类推, -1 为最小, -2 为次小, 以此类推。

返回: 订单号, 0=查无订单。

【范例】

这条命令可以设置非常丰富的条件组合来查找相应的持仓单。例如:

- 查找所有历史单中浮亏最小的单号, 全部浮盈则返回-1:

```
1. egOrderSearchHst(HO, "*", 3, -1, -1, -1);
```

- 查找 buy 单组中建仓时间最近的单号:

```
1. egOrderSearchHst(HO, "*", 0, 0, -1, 1);
```

- 查找 sell 单组中建仓价最低的单号:

```
1. egOrderSearchHst(HO, "*", 1, 1, -1, -1);
```

- 如果 HO 包括了多个商品, 可以这样查找 EURUSD 的 sell 单组中建仓价最低的单号:

```
1. egOrderSearchHst(HO, "EURUSD", 1, 1, -1, -1);
```

- 如果 HO 中有多个程序控制单, 就可以这样查找程序识别码为 123456 的 sell 单组中建仓价最低的单号:

```
1. egOrderSearchHst(HO, "*", 1, 1, 123456, -1);
```

正文参考: [检索](#)。

16.3.6 历史单定位

```
1. /*
```

```

2. 函    数：历史单定位
3. 输出参数：持仓单在 myHO 数组中的位置, -1 表示找不到订单
4. 算法说明：
5. */
6. int egOrderPosHst(HistoryOrders    &myHO[],    //历史单数组
7.                  ulong              myTicket    //订单号
8.                  )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myHO[]: 持仓单数组。 [历史单信息 HistoryOrders\(HO\)](#)。

myTicket: 订单号。指定一个持仓单号。

返回: 指定订单在 myHO 数组中的序号。

【范例】

myHO 是一个数组，历史单保存在每一个 myHO[i]里，用 for 命令遍历所有历史单时，不能通过订单号实现循环，只能通过订单所在数组的序号来实现。这条命令返回指定订单在 myHO 数组中的序号。例如：

```

1. int i=egOrderPosHst(HO,123456);

```

返回单号为 “123456” 的历史单在 myHO 数组中的序号。

HO[i].closeprice 表示序号为 i 的历史单平仓价。其他属性以此类推。

16.3.7 历史单总数

```

1. /*
2. 函    数：历史单总数
3. 输出参数：历史单数量
4. 算法说明：
5. */
6. int egHstTotal()

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egOC5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egOC4.mqh 文件中。

MT4 和 MT5 获取历史单数量命令不同，为了兼容，编制该函数，该函数返回历史单总数。

16.4 订单操作

16.4.1 建仓

```

1. /*

```

```

2. 函 数:创建订单
3. 输出参数:true-建仓成功, false-建仓失败
4. 算 法:
5. */
6. bool egOrderCreat(string mySymbol, //商品名称
7. int myType, //建仓类
   型 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop
8. double myLots, //建仓量
9. string myComment, //订单注释
10. int myMagicNum, //程序控制码
11. double myPrice, //建仓价(挂单有效)
12. int myDeviation //报价偏差(滑点)
13. )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egOC5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egOC4.mqh 文件中。

【参数说明】

mySymbol: 商品名称。

myType: 建仓类型, 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop。

myLots: 建仓量。

myComment: 订单注释。

myMagicNum: 程序控制码。

myPrice: 建仓价(挂单有效)。Buy 单和 Sell 单建仓价自适应。

myDeviation: 报价偏差(滑点)。输入 0 表示接受任意报价滑点, 输入 10 表示报价滑点不得超过 10 点。

返回: true 表示建仓成功, false 表示建仓不成功。

注意: 由于大多数平台禁止建仓时设置止盈止损, 故建仓命令不预设止盈止损。订单的止盈止损由[持仓单设置止盈止损](#)命令执行。

【范例】

详见[建仓 egCreat](#)。

16.4.2 建仓量整形

```

1. /*
2. 函 数:建仓量整形
3. 输出参数:符合平台规定格式的开仓量。-1 表示整形失败
4. 算 法:调整不规范的开仓量数据, 按照四舍五入原则及平台开仓量格式规范数据
5. */
6. double egLotsFormat(string mySymbol, //货币对名称
7. double myLots //需要整形的开仓量
8. )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

mySymbol: 货币对名称。

myLots: 需要整形的建仓量。

返回: 整形后的建仓量。 -1 表示建仓量无效。

【范例】

```
1. double myLots=egLotsFormat("EURUSD",0.052);
```

建仓量 0.052 手整形后，如果平台最小建仓量为 0.01 手，结果是 0.05 手；如果平台最小建仓量为 0.1 手，整形后的结果为 0.1 手。

16.4.3 同类单间距

```
1. /*
2. 函 数:同类单间距
3. 输出参数:true=允许插入, false=禁止插入
4. 算 法:
5. 目标建仓价正负间隔范围内没有其他同类型持仓单, 允许插入, 否则禁止插入
6. */
7. bool egOrderInsert(TradesOrders &myTO[], //持仓单数组
8.                    int myOrderType, //建仓类
   型 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=任意
9.                    double myPrice, //建仓价
10.                     double myInterval //建仓间隔(报价单位)
11.                    )
```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myTO[]: 持仓单数组。 [持仓单信息 TradesOrders\(TO\)](#)。

myOrderType: 建仓类型 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=任意。

myPrice: 建仓价。buy 单建仓使用 Ask 报价, sell 单建仓使用 Bid 报价, 挂单使用指定价格。

myInterval: 建仓间隔(报价单位)。习惯上用“点数”为单位表达加仓间距, 这里需要做一个换算, 换算成“元”。

返回: true 表示有间距。

16.4.4 指定 k 线持仓单数量

```
1. /*
2. 函 数:指定 k 线持仓单数量
3. 输出参数:指定 k 线持仓单数量
4. 算 法:
5. */
6. int egOrdersInShift(TradesOrders &myTO[], //持仓单数组
```

```

7.          string          mySymbol,          //商品名称
8.          ENUM_TIMEFRAMES myTimeFrame,       //时间框架
9.          int              myType,            //订单类型,
           0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有
10.         int              myMagicNum,        //程序识别码,-1=全
           部
11.         int              myShift,           //指定 k 线, 0=当前 k
           线, 1=左边第 1 根 k 线, 以此类推
12.         ulong            &myTicket[]       //订单号数组
13.         )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

myTO[]: 持仓单数组。[持仓单信息 TradesOrders\(TO\)](#)。

mySymbol: 商品名称

myTimeFrame: 时间框架。MT4 和 MT5 通用, MT5 默认时间周期比 MT4 多。

myType: 订单类型, 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有。

myMagicNum: 程序识别码,-1=全部。

myShift: 指定 k 线, 0=当前 k 线, 1=左边第 1 根 k 线, 以此类推。

myTicket[]: 保存指定 k 线上的持仓单号。

返回: 持仓单数量和 myTicket[]。

这个函数通常用于当前 k 线只加仓一次的操作。

16.4.5 指定 k 线历史单平仓数量

```

1.  /*
2.  函    数:指定 k 线历史单平仓数量
3.  输出参数:指定 k 线历史单平仓数量
4.  算    法:
5.  */
6.  int egOrdersInShiftHst(HistoryOrders &myHO[],          //历史单数组
7.          string          mySymbol,          //商品名称
8.          ENUM_TIMEFRAMES myTimeFrame,       //时间框架
9.          int              myType,            //订单类型,
           0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有
10.         int              myMagicNum,        //程序识别码,-1=全
           部
11.         int              myShift,           //指定 k 线, 0=当前 k
           线, 1=左边第 1 根 k 线, 以此类推
12.         ulong            &myTicket[]       //订单号数组
13.         )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

myHO[]: 持仓单数组。 [历史单信息 HistoryOrders\(HO\)](#)。

mySymbol: 商品名称

myTimeFrame: 时间框架。MT4 和 MT5 通用，MT5 默认时间周期比 MT4 多。

myType: 订单类型，0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有。

myMagicNum: 程序识别码,-1=全部。

myShift: 指定 k 线，0=当前 k 线，1=左边第 1 根 k 线，以此类推。

myTicket[]: 保存指定 k 线上的历史单单号。

返回: 历史单数量和 myTicket[]。

这个函数通常用于当前 k 线有一次平仓不执行建仓加仓的操作。

16.4.6 持仓单设置止盈止损

```

1.  /*
2.  函    数:持仓单设置止盈止损
3.  输出参数:true-成功, false-失败
4.  算法说明:
5.  Buy 单止损价<=Bid-StopLevel      Buy 单止盈价>=Bid+StopLevel
6.  Sell 单止损价>=Ask+StopLevel      Sell 单止盈价<=Ask-StopLevel
7.  */
8.  bool egOrderTPSL(ulong  ticket, //订单号
9.                  int     type,   //止盈=0, 止损=1
10.                 double price    //止盈止损价
11.                 )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egOC5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egOC4.mqh 文件中。

【参数说明】

ticket: 订单号。

type: 止盈=0, 止损=1。

price: 止盈止损价。

返回: 执行成功返回 true，否则返回 false。

这条命令会根据平台规则自动检测并执行符合要求的止盈止损价，避免在 EA 日志中反复报错。

16.4.7 订单平仓

```

1.  /*
2.  函    数:订单平仓
3.  输出参数:true-平仓成功, false-平仓失败
4.  算    法:
5.  */
6.  bool egOrderClose(ulong  ticket, //持仓单单号
7.                  double   lots    //平仓量, 0=全部

```

```
8.          )
```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egOC5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egOC4.mqh 文件中。

【参数说明】

ticket: 订单号。

lots: 平仓量,0=整单平仓,平仓量小于持仓量的,执行减仓操作。

返回: 执行成功返回 true, 否则返回 false。

持仓单为挂单的,删除挂单。

注意: MT5 减仓后,原订单号不变,减仓逻辑需要特别关注。

16.4.8 数组平仓

```
1.  /*
2.  函    数:数组平仓
3.  输出参数:false-平仓失败,true-平仓成功
4.  算    法:
5.  */
6.  bool egOrdersCloseByArray(ulong &myCloseArray[]) //平仓数组
```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

myCloseArray[]: 平仓数组。

返回: 执行成功返回 true, 否则返回 false。

注意: 两张以上持仓单全部平仓务必采用这个命令, 确保平仓动作完全执行完毕。

16.4.9 条件平仓

```
1.  /*
2.  函    数:条件平仓
3.  输出参数:false-平仓失败,true-平仓成功,myCloseTicket
4.  算    法:
5.  */
6.  bool egOrdersCloseByCondition(TradesOrders  &myTO[],           //持仓单源数
                                ulong           &myCloseTicket[], //平仓单目标
                                int             myType,           //平仓单类
                                型 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有
                                int             myMode             //平仓模式
                                0=浮赢,1=浮亏,-1=任意
10.          )
```


命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myTO[]: 持仓单源数组。 [持仓单信息 TradesOrders\(TO\)](#)。

myCloseTicket[]: 平仓单目标数组。 [平仓数组 CloseTicket](#)。

myType: 平仓单类型 0=Buy,1=Sell,2=BuyLimit,3=SellLimit,4=BuyStop,5=SellStop,-1=所有。

myMode: 平仓模式 0=浮赢,1=浮亏,-1=任意。

返回: 执行成功返回 true，否则返回 false。

执行完毕后，平仓数组中保存相关单号，在主程序中由[数组平仓](#)命令执行平仓。

16.4.10 同向对冲

```

1.  /*
2.  函    数:同向对冲
3.  输出参数:true--成功,false--不成功
4.  算    法:所有盈利单与最远开始所有可保本亏损单一起平仓
5.  */
6.  bool egSDHR(TradesOrders    &myTO[],           //持仓单数组
7.              int              myType,           //持仓单类型 0=Buy,1=Sell
8.              ulong            &myCloseTicket[], //平仓数组
9.              double           myInterval1,      //回调间距(价格)
10.             double           myBudgetRate      //对冲平衡系数
11.             )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myTO[]: 持仓单源数组。 [持仓单信息 TradesOrders\(TO\)](#)。

myType: 持仓单类型 0=Buy,1=Sell。

myCloseTicket[]: 平仓数组。

myInterval1: 回调间距(价格)。最远单浮盈达到指定间距后开始执行，习惯上用点数表达，这个参数需要转换成报价格式。

myBudgetRate: 对冲平衡系数=浮盈金额/浮亏金额，如果设置比例为 1 表示正好盈亏相抵，由于平台滑点原因，可能导致平仓后出现小额亏损。一般设置系数为 1.1，表示让平仓结果略微盈利。

返回: true 表示成功，false 表示失败。

myTO 数组中持仓单可以是多商品。

【范例】

```

1.  //--- 同向对冲
2.      if (    true
3.          && TS.buy_orders>1 //有 2 张以上 buy 单
4.          && egSDHR(TO,0,CloseTicket,100*SI.point,1.1)==true
5.          && egPrint(true,"【同向对冲】buy 单组执行同向对冲")
6.      )

```

```

7.    {
8.        return(true);
9.    }

```

这段代码表示当最低价 buy 单浮盈 100 点时，开始执行 buy 单组同向对冲。

这段代码略加修改，可以适应各种对冲条件。例如：回调间距可以设置为 0，前置一个“浮盈单数量>=3”，表示有 3 张浮盈单时，开始执行同向对冲。

16.4.11 反向对冲

```

1.  /*
2.  函 数:反向对冲
3.  输出参数:true--成功,false--不成功
4.  算 法:单边浮盈单与反向类型从最远开始所有可保本亏损单一起平仓
5.  */
6.  bool egODHR(TradesOrders &myTO[],           //持仓单数组
7.              int          myType,             //浮亏组类型 0=Buy,1=Sell
8.              ulong        &myCloseTicket[],   //平仓数组
9.              double        myInterval1,        //回调间距(价格)
10.             double        myBudgetRate        //对冲平衡系数
11.             )

```

命令源码在“\Include\egMT45\”中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myTO[]: 持仓单源数组。[持仓单信息 TradesOrders\(TO\)](#)。

myType: 浮亏组类型 0=Buy,1=Sell。

myCloseTicket[]: 平仓数组。

myInterval1: 回调间距(价格)。最远单浮盈达到指定间距后开始执行，习惯上用点数表达，这个参数需要转换成报价格式。

myBudgetRate: 对冲平衡系数=浮盈金额/浮亏金额，如果设置比例为 1 表示正好盈亏相抵，由于平台滑点原因，可能导致平仓后出现小额亏损。一般设置系数为 1.1，表示让平仓结果略微盈利。

返回: true 表示成功，false 表示失败。

myTO 数组中持仓单可以是多商品。

【范例】

```

1.  //--- 反向对冲
2.      if ( true
3.          && TS.buy_orders>1 //有 2 张以上 buy 单
4.          && egODHR(TO,1,CloseTicket,100*SI.point,1.1)==true
5.          && egPrint(true,"【反向对冲】浮亏 sell 单组执行反向对冲")
6.      )
7.      {

```

```

8.         return(true);
9.     }

```

这段代码表示当最低价 buy 单浮盈 100 点时，开始执行 sell 单组浮亏单反向对冲。

这段代码略加修改，可以适应各种对冲条件。例如：回调间距可以设置为 0，前置一个“浮盈单数量>=3”，表示有 3 张浮盈单时，开始执行反向对冲。

16.4.12 利润保护

```

1.  /*
2.  函 数:利润保护
3.  输出参数:true=执行利保, false=不执行,
4.  算 法:
5.  针对持仓单数组执行利润保护
6.  平仓单数组不为空时, 函数返回 true
7.  */
8.  bool egProfitProtect(TradesOrders &myTO[],           //持仓单数组
9.                      ulong &myCloseTicket[],         //平仓单数组
10.                     ProfitProtect &myPP              //利保变量
11.                     )

```

命令源码在“\Include\egMT45\”中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myTO[]: 持仓单源数组。持仓单信息 [TradesOrders\(TO\)](#)。

myCloseTicket[]: 平仓单数组。平仓数组 [CloseTicket](#)。

myPP: 利保变量。利润保护 [ProfitProtect\(PP\)](#)。

返回: true 表示成功，false 表示失败。

交易终端重启后，该命令重新计算最高浮动盈利。

【范例】

在程序头部预定义利润保护变量：

```

1.  input  string  npt_Note60="===利润保护参数==="; //【利润保护】
2.  input  double  npt_60_1=0.00; //当前货币利保启动金额(正数=金额,0=不执行)
3.  input  double  npt_60_2=0.20; //当前货币利保回撤比例
4.  ProfitProtect PP[1]; //利保变量

```

利保变量可以定义为多维数组，以对应不同的持仓单组。本例定义 1 维。

在主程序 egReduce() 模块中输入以下代码：

```

1.  //--- 利润保护
2.      PP[0].name="当前商品";
3.      PP[0].current_profit=TS.buy_grp_profit+TS.sell_grp_profit+TS.buy_grp_cost
      +TS.sell_grp_cost; //当前利润 浮动利润+成本

```

```

4.     PP[0].profit_start=npt_60_1; //启动金额
5.     PP[0].profit_backrate=npt_60_2; //利润回撤
6.     if ( true
7.         && egProfitProtect(T0,CloseTicket,PP[0])==true
8.         && egPrint(true,"【利润保护】当前商品利润保护达标,市价平仓")
9.         && egPrint(true,"最高利润:"+DoubleToString(PP[0].profit_max,2)+" 当前利润:"+DoubleToString(PP[0].current_profit,2))
10.    )
11.    {
12.        return(true);
13.    }

```

16.4.13 盈损对冲

```

1.  /*
2.  函 数: 盈损对冲
3.  输出参数: true=执行,false=未执行
4.  算法说明:
5.  */
6.  bool egTSHR(TradesStatistical &myTS, //持仓单统计变量
7.              TradesOrders &myTO[], //持仓单信息变量
8.              double myinterval, //盈损单浮盈间距(报价单位)
9.              double myinterval2, //盈损价与现价间距(报价单位)
10.             double myslratio, //对冲系数
11.             double myinterval1, //浮亏减仓间距(报价单位)
12.             double myhedgelots, //单位对冲仓量(正数=减仓量,负数
    =目标仓位比例,0=自适应最小建仓量),持仓单不足单位仓量的按实际仓位计算
13.             int mytype //减仓类型(1=-buy+buy,2=-sell+sell,3=-buy+sell,4=-sell+buy)
14.    )

```

命令源码在“\Include\egMT45\”中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

myTS: 持仓单统计变量。持仓单统计信息 [TradesStatistical\(TS\)](#)。

myTO[]: 持仓单源数组。持仓单信息 [TradesOrders\(TO\)](#)。

myinterval: 盈损单浮盈间距(报价单位)。浮盈单必须达到指定浮盈点数,需要折算成报价单位。如果浮盈单带有止损价,按照止损价计算浮盈间距。

myinterval2: 盈损价与现价间距(报价单位)。预计盈损价与现价的间距,目的是为了防止行情因滑点等因素回调带走有利持仓单。

myslratio: 对冲系数。对冲平衡系数=浮盈金额/浮亏金额,如果设置比例为 1 表示正好盈亏相抵,由于平台滑点原因,可能导致平仓后出现小额亏损。一般设置系数为 1.1,表示让平仓结果略微盈利。

myinterval1: 浮亏减仓间距(报价单位)。浮亏单必须达到指定浮亏点数,需要折算成报价单位。

myhedgelots: 单位对冲仓量。是指每次减仓的量，正数=减仓量，负数=目标仓位比例，0=自适应最小建仓量，持仓单不足单位仓量的按实际仓位计算。

mytype: 减仓类型。1=浮亏 buy 单与浮盈 buy 单对冲，2=浮亏 sell 单与浮盈 sell 单对冲，3=浮亏 buy 单与浮盈 sell 单对冲，4=浮亏 sell 单与浮盈 buy 单对冲。

返回: true 表示成功，false 表示失败。

【范例】

在程序头部预定义盈损对冲变量：

```
1. input    string  npt_Note65=="==盈损对冲参数==" ; // 【盈损对冲】
2. input    double  npt_65_1=-1.0; //盈损单浮盈间距(正数=点,负数=单位高度倍数)
3. input    double  npt_65_2=-3.0; //盈损价与现价间距(正数=点,负数=单位高度倍数)
4. input    double  npt_65_3=-1.0; //浮亏减仓间距(正数=点,负数=单位高度倍数)
5. input    double  npt_65_4=1.1; //对冲系数
6. input    double  npt_65_5=0; //单位对冲仓量(正数=减仓量,负数=目标仓位系数,0=自适应最小建仓量)
7. input    bool    npt_65_6_1=true; //启动浮亏 buy 单与浮盈 buy 组盈损对冲
8. input    bool    npt_65_6_2=true; //启动浮亏 sell 单与浮盈 sell 组盈损对冲
9. input    bool    npt_65_6_3=true; //启动浮亏 buy 单与浮盈 sell 组盈损对冲
10. input    bool    npt_65_6_4=true; //启动浮亏 sell 单与浮盈 buy 组盈损对冲
```

其中“单位高度”为预定义最小波幅，赋值到 SI.unit_high 变量，单位：点。预设参数也可以直接设置为 xx 点。

在主程序 egReduce() 模块中输入以下代码：

```
1. //--- 盈损对冲
2. //参数转换
3. double tshr3=npt_65_1*SI.point; //盈损单浮盈间距(正数=点,负数=单位高度倍数)
4. if (npt_65_1<0) tshr3=-npt_65_1*SI.unit_high*SI.point;
5. double tshr4=npt_65_2*SI.point; //盈损价与现价间距(正数=点,负数=单位高度倍数)
6. if (npt_65_2<0) tshr4=-npt_65_2*SI.unit_high*SI.point;
7. double tshr5=npt_65_4; //对冲系数
8. double tshr6=npt_65_3*SI.point; //浮亏减仓间距(正数=点,负数=单位高度倍数)
9. if (npt_65_3<0) tshr6=-npt_65_3*SI.unit_high*SI.point;
10. double tshr7=npt_65_5; //单位对冲仓量(正数=减仓量,负数=目标仓位系数,0=自适应最小建仓量)
11. if (npt_65_5>0 && npt_65_5<SI.volume_min) tshr7=SI.volume_min;
12. //-buy+buy
13. if ( true
14.     && npt_65_6_1==true
15.     && SignalCode[0]!="buy" //没有 buy 信号
16.     && egTSHR(TS,T0,tshr3,tshr4,tshr5,tshr6,tshr7,1)
```

```

17.      && egPrint(true,"【盈损对冲 BB】当前商品执行浮亏 buy 单与浮盈 buy 单组盈损
    对冲")
18.    )
19.    {
20.      return(true);
21.    }
22.    //-sell+sell
23.    if ( true
24.      && npt_65_6_2==true
25.      && SignalCode[0]!="sell" //没有 sell 信号
26.      && egTSHR(TS,T0,tshr3,tshr4,tshr5,tshr6,tshr7,2)
27.      && egPrint(true,"【盈损对冲 SS】当前商品执行浮亏 sell 单与浮盈 sell 单组盈
    损对冲")
28.    )
29.    {
30.      return(true);
31.    }
32.    //-buy+sell
33.    if ( true
34.      && npt_65_6_3==true
35.      && SignalCode[0]!="sell" //没有 sell 信号
36.      && egTSHR(TS,T0,tshr3,tshr4,tshr5,tshr6,tshr7,3)
37.      && egPrint(true,"【盈损对冲 BS】当前商品执行浮亏 buy 单与浮盈 sell 单组盈损
    对冲")
38.    )
39.    {
40.      return(true);
41.    }
42.    //-sell+buy
43.    if ( true
44.      && npt_65_6_4==true
45.      && SignalCode[0]!="buy" //没有 buy 信号
46.      && egTSHR(TS,T0,tshr3,tshr4,tshr5,tshr6,tshr7,4)
47.      && egPrint(true,"【盈损对冲 SB】当前商品执行浮亏 sell 单与浮盈 buy 单组盈损
    对冲")
48.    )
49.    {
50.      return(true);
51.    }

```

16.5 常规函数

包含所有策略控制中常用的计算函数。

16.5.1 交易状态

```

1.  /*
2.  函    数: 交易状态
3.  输出参数: true=正在交易, false=交易停止
4.  算法说明: 如果服务器时间刷新, 表示市场可以交易, 避免没有交易的时候, 程序不断向服务器
              发送订单操作请求, 导致服务器屏蔽终端。
5.  */
6.  bool egTradingStatus()

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

无参数。

返回: true 表示正在交易, false 表示交易停止。

外汇交易有短暂的休市, 时间长度不确定, 服务器不会发送任何消息。在终端上检测服务器时间变化, 如果服务器时间有更新, 表示市场可以交易, 否则不能执行交易。这样做的目的是为了~~避免没有交易的时候~~, 程序不断向服务器发送订单操作请求, 导致服务器屏蔽终端端口。

本条命令使用了预定义变量[前一个服务器时间 prv_servertime](#)。

【注意】

系统时间 “TimeCurrent()” 表示平台任意商品报价更新, 这个时间都会跟随变化。如果当前商品处于临时休市状态, 函数仍然会返回 “true”, 导致不断向服务器发送指令。在实际应用中, 可以改造一下这个函数代码, 将 “TimeCurrent()” 替换成 “Sl.time”。

如果主程序启用了时间事件触发模块 OnTime(), 则 “TimeCurrent()” 会实时变化, 无法判断市场是否停止交易。

以下是 “egTradingStatus()” 函数代码。

```

1.  /*
2.  函    数: 交易状态
3.  输出参数: true=正在交易, false=交易停止
4.  算法说明: 如果服务器时间刷新, 表示市场可以交易, 避免没有交易的时候, 程序不断向服务器
              发送订单操作请求, 导致服务器屏蔽终端。
5.  */
6.  bool egTradingStatus()
7.  {
8.      //初始化前一时间
9.      if (prv_servertime==0) prv_servertime=TimeCurrent();
10.     if (prv_servertime!=TimeCurrent())
11.     {
12.         prv_servertime=TimeCurrent(); //刷新前一时间
13.         return(true);
14.     }
15.     return(false);

```

```
16. }
```

16.5.2k 线平均高度(点)

```
1.  /*
2.  函 数:k 线平均高度(点)
3.  输出参数:0=无效输出
4.  算 法:从 myStart 开始, myNum 根 k 线的均高
5.  */
6.  int egBarsAvgHigh(string      mySymbol,    //商品名
7.                      ENUM_TIMEFRAMES myPeriod, //周期 0=当前图表时间周期
8.                      int          myStart,    //k 线起点
9.                      int          myNum,      //k 线数量
10.                     int          myMode     //0=开盘收盘,1=最高最低
11.                     )
```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

mySymbol: 商品名称。

myPeriod: 图表时间周期。0=当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同, MT4 只有 9 个标准时间周期, MT5 有很多, 调用时需要特别注意。

myStart: k 线起点。

myNum: k 线数量。从起点开始向左边的数量。

myMode: 0=计算开盘收盘价, 1=计算最高最低价。

返回: 平均高度, 单位为“点”, 0 表示无结果。

【范例】

```
1. double mydata=egBarsAvgHigh(“EURUSD”,0,1,14,0);
```

计算 EURUSD 当前图表从 k1 开始 14 根 k 线实体部分的平均高度。改变图表时间周期, 数值会变化。

```
1. double mydata=egBarsAvgHigh(“EURUSD”,PERIOD_H1,0,55,1);
```

计算 EURUSD 的 1 小时图表从 k0 开始 55 根 k 线整体平均高度。改变图表时间周期, 数值不会变化。

16.5.3 查找订单注释中是否有指定关键字的订单

```
1.  /*
2.  函 数:查找订单注释中是否有指定关键字的订单
3.  输出参数:订单号, -1 为没有
```



```

4. 算 法:
5. 订单注释可以是关键字
6. */
7. int egFindCommentOrderByKey(TradesOrders &myTO[], //持仓单数组
8.                               string      mySymbol, //指定商品, "*"表示所有持
   仓单
9.                               string      myKey, //关键字
10.                              int         myType, //订单类
   型, 0=Buy, 1=Sell, 2=BuyLimit, 3=SellLimit, 4=BuyStop, 5=SellStop, -1=所有
11.                              int         myMagicNum //程序识别码, -1 全部
12.                              )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

myTO[]: 持仓单数组。 [持仓单信息 TradesOrders\(TO\)](#)。

mySymbol: 指定商品, "*"表示所有持仓单。

myKey: 关键字。

myType: 订单类型, 0=Buy, 1=Sell, 2=BuyLimit, 3=SellLimit, 4=BuyStop, 5=SellStop, -1=所有。

myMagicNum: 程序识别码, -1 全部。

返回: 订单号, -1 为没有找到订单。

16.5.4 查找是否有指定程序识别码的订单

```

1. /*
2. 函 数: 查找是否有指定程序识别码的订单
3. 输出参数: 订单号, -1 为没有
4. 算 法:
5. */
6. int egFindMagicNumberOrder(TradesOrders &myTO[], //持仓单数组
7.                              int         myMagicNum, //订单特征码, -1=全部
8.                              int         myType      //订单类
   型, 0=Buy, 1=Sell, 2=BuyLimit, 3=SellLimit, 4=BuyStop, 5=SellStop, -1=所有
9.                              )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中, MT4/5 通用。

【参数说明】

myTO[]: 持仓单数组。 [持仓单信息 TradesOrders\(TO\)](#)。

myMagicNum: 程序识别码, -1 全部。

myType: 订单类型, 0=Buy, 1=Sell, 2=BuyLimit, 3=SellLimit, 4=BuyStop, 5=SellStop, -1=所有。

返回: 订单号, -1 为没有找到订单。

16.5.5 金额换算建仓量

```

1.  /*
2.  函    数: 金额换算建仓量
3.  输出参数: -1 表示转换失败
4.  算    法:
5.  */
6.  double egFundsToHands(SymbolInfo  &mySI,      //商品信息
7.                      double      myFunds,      //资金基数
8.                      int          myType        //订单类型,0=Buy,1=Sell
9.                      )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

mySI: 商品信息，同一金额，不同商品仓位不同。[商品信息 SymbolInfo\(SI\)](#)。

myFunds: 资金基数，单位元。

myType: 订单类型,0=Buy,1=Sell。MT5 的 buy 和 sell 对仓位要求不同。

返回: 手数，-1 表示转换失败。

【范例】

```

1. Comment(egFundsToHands(SI,150,0));

```

屏幕左上角显示 SI 指定商品，金额为\$150 元，建 buy 单对应的手数。

16.5.6 在字符串中获取特征码

```

1.  /*
2.  函    数: 在字符串中获取特征码
3.  输出参数: 特征码
4.  算    法:
5.  */
6.  string egGetFeatureCode(string mystring,      //原字符串
7.                      string mystart,        //开始特征符
8.                      string myend          //结束特征符
9.                      )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

mystring: 原字符串。

mystart: 开始特征符。

myend: 结束特征符。

返回: 特征码。

【范例】

```
1. string myData=egGetFeatureCode("aaa#123*bbb","#","*");
```

一个字符串“aaa#123*bbb”，特征码开始特征符为“#”，结束特征符为“*”，函数返回特征码为“123”。

16.5.7 有效时间段

```
1. /*
2. 函 数：有效时间段
3. 输出参数：true=有效 false=无效
4. 算 法：
5. */
6. bool egTimeValid(string myStartTime, //开始时间，标准格式为 hh:mm
7.                  string myEndTime,   //结束时间，标准格式为 hh:mm
8.                  bool myServerTime   //true 为服务器时间，false 为计算机时
   间
9.                  )
```

命令源码在“\Include\egMT45\”中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myStartTime: 开始时间，标准格式为 hh:mm。

myEndTime: 结束时间，标准格式为 hh:mm。

myServerTime: true 为服务器时间, false 为计算机时间。

返回: true=有效 false=无效。

如果开始时间大于结束时间，则开始时间为前一天时间。

【范例】

```
1. Comment(TimeCurrent()," ",egTimeValid("8:00","19:00",true)
2. ,"\n",TimeLocal()," ",egTimeValid("16:00","15:00",false));
```

这两条命令在屏幕左上角显示有效时间结果：

按服务器时间，每天 8:00~19:00 有效；按计算机时间前一天 16:00~当天 15:00 有效。

16.5.8 双线交叉

```
1. /*
2. 函 数：双线交叉
3. 输出参数：0=金叉,1=死叉,-1=无交叉
4. 算 法：计算快慢线是否交叉
5. */
```

```

6.  int egTwoLineCross(double fastline1, //快线当前 k 线数据
7.                      double slowline1, //慢线当前 k 线数据
8.                      double fastline2, //快线前一 k 线数据
9.                      double slowline2  //慢线前一 k 线数据
10. )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

fastline1: 快线当前 k 线数据

slowline1: 慢线当前 k 线数据

fastline2: 快线前一 k 线数据

slowline2: 慢线前一 k 线数据

返回: 0=金叉,1=死叉,-1=无交叉。

“金叉死叉” 是计算交易信号反转常用的识别方法。

【范例】

```

1.  int mydata=egTwoLineCross(ma_fast1,ma_slow1,ma_fast2,ma_slow2);

```

先计算 k1 的快慢均线，分别赋值到 ma_fast1, ma_slow1, 再计算 k2 的快慢均线，分别赋值到 ma_fast2, ma_slow2。执行该命令后，如果 mydata=0，表示均线在 k1 有一个金叉，如果 mydata=1，表示均线在 k1 有一个死叉，如果 mydata=-1，表示均线在 k1 没有交叉。

参见[均线 egIdx MA](#)。

16.5.9 文件写入

```

1.  /*
2.  函    数: 文件写入
3.  输出参数:true-操作成功, false-操作失败
4.  算    法:
5.  myString 输入输出读写内容
6.  */
7.  bool egFileRW(string    myString,          //读写结果变量
8.                string    name="",          //文件名,含路径"\"
9.                int        modifytype=0,     //写方式,0-新建,1-追加
10.               int        filetype=8        //文件格式,4-BIN,8-CSV,16-TXT
11. )

```

命令源码在 “\Include\egMT45\” 中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myString: 读写结果变量。写入文件必须用字符类型变量，在程序中将其他类型转换成字符类型。如果有多列，则需要加上 “,” 作为间隔。例如：

```
1. string myString=(string)BID+","+(string)ASK
```

表示将 bid 和 ask 报价写入一个 csv 文件。

name: 文件名，注意填写文件后缀名。含路径“\”。实盘模式，目标文件在数据文件夹的“\MQLx\Files”里面。历史测试模式，目标文件在数据文件夹的“tester”里面。

modifytype: 写方式，0-新建，1-追加。新建方式每次都会销毁并重建目标文件，追加方式每次都会将数据添加到文件末尾。

filetype: 文件格式，4-BIN，8-CSV，16-TXT。

返回: true=操作成功，false=操作失败。

16.5.10 读 csv 文件数据到二维数组

```
1. /*
2. 函 数:读 csv 文件数据到二维数组
3. 输出参数:记录数
4. 算 法:
5. 把指定文件 myFileName.csv 的内容按行列顺序读入到 myStrArray 中
6. 二维字符串数组必须预先定义行列尺寸
7. */
8. int egFileCSV_to_Array(string myFileName,    //文件名
9.                        string &myStrArray[][] //字符串数组
10.                      )
```

命令源码在“\Include\egMT45\”中的 egOC.mqh 文件中，MT4/5 通用。

【参数说明】

myFileName: 文件名。文件必须是 csv 格式。

myStrArray[][]: 字符串数组。根据 csv 格式文件的行列预定义。例如文件有 5 行 7 列，则需要预定义数组变量如下：

```
1. string myData[5][7];
```

从文件中读取的数据都是字符类型（string）的，使用前需要按实际情况转换类型。

返回: true=操作成功，false=操作失败。

【范例】

```
1. egFileCSV_to_Array("abc.csv",myData);
```

将 abc.csv 文件中的数据读取到 myData 变量中。

16.6 技术指标

虽然 MT4 与 MT5 读取技术指标的方式不同，但是指标输入输出参数相同，为了实现 MT4/5 兼容，故编制一套技术指标命令集，这套集合不包含获取自定义指标输出命令。

与交易量相关的指标，MT5 比 MT4 更强大一些，预设参数有所不同，故不做融合命令。本次收纳了 MT4、MT5 共用的技术指标共计 25 个。

16.6.1k 线数据

k 线标号约定：当前正在运行尚未完结的 k 线标记为 k0，左边第一根 k 线标记为 k1，左边第二根 k 线标记为 k2，以此类推。

为了兼容，获取 k 线数据约定采用 MT4/5 通用的内部命令，禁止使用 MT4 特有的 Open[] 等命令。

MT4/5 内部命令	获取值
iOpen	开盘价
iClose	收盘价
iHigh	最高价
iLow	最低价
iTime	开盘时间
iVolume	成交量

以上命令可获取指定 k 线的相应数据。详情查看系统命令手册。

16.6.2egIdc_AC

```

1. 函    数：AC 指标
2. 输出参数：指定 k 线序号的 AC 数值
3. 算法说明：
4. */
5. double egIdc_AC(string      symbol,      //交易品种名称
6.                  ENUM_TIMEFRAMES period,  //周期
7.                  int         out,         //输出项
8.                  int         k           //指定 k 线序号
9.                  )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

out: 输出项 0。

k: 指定 k 线序号。

返回: AC 数值。

【范例】

```
1. double myIdc=egIdc_AC(_Symbol,PERIOD_CURRENT,0,1);
```

将当前图表 AC 在左边第一根 k 线（k1）的-DI 值赋给变量 myIdc。

16.6.3 egIdc_ADX

```
1. /*
2. 函 数: ADX 指标
3. 输出参数: 指定 k 线序号的 ADX 数值
4. 算法说明:
5. */
6. double egIdc_ADX(string          symbol,          //交易品种名称
7.                  ENUM_TIMEFRAMES period,          //周期
8.                  int             adx_period,       // 平均周期
9.                  int             out,              //输出项(0-MAIN_LINE, 1-
              PLUSDI_LINE, 2-MINUSDI_LINE)
10.                  int             k                 //指定 k 线序号
11.                  )
```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

adx_period: 平均周期。

out: 输出项，0-MAIN_LINE，1-PLUSDI_LINE，2-MINUSDI_LINE。

k: 指定 k 线序号。

返回: ADX 数值。

注意: 该指标在 MT4 中比 MT5 多一个价格类型选项，为了确保 MT4/5 兼容，默认使用 CLOSE 作为价格类型。

【范例】

```
1. double myIdc=egIdc_ADX(_Symbol,PERIOD_CURRENT,14,2,1);
```

将当前图表 ADX 在左边第一根 k 线（k1）的-DI 值赋给变量 myIdc。

16.6.4 egIdc_Alligator

```
1. /*
2. 函 数: Alligator 指标
```

```

3. 输出参数: 指定 k 线序号的 Alligator 数值
4. 算法说明:
5. */
6. double egIdc_Alligator(string      symbol,      //交易品种名称
7.                        ENUM_TIMEFRAMES period,    //周期
8.                        int          jaw_period,    // 咽喉计算周期
9.                        int          jaw_shift,     // 咽喉平移
10.                       int          teeth_period,  // 牙齿计算周期
11.                       int          teeth_shift,   // 牙齿平移
12.                       int          lips_period,   // 唇部计算周期
13.                       int          lips_shift,    // 唇部平移
14.                       ENUM_MA_METHOD ma_method,  // 平滑类型
15.                       ENUM_APPLIED_PRICE applied_price, // 价格或者处理器类
    型
16.                       int          out,          //输出项, 0-
    GATORJAW_LINE, 1-GATORTEETH_LINE, 2-GATORLIPS_LINE
17.                       int          k            //指定 k 线序号
18.                       )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

jaw_period: 咽喉计算周期。

jaw_shift: 咽喉平移。

teeth_period: 牙齿计算周期。

teeth_shift: 牙齿平移。

lips_period: 唇部计算周期。

lips_shift: 唇部平移。

ma_method: 平滑类型。

applied_price: 价格类型。0=收盘价 (CLOSE)，1=开盘价 (OPEN)，2=最高价 (HIGH)，3=最低价 (LOW)，4=中间价 (MEDIAN (高+低)/2)，5=典型价 (TYPICAL (高+低+收盘价)/3)，6=平均价 (WEIGHTED (高+低+收盘价格+开盘价格)/4)。

out: 输出项，0-GATORJAW_LINE，1-GATORTEETH_LINE，2-GATORLIPS_LINE。

k: 指定 k 线序号。

返回: Alligator 数值。

【范例】

```

1. double myIdc=egIdc_Alligator(_Symbol,PERIOD_CURRENT,13,8,8,5,5,3,MODE_SMMA,P
    RICE_MEDIAN,0,1);

```


将当前图表 Alligator 在左边第一根 k 线（k1）的唇部值赋给变量 myIdc。

16.6.5 egIdc_AO

```

1.  /*
2.  函    数：AO 指标
3.  输出参数：指定 k 线序号的 AC 数值
4.  算法说明：
5.  */
6.  double egIdc_AO(string          symbol,          //交易品种名称
7.                  ENUM_TIMEFRAMES period,          //周期
8.
9.                  int             out,              //输出项
10.                 int             k                 //指定 k 线序号
11.                 )
12. {
13.     return(iaO(symbol,period,k));
14. }
```

MT5 命令源码在“\Include\egMT45\MT5”中的 egIdc5.mqh 文件中。

MT4 命令源码在“\Include\egMT45\MT4”中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入“PERIOD_”开头的关键字，不能输入数字。

out: 输出项 0。

k: 指定 k 线序号。

返回: AO 数值。

【范例】

```
1. double myIdc=egIdc_AO(_Symbol,PERIOD_CURRENT,0,1);
```

将当前图表 AO 在左边第一根 k 线（k1）的-DI 值赋给变量 myIdc。

16.6.6 egIdc_ATR

```

1.  /*
2.  函    数：ATR 指标
3.  输出参数：指定 k 线序号的 ATR 数值
4.  算法说明：
5.  */
```

```

6. double egIdc_ATR(string          symbol,          //交易品种名称
7.                  ENUM_TIMEFRAMES period,          //周期
8.                  int             ma_period,        // 平均周期
9.
10.                 int             out,              //输出项
11.                 int             k                 //指定 k 线序号
12.                 )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

ma_period: 采样周期。

out: 输出项。

k: 指定 k 线序号。

返回: ATR 数值。

【范例】

```

1. double myIdc=egIdc_ATR(_Symbol,PERIOD_CURRENT,14,0,1);

```

将当前图表 ATR 在左边第一根 k 线（k1）的值赋给变量 myIdc。

16.6.7 egIdc_Bands

```

1. /*
2. 函 数: 布林线指标
3. 输出参数: 指定 k 线序号的布林数值
4. 算法说明:
5. */
6. double egIdc_Bands(string          symbol,          // 交易品种名称
7.                  ENUM_TIMEFRAMES period,          // 周期
8.                  int             bands_period,      // 平均线计算周期
9.                  int             bands_shift,       // 指标平移
10.                 double          deviation,         // 标准差数
11.                 ENUM_APPLIED_PRICE applied_price,   // 价格或处理器类型
12.
13.                 int             out,              //输出项(0-
14.                 int             k                 //指定 k 线序号
15.                 )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

bands_period: 采样周期。

bands_shift: 平移。

deviation: 标准差。

applied_price: 价格类型。0=收盘价 (CLOSE)，1=开盘价 (OPEN)，2=最高价 (HIGH)，3=最低价 (LOW)，4=中间价 (MEDIAN (高+低)/2)，5=典型价 (TYPICAL (高+低+收盘价)/3)，6=平均价 (WEIGHTED (高+低+收盘价格+开盘价格)/4)。

out: 输出项，0-Middle,1=Upper,2=Lower。

k: 指定 k 线序号。

返回: Bands 数值。

【范例】

```
1. double myIdc=egIdc_Bands(_Symbol,0,20,0,2,PRICE_CLOSE,0,1);
```

将当前图表布林指标在左边第一根 k 线 (k1) 的中线值赋给变量 myIdc。

16.6.8 egIdc_BearsPower

```
1. /*
2. 函 数: BearsPower 指标
3. 输出参数: 指定 k 线序号的 BearsPower 数值
4. 算法说明:
5. */
6. double egIdc_BearsPower(string          symbol,          // 交易品种名
   称
7.                          ENUM_TIMEFRAMES period,          // 周期
8.                          int             ma_period,        // 平均线计算周
   期
9.
10.                         int             out,               //输出项(0-
   UPPER_LINE,1-LOWER_LINE)
11.                         int             k                   //指定 k 线序号
12.                         )
```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入“PERIOD_”开头的关键字，不能输入数字。

ma_period: 平均周期。

out: 输出项。

k: 指定 k 线序号。

返回: BearsPower 数值。

注意：该指标在 MT4 中比 MT5 多一个价格类型选项，为了确保 MT4/5 兼容，默认使用 CLOSE 作为价格类型。

【范例】

```
1. double myIdc=egIdc_BearsPower(_Symbol,PERIOD_CURRENT,13,0,1);
```

将当前图表 BearsPower 在左边第一根 k 线（k1）的值赋给变量 myIdc。

16.6.9 egIdc_BullsPower

```
1. /*
2. 函 数: BullsPower 指标
3. 输出参数: 指定 k 线序号的 BullsPower 数值
4. 算法说明:
5. */
6. double egIdc_BullsPower(string symbol, // 交易品种名称
7.                          ENUM_TIMEFRAMES period, // 周期
8.                          int ma_period, // 平均线计算周期
9.
10.                         int out, //输出项(0-
11.                         UPPER_LINE,1-LOWER_LINE)
12.                         int k //指定 k 线序号
13. )
```

MT5 命令源码在“\Include\egMT45\MT5”中的 egIdc5.mqh 文件中。

MT4 命令源码在“\Include\egMT45\MT4”中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入“PERIOD_”开头的关键字，不能输入数字。

ma_period: 平均周期。

out: 输出项。

k: 指定 k 线序号。

返回: BullsPower 数值。

注意: 该指标在 MT4 中比 MT5 多一个价格类型选项, 为了确保 MT4/5 兼容, 默认使用 CLOSE 作为价格类型。

【范例】

```
1. double myIdc=egIdc_BearsPower(_Symbol,PERIOD_CURRENT,13,0,1);
```

将当前图表 BullsPower 在左边第一根 k 线 (k1) 的值赋给变量 myIdc。

16.6.10 egIdc_CCI

```
1. /*
2. 函 数: CCI 指标
3. 输出参数: 指定 k 线序号的 CCI 数值
4. 算法说明:
5. */
6. double egIdc_CCI(string          symbol,          // 交易品种名称
7.                  ENUM_TIMEFRAMES period,          // 周期
8.                  int             ma_period,        // 平均线计算周期
9.                  ENUM_APPLIED_PRICE applied_price, // 价格或者处理器类型
10.
11.                  int             out,              //输出项
12.                  int             k                 //指定 k 线序号
13.                  )
```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同, MT4 只有 9 个标准时间周期, MT5 有很多, 调用时需要输入 “PERIOD_” 开头的关键字, 不能输入数字。

ma_period: 采样周期。

applied_price: 价格类型。0=收盘价 (CLOSE), 1=开盘价 (OPEN), 2=最高价 (HIGH), 3=最低价 (LOW), 4=中间价 (MEDIAN (高+低)/2), 5=典型价 (TYPICAL (高+低+收盘价)/3), 6=平均价 (WEIGHTED (高+低+收盘价格+开盘价格)/4)。

out: 输出项。

k: 指定 k 线序号。

返回: CCI 数值。

【范例】

```
1. double myIdc=egIdc_CCI(_Symbol,PERIOD_CURRENT,14,PRICE_TYPICAL,0,1);
```

将当前图表 CCI 指标在左边第一根 k 线（k1）的值赋给变量 myIdc。

16.6.11 egIdc_DeMarker

```

1.  /*
2.  函    数: DeMarker 指标
3.  输出参数: 指定 k 线序号的 DeMarker 数值
4.  算法说明:
5.  */
6.  double egIdc_DeMarker(string          symbol,          // 交易品种名称
7.                        ENUM_TIMEFRAMES period,          // 周期
8.                        int              ma_period,        // 平均线计算周
9.                        int              out,              // 输出项
10.                       int              k                // 指定 k 线序号
11.                       )
12.

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

ma_period: 采样周期。

out: 输出项。

k: 指定 k 线序号。

返回: DeMarker 数值。

【范例】

```

1.  double myIdc=egIdc_DeMarker(_Symbol,PERIOD_CURRENT,14,0,1);

```

将当前图表 DeMarker 指标在左边第一根 k 线（k1）的值赋给变量 myIdc。

16.6.12 egIdc_Envelopes

```

1.  /*
2.  函    数: Envelopes 指标
3.  输出参数: 指定 k 线序号的 Envelopes 数值
4.  算法说明:
5.  */

```

```

6. double egIdc_Envelopes(string symbol, // 交易品种名
   称
7.          ENUM_TIMEFRAMES period, // 周期
8.          int ma_period, // 均线计算周
   期
9.          int ma_shift, // 指标平移
10.         ENUM_MA_METHOD ma_method, // 平滑类型
11.         ENUM_APPLIED_PRICE applied_price, // 价格或者处理器
   类型
12.         double deviation, // 中线边界差(百
   分率)
13.
14.         int out, //输出项(0-
   UPPER_LINE,1-LOWER_LINE)
15.         int k //指定 k 线序号
16.     )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egldc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egldc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

ma_period: 采样周期。

ma_shift: 平移。

ma_method: 平滑类型。0=简单平均（SMA），1=指数平均（EMA），2=平滑平均（SMMA），3=线型平均（LWMA）。

applied_price: 价格类型。0=收盘价（CLOSE），1=开盘价（OPEN），2=最高价（HIGH），3=最低价（LOW），4=中间价（MEDIAN（高+低）/2），5=典型价（TYPICAL（高+低+收盘价）/3），6=平均价（WEIGHTED（高+低+收盘价格+开盘价格）/4）。

deviation: 中线边界差(百分率)

out: 输出项，0-UPPER_LINE,1-LOWER_LINE。

k: 指定 k 线序号。

返回: MA 数值。

【范例】

```

1. double myIdc=egIdc_Envelopes(_Symbol,PERIOD_CURRENT,14,0,MODE_SMA,PRICE_CLOSE,0.1,0,1);

```

将当前图表 Envelopes 指标在左边第一根 k 线（k1）的上线线值赋给变量 myIdc。

16.6.13 egIdc_iFractals

```

1.  /*
2.  函 数: Fractals 指标
3.  输出参数: 指定 k 线序号的 Fractals 数值
4.  算法说明:
5.  */
6.  double egIdc_Fractals(string      symbol,      //交易品种名称
7.                        ENUM_TIMEFRAMES period,    //周期
8.
9.                        int          out,          //输出项(0-
10.                     UPPER_LINE, 1-LOWER_LINE)
11.                     int          k              //指定 k 线序号
12.                     )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

out: 输出项 0。

k: 指定 k 线序号。

返回: Fractals 数值。

【范例】

```

1.  double myIdc=egIdc_Fractals(_Symbol,PERIOD_CURRENT,0,1);

```

将当前图表 Fractals 在左边第一根 k 线（k1）的上行值赋给变量 myIdc，空值返回 EMPTY_VALUE。

16.6.14 egIdc_Gator

```

1.  /*
2.  函 数: Gator 指标
3.  输出参数: 指定 k 线序号的 Gator 数值
4.  算法说明:
5.  */
6.  double egIdc_Gator(string      symbol,      //交易品种名称
7.                    ENUM_TIMEFRAMES period,    //周期
8.                    int          jaw_period,    // 咽喉计算周期

```



```

9.          int          jaw_shift,    // 咽喉平移
10.         int          teeth_period,  // 牙齿计算周期
11.         int          teeth_shift,   // 牙齿平移
12.         int          lips_period,   // 唇部计算周期
13.         int          lips_shift,    // 唇部平移
14.         ENUM_MA_METHOD ma_method,   // 平滑类型
15.         ENUM_APPLIED_PRICE applied_price, // 价格或者处理器类型
16.
17.         int          out,           //输出项,0-
        UPPER_HISTOGRAM, 1-LOWER_HISTOGRAM
18.         int          k             //指定 k 线序号
19.     )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egldc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egldc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

jaw_period: 咽喉计算周期。

jaw_shift: 咽喉平移。

teeth_period: 牙齿计算周期。

teeth_shift: 牙齿平移。

lips_period: 唇部计算周期。

lips_shift: 唇部平移。

ma_method: 平滑类型。

applied_price: 价格类型。0=收盘价 (CLOSE)，1=开盘价 (OPEN)，2=最高价 (HIGH)，3=最低价 (LOW)，4=中间价 (MEDIAN (高+低)/2)，5=典型价 (TYPICAL (高+低+收盘价)/3)，6=均价 (WEIGHTED (高+低+收盘价格+开盘价格)/4)。

out: 输出项，0-UPPER_HISTOGRAM，1-LOWER_HISTOGRAM。

k: 指定 k 线序号。

返回: Gator 数值。

【范例】

```

2. double myIdc=egIdc_Gator(_Symbol,PERIOD_CURRENT,13,8,8,5,5,3,MODE_SMMA,PRICE
   _MEDIAN,0,1);

```

将当前图表 Alligator 在左边第一根 k 线 (k1) 的正数值 UPPER_HISTOGRAM 赋给变量 myIdc。

16.6.15 egIdc_Ichimoku

```

1.  /*
2.  函 数: Ichimoku 指标
3.  输出参数: 指定 k 线序号的 Ichimoku 数值
4.  算法说明:
5.  */
6.  double egIdc_Ichimoku(string      symbol,      //交易品种名称
7.                        ENUM_TIMEFRAMES period,    //周期
8.                        int          tenkan_sen,    // Tenkan-sen 转换线周
9.                        int          kijun_sen,     // Kijun-sen 基准线周
10.                       int          senkou_span_b, // Senkou Span B 周
11.
12.                       int          out,           //输出项(0-
13.                       TENKANSEN_LINE, 1-KIJUNSEN_LINE, 2-SENKOUSPANA_LINE, 3-SENKOUSPANB_LINE, 4-
14.                       CHIKOUPSPAN_LINE)
15.                       int          k              //指定 k 线序号
16.                       )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

tenkan_sen: Tenkan-sen 转换线周期

kijun_sen: Kijun-sen 基准线周期

senkou_span_b: Senkou Span B 周期

out: 输出项，0-TENKANSEN_LINE, 1-KIJUNSEN_LINE, 2-SENKOUSPANA_LINE, 3-SENKOUSPANB_LINE, 4-CHIKOUPSPAN_LINE。

k: 指定 k 线序号。

返回: Ichimoku 数值。

【范例】

```
1. double myIdc=egIdc_Ichimoku(_Symbol,PERIOD_CURRENT,9,26,52,0,1);
```

将当前图表 Ichimoku 在左边第一根 k 线（k1）的 TENKANSEN_LINE 值赋给变量 myIdc。

16.6.16 egIdc_MA

```

1.  /*
2.  函    数: MA 均线指标
3.  输出参数: 指定 k 线序号的 MA 数值
4.  算法说明:
5.  */
6.  double egIdc_MA(string          symbol,          //交易品种名称
7.                  ENUM_TIMEFRAMES period,          //周期
8.                  int             ma_period,        //采样周期
9.                  int             ma_shift,         //平移
10.                 int             ma_method,        //平滑类型
11.                 ENUM_APPLIED_PRICE applied_price,  //价格类型
12.
13.                 int             out,              //输出项(0)
14.                 int             k                 //指定 k 线序号
15.                 )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

ma_period: 采样周期。

ma_shift: 平移。

ma_method: 平滑类型。0=简单平均 (SMA)，1=指数平均 (EMA)，2=平滑平均 (SMMA)，3=线型平均 (LWMA)。

applied_price: 价格类型。0=收盘价 (CLOSE)，1=开盘价 (OPEN)，2=最高价 (HIGH)，3=最低价 (LOW)，4=中间价 (MEDIAN (高+低)/2)，5=典型价 (TYPICAL (高+低+收盘价)/3)，6=平均价 (WEIGHTED (高+低+收盘价格+开盘价格)/4)。

out: 输出项。

k: 指定 k 线序号。

返回: MA 数值。

【范例】

```
1. double myIdc=egIdc_MA(_Symbol,0,10,0,0,PRICE_CLOSE,0,1);
```

将当前图表 MA 在左边第一根 k 线 (k1) 的值赋给变量 myIdc。

16.6.17 egIdc_MACD

```

1.  /*
2.  函    数: MACD 指标
3.  输出参数: 指定 k 线序号的 MACD 数值
4.  算法说明:
5.  */
6.  double egIdc_MACD(string          symbol,          // 交易品种名称
7.                    ENUM_TIMEFRAMES period,          // 周期
8.                    int             fast_ema_period,  // 快速移动平均数周
9.                    int             slow_ema_period,  // 慢速移动平均数周
10.                   int             signal_period,    // 不同点的平均周
11.                   ENUM_APPLIED_PRICE applied_price, // 价格或者处理器类
12.                   int             out,              // 输出项(0-
13.                   MAIN_LINE, 1-SIGNAL_LINE)
14.                   int             k                 // 指定 k 线序号
15.                   )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

fast_ema_period: 快速移动平均数周期

slow_ema_period: 慢速移动平均数周期

signal_period: 不同点的平均周期

applied_price: 价格类型。0=收盘价 (CLOSE)，1=开盘价 (OPEN)，2=最高价 (HIGH)，3=最低价 (LOW)，4=中间价 (MEDIAN (高+低)/2)，5=典型价 (TYPICAL (高+低+收盘价)/3)，6=均价 (WEIGHTED (高+低+收盘价格+开盘价格)/4)。

out: 输出项，0-MAIN_LINE，1-SIGNAL_LINE。

k: 指定 k 线序号。

返回: MACD 数值。

【范例】

```

1.  double myIdc=egIdc_MACD(_Symbol,PERIOD_CURRENT,12,26,9,PRICE_CLOSE,0,1);

```

将当前图表 MACD 在左边第一根 k 线（k1）的值赋给变量 myIdc。

16.6.18 egIdc_Momentum

```

1.  /*
2.  函 数: Momentum 指标
3.  输出参数: 指定 k 线序号的 Momentum 数值
4.  算法说明:
5.  */
6.  double egIdc_Momentum(string          symbol,          // 交易品种名称
7.                          ENUM_TIMEFRAMES period,          // 周期
8.                          int             ma_period,         // 采样周期
9.                          ENUM_APPLIED_PRICE applied_price, // 价格或者处理器类
                                型
10.
11.                          int             out,               // 输出项
12.                          int             k                   // 指定 k 线序号
13.                          )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

ma_period: 采样周期

applied_price: 价格类型。0=收盘价（CLOSE），1=开盘价（OPEN），2=最高价（HIGH），3=最低价（LOW），4=中间价（MEDIAN（高+低）/2），5=典型价（TYPICAL（高+低+收盘价）/3），6=均价（WEIGHTED（高+低+收盘价格+开盘价格）/4）。

out: 输出项。

k: 指定 k 线序号。

返回: Momentum 数值。

【范例】

```

1.  double myIdc=egIdc_Momentum(_Symbol,PERIOD_CURRENT,14,PRICE_CLOSE,0,1);

```

将当前图表 Momentum 在左边第一根 k 线（k1）的值赋给变量 myIdc。

16.6.19 egIdc_OsMA

```

1.  /*

```

```

2. 函 数: OsMA 指标
3. 输出参数: 指定 k 线序号的 MACD 数值
4. 算法说明:
5. */
6. double egIdc_OsMA(string          symbol,          // 交易品种名称
7.                  ENUM_TIMEFRAMES  period,          // 周期
8.                  int               fast_ema_period,  // 快速移动平均数周
   期
9.                  int               slow_ema_period,  // 慢速移动平均数周
   期
10.                 int               signal_period,    // 不同点的平均周
   期
11.                 ENUM_APPLIED_PRICE applied_price,    // 价格或者处理器类
   型
12.
13.                 int               out,              // 输出项(0-
   MAIN_LINE, 1-SIGNAL_LINE)
14.                 int               k                 // 指定 k 线序号
15.                 )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

fast_ema_period: 快速移动平均数周期

slow_ema_period: 慢速移动平均数周期

signal_period: 不同点的平均周期

applied_price: 价格类型。0=收盘价 (CLOSE)，1=开盘价 (OPEN)，2=最高价 (HIGH)，3=最低价 (LOW)，4=中间价 (MEDIAN (高+低)/2)，5=典型价 (TYPICAL (高+低+收盘价)/3)，6=均价 (WEIGHTED (高+低+收盘价格+开盘价格)/4)。

out: 输出项，0-MAIN_LINE，1-SIGNAL_LINE。

k: 指定 k 线序号。

返回: OsMA 数值。

【范例】

```
1. double myIdc=egIdc_OsMA(_Symbol,PERIOD_CURRENT,12,26,9,PRICE_CLOSE,0,1);
```

将当前图表 OsMA 在左边第一根 k 线 (k1) 的值赋给变量 myIdc。

16.6.20 egIdc_RSI

```

1.  /*
2.  函    数: RSI 指标
3.  输出参数: 指定 k 线序号的 RSI 数值
4.  算法说明:
5.  */
6.  double egIdc_RSI(string          symbol,          //交易品种名称
7.                  ENUM_TIMEFRAMES period,          //周期
8.                  int             ma_period,        //采样周期
9.                  ENUM_APPLIED_PRICE applied_price,  //价格类型
10.
11.                  int             out,              //输出项(0)
12.                  int             k                 //指定 k 线序号
13.                  )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

ma_period: 采样周期。

applied_price: 价格类型。0=收盘价 (CLOSE)，1=开盘价 (OPEN)，2=最高价 (HIGH)，3=最低价 (LOW)，4=中间价 (MEDIAN (高+低)/2)，5=典型价 (TYPICAL (高+低+收盘价)/3)，6=平均价 (WEIGHTED (高+低+收盘价格+开盘价格)/4)。

out: 输出项。

k: 指定 k 线序号。

返回: RSI 数值。

【范例】

```

1.  double myIdc=egIdc_RSI(_Symbol,PERIOD_CURRENT,14,PRICE_CLOSE,0,1);

```

将当前图表 RSI 指标在左边第一根 k 线 (k1) 的值赋给变量 myIdc。

16.6.21 egIdc_RVI

```

1.  /*
2.  函    数: RVI 指标
3.  输出参数: 指定 k 线序号的 RVI 数值
4.  算法说明:

```

```

5.  */
6. double egIdc_RVI(string          symbol,      // 交易品种名称
7.                  ENUM_TIMEFRAMES period,      // 周期
8.                  int             ma_period,    // 快速移动平均数周期
9.
10.                 int             out,          // 输出项(0-MAIN_LINE, 1-
11.                 int             k            // 指定 k 线序号
12.                 )

```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

ma_period: 快速移动平均数周期

out: 输出项，0-MAIN_LINE，1-SIGNAL_LINE。

k: 指定 k 线序号。

返回: RVI 数值。

【范例】

```

1. double myIdc=egIdc_RVI(_Symbol,PERIOD_CURRENT,10,0,1);

```

将当前图表 RVI 在左边第一根 k 线（k1）的值赋给变量 myIdc。

16.6.22 egIdc_SAR

```

1.  /*
2.  函    数: SAR 指标
3.  输出参数: 指定 k 线序号的 SAR 数值
4.  算法说明:
5.  */
6. double egIdc_SAR(string          symbol,      //交易品种名称
7.                  ENUM_TIMEFRAMES period,      //周期
8.                  double          step,        //逐步增加
9.                  double          maximum,     //最大止损水平
10.
11.                 int             out,          //输出项(0)
12.                 int             k            //指定 k 线序号
13.                 )

```


MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

step: 逐步增加。

maximum: 最大止损水平。

out: 输出项。

k: 指定 k 线序号。

返回: SAR 数值。

【范例】

```
1. double myIdc=egIdc_SAR(_Symbol,PERIOD_CURRENT,0.02,0.2,0,1);
```

将当前图表 SAR 在左边第一根 k 线（k1）的值赋给变量 myIdc。

16.6.23 egIdc_StdDev

```
1. /*
2. 函 数: StdDev 指标
3. 输出参数: 指定 k 线序号的 StdDev 数值
4. 算法说明:
5. */
6. double egIdc_StdDev(string          symbol,          //交易品种名称
7.                      ENUM_TIMEFRAMES period,          //周期
8.                      int             ma_period,        //采样周期
9.                      int             ma_shift,         //平移
10.                     ENUM_MA_METHOD   ma_method,       //平滑类型
11.                     ENUM_APPLIED_PRICE applied_price,  //价格类型
12.
13.                     int             out,               //输出项(0)
14.                     int             k                 //指定 k 线序号
15.                     )
```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

ma_period: 采样周期。

ma_shift: 平移。

ma_method: 平滑类型。0=简单平均 (SMA), 1=指数平均 (EMA), 2=平滑平均 (SMMA), 3=线型平均 (LWMA)。

applied_price: 价格类型。0=收盘价 (CLOSE), 1=开盘价 (OPEN), 2=最高价 (HIGH), 3=最低价 (LOW), 4=中间价 (MEDIAN (高+低)/2), 5=典型价 (TYPICAL (高+低+收盘价)/3), 6=平均价 (WEIGHTED (高+低+收盘价格+开盘价格)/4)。

out: 输出项。

k: 指定 k 线序号。

返回: StdDev 数值。

【范例】

```
1. double myIdc=egIdc_StdDev(_Symbol,PERIOD_CURRENT,20,0,0,PRICE_CLOSE,0,1);
```

将当前图表 StdDev 在左边第一根 k 线 (k1) 的值赋给变量 myIdc。

16.6.24 egIdc_Stochastic

```
1. /*
2. 函 数: Stochastic 指标
3. 输出参数: 指定 k 线序号的 Stochastic 数值
4. 算法说明:
5. */
6. double egIdc_Stochastic(string symbol, // 交易品种名
   称
7. ENUM_TIMEFRAMES period, // 周期
8. int Kperiod, // K 线周期 (用于
   计算的柱数)
9. int Dperiod, // D 线周期 (开始
   平滑周期)
10. int slowing, // 最终平滑
11. ENUM_MA_METHOD ma_method, //平滑类型
12. ENUM_STO_PRICE applied_price, //价格类型 0-
   STO_LOWHIGH,1-STO_CLOSECLOSE
13.
14. int out, //输出项(0-
   MAIN_LINE, 1-SIGNAL_LINE)
15. int k //指定 k 线序号
16. )
```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入“PERIOD_”开头的关键字，不能输入数字。

Kperiod: K 线周期 (用于计算的柱数)

Dperiod: D 线周期 (开始平滑周期)

slowing: 最终平滑

ma_method: 平滑类型。0=简单平均 (SMA)，1=指数平均 (EMA)，2=平滑平均 (SMMA)，3=线型平均 (LWMA)。

applied_price: 价格类型 0-STO_LOWHIGH,1-STO_CLOSECLOSE

out: 输出项，0-MAIN_LINE，1-SIGNAL_LINE。

k: 指定 k 线序号。

返回: Stochastic 数值。

【范例】

```
double myIdc=egIdc_Stochastic(_Symbol,PERIOD_CURRENT,5,3,3,0,0,0,1);
```

将当前图表 Stochastic 在左边第一根 k 线 (k1) 的 MAIN_LINE 值赋给变量 myIdc。

16.6.25 egIdc_WPR (%R)

```
1.  /*
2.  函    数: WPR 指标
3.  输出参数: 指定 k 线序号的 WPR 数值
4.  算法说明:
5.  */
6.  double egIdc_WPR(string          symbol,      // 交易品种名称
7.                  ENUM_TIMEFRAMES  period,      // 周期
8.                  int               ma_period,    // 快速移动平均数周期
9.
10.                 int               out,          // 输出项
11.                 int               k             // 指定 k 线序号
12.                 )
```

MT5 命令源码在“\Include\egMT45\MT5”中的 egIdc5.mqh 文件中。

MT4 命令源码在“\Include\egMT45\MT4”中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入“PERIOD_”开头的关键字，不能输入数字。

ma_period: 采样周期。

out: 输出项。

k: 指定 k 线序号。

返回：WPR 数值。

【范例】

```
1. double myIdc=egIdc_WPR(_Symbol,PERIOD_CURRENT,14,0,1);
```

将当前图表 WPR 指标在左边第一根 k 线（k1）的值赋给变量 myIdc。

16.6.26 egIdc_ZigZag

```
1. /*
2. 函 数: ZigZag 指标
3. 输出参数: 指定 k 线序号的 ZigZag 数值
4. 算法说明:
5. */
6. double egIdc_ZigZag(string      symbol,      // 交易品种名称
7.                      ENUM_TIMEFRAMES period,    // 周期
8.                      int         InpDepth,      // Depth
9.                      int         InpDeviation,  // Deviation
10.                     int         InpBackstep,   // Backstep
11.
12.                     int         out,           //输出项
13.                     int         k             //指定 k 线序号
14.                     )
```

MT5 命令源码在 “\Include\egMT45\MT5” 中的 egIdc5.mqh 文件中。

MT4 命令源码在 “\Include\egMT45\MT4” 中的 egIdc4.mqh 文件中。

【参数说明】

Symbol: 交易品种名称。

period: 图表周期。0 表示当前图表时间周期。MT4 和 MT5 在图表时间周期上的定义不同，MT4 只有 9 个标准时间周期，MT5 有很多，调用时需要输入 “PERIOD_” 开头的关键字，不能输入数字。

InpDepth: Depth。

InpDeviation: Deviation。

InpBackstep: Backstep。

out: 输出项。

k: 指定 k 线序号。

返回：ZigZag 数值。

【范例】

```
1. double myIdc=egIdc_ZigZag(SI.symbol,PERIOD_CURRENT,12,5,3,0,11);
```

将当前图表 ZigZag 指标在左边第 11 根 k 线（k11）的值赋给变量 myIdc。

16.7 图形对象

16.7.1 输出信息到图表

```

1.  /*
2.  函    数:输出信息到图表
3.  输出参数:true-成功
4.  算    法:
5.  */
6.  bool egComment(bool    isout=true,    //允许输出
7.                string  text="text"    //输出信息
8.                )

```

命令源码在“\Include\egMT45\”中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

isout: 允许输出。

text: 输出信息。

改造系统命令“Comment”为 bool 类型，方便插入到条件语句中执行调式。

返回: true 表示执行成功。

16.7.2 输出信息到日志

```

1.  /*
2.  函    数:输出信息到日志
3.  输出参数:true-成功
4.  算    法:
5.  */
6.  bool egPrint(bool    isout=true,    //允许输出
7.                string  text=""        //输出信息
8.                )

```

命令源码在“\Include\egMT45\”中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

isout: 允许输出。

text: 输出信息。

改造系统命令“Print”为 bool 类型，方便插入到条件语句中执行调式。

返回: true 表示执行成功。

16.7.3 输出向下箭头

```

1.  /*

```

```

2. 函数:在指定时间价位标注向下箭头
3. 输出参数:true=输出成功
4. 算法:
5. */
6. bool egObj_ArrowDown(const long          chart_ID=0,          // chart'
   s ID
7.                  const string          name="ArrowUp",        // sign n
   ame
8.                  const int            sub_window=0,          // subwin
   dow index
9.                  datetime              time=0,                // anchor
   point time
10.                 double                price=0,                // anchor
   point price
11.                 const ENUM_ARROW_ANCHOR anchor=ANCHOR_TOP,    // anchor
   type
12.                 const color           clr=clrGreen,          // sign c
   olor
13.                 const ENUM_LINE_STYLE style=STYLE_SOLID,      // border
   line style
14.                 const int             width=3,                // sign s
   ize
15.                 const bool            back=false,             // in the
   background
16.                 const bool            selection=true,          // highli
   ght to move
17.                 const bool            hidden=true,            // hidden
   in the object list
18.                 const long            z_order=0                // priori
   ty for mouse click
19. )

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回: true 表示执行成功。

16.7.4 输出向上箭头

```

1. /*
2. 函数:在指定时间价位标注向上箭头
3. 输出参数:true=输出成功
4. 算法:
5. */

```

```

6.  bool egObj_ArrowUp(const long          chart_ID=0,          // chart's
   ID
7.                      const string      name="ArrowUp",      // sign nam
   e
8.                      const int         sub_window=0,        // subwindo
   w index
9.                      datetime          time=0,              // anchor p
   oint time
10.                     double            price=0,              // anchor p
   oint price
11.                     const ENUM_ARROW_ANCHOR anchor=ANCHOR_BOTTOM, // anchor t
   ype
12.                     const color        clr=clrRed,          // sign col
   or
13.                     const ENUM_LINE_STYLE style=STYLE_SOLID, // border l
   ine style
14.                     const int          width=3,              // sign siz
   e
15.                     const bool         back=false,          // in the b
   ackground
16.                     const bool         selection=true,       // highligh
   t to move
17.                     const bool         hidden=true,          // hidden i
   n the object list
18.                     const long         z_order=0             // priority
   for mouse click
19.                                     )

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.5 以像素坐标输出 BMP 图形

```

1.  /*
2.  函    数: 以像素坐标输出 BMP 图形
3.  输出参数: true=输出成功
4.  算    法:
5.  参    数
6.  */
7.  bool egObj_BMP(const long          chart_ID=0,          // 图
   表 ID

```

8. 称	<code>const string</code>	<code>name="BmpLabel",</code>	<code>// 标签名</code>
9. 数	<code>const int</code>	<code>sub_window=0,</code>	<code>// 子窗口指</code>
10.	<code>const int</code>	<code>x=0,</code>	<code>// X 坐标</code>
11.	<code>const int</code>	<code>y=0,</code>	<code>// Y 坐标</code>
12. 图像	<code>const string</code>	<code>file_on="",</code>	<code>// On 模式的</code>
13. 的图像	<code>const string</code>	<code>file_off="",</code>	<code>// Off 模式</code>
14. 度	<code>const int</code>	<code>width=0,</code>	<code>// 图形宽</code>
15. 度	<code>const int</code>	<code>height=0,</code>	<code>// 图形高</code>
16. 沿着 X 轴移动	<code>const int</code>	<code>x_offset=10,</code>	<code>// 可见范围</code>
17. 沿着 Y 轴移动	<code>const int</code>	<code>y_offset=10,</code>	<code>// 可见范围</code>
18. 布	<code>const bool</code>	<code>state=false,</code>	<code>// 出版/发</code>
19. 角	<code>const ENUM_BASE_CORNER</code>	<code>corner=CORNER_LEFT_UPPER,</code>	<code>// 图表定位</code>
20. 型	<code>const ENUM_ANCHOR_POINT</code>	<code>anchor=ANCHOR_LEFT_UPPER,</code>	<code>// 定位类</code>
21. 时的边界颜色	<code>const color</code>	<code>clr=clrRed,</code>	<code>// 高亮显示</code>
22. 时的线条风格	<code>const ENUM_LINE_STYLE</code>	<code>style=STYLE_SOLID,</code>	<code>// 高亮显示</code>
23. 小	<code>const int</code>	<code>point_width=1,</code>	<code>// 移动点大</code>
24. 中	<code>const bool</code>	<code>back=false,</code>	<code>// 在背景</code>
25. 动	<code>const bool</code>	<code>selection=false,</code>	<code>// 突出移</code>
26. 象列表	<code>const bool</code>	<code>hidden=true,</code>	<code>// 隐藏在对</code>
27. 优先	<code>const long</code>	<code>z_order=0</code>	<code>// 鼠标单击</code>
28.	<code>)</code>		

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.6 输出按钮

```

1.  /*
2.  函 数:输出按钮
3.  输出参数:true=输出成功
4.  算 法:
5.  */
6.  bool      egObj_Button(const long      chart_ID=0,          // 图
   表 ID
7.              const string      name="Button",          // 按
   钮名称
8.              const int      sub_window=0,          // 子
   窗口指数
9.              const int      x=0,          // X
   坐标
10.             const int      y=0,          // Y
   坐标
11.             const int      width=50,          // 按
   钮宽度
12.             const int      height=18,          // 按
   钮高度
13.             const ENUM_BASE_CORNER corner=CORNER_LEFT_UPPER, // 图
   表定位角
14.             const string      text="Button",          // 文
   本
15.             const string      font="Arial",          // 字
   体
16.             const int      font_size=10,          // 字
   体大小
17.             const color      clr=clrBlack,          // 文
   本颜色
18.             const color      back_clr=C'236,233,216', // 背
   景色
19.             const color      border_clr=clrNONE,      // 边
   界颜色
20.             const bool      state=false,          // 按
   下/弹起
21.             const bool      back=false,          // 在
   背景中
22.             const bool      selection=false,          // 突
   出移动
23.             const bool      hidden=true,          // 隐
   藏在对象列表

```

```

24.          const long          z_order=0          // 鼠
    标单击优先
25.          )

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.7 输出可编辑文字框

```

1.  /*
2.  函 数:输出可编辑文字框
3.  输出参数:true=输出成功
4.  算 法:
5.  */
6.  bool egObj_Edit(const long          chart_ID=0,          // 图表 ID
7.                  const string        name="Edit",          // 对象名
   称
8.                  const int           sub_window=0,          // 子窗口指
   数
9.                  const int           x=0,                   // X 坐标
10.                 const int           y=0,                   // Y 坐标
11.                 const int           width=50,               // 宽度
12.                 const int           height=18,              // 高度
13.                 const string        text="Text",            // 文本
14.                 const string        font="Arial",           // 字体
15.                 const int           font_size=10,            // 字体大
   小
16.                 const ENUM_ALIGN_MODE align=ALIGN_CENTER,   // 对齐类
   型
17.                 const bool          read_only=true,          // 编辑
   false/只读 true
18.                 const ENUM_BASE_CORNER corner=CORNER_LEFT_UPPER, // 图表定位
   角
19.                 const color         clr=clrBlack,            // 文本颜
   色
20.                 const color         back_clr=clrWhite,        // 背景色
21.                 const color         border_clr=clrNONE,       // 边框颜
   色
22.                 const bool          back=false,              // 前景
   false/背景 true
23.                 const bool          selection=false,          // 突出移
   动

```

```

24.          const bool          hidden=true,          // 隐藏在对
    象列表
25.          const long          z_order=0            // 鼠标单击
    优先
26.          )

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.8 输出水平线

```

1.  /*
2.  函 数:输出水平线
3.  输出参数:true-输出成功
4.  算 法:
5.  */
6.  bool egObj_HLine(const long chart_ID=0,          //主图 ID
7.                  const string name="HLine",       //对象名称
8.                  const int sub_window=0,          //副图编号
9.                  const double price=0,            //线价格
10.                 const color clr=clrRed,           //线色
11.                 const ENUM_LINE_STYLE style=STYLE_SOLID, //线型
12.                 const int width=1,                //线宽
13.                 const bool back=false,            //设置为背景
14.                 const bool selection=true,         //高亮移动
15.                 const bool hidden=true,           //列表中隐藏对
    象名
16.                 const long z_order=0              // priority fo
    r mouse click
17.          )

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.9 输出矩形标签

```

1.  /*
2.  函 数:输出矩形标签
3.  输出参数:true=输出成功

```

```

4. 算 法:
5. */
6. bool egObj_RectLabel(const long          chart_ID=0,          // 图
   表 ID
7.                  const string          name="RectLabel",      // 标签
   名称
8.                  const int            sub_window=0,          // 子窗
   口指数
9.                  const int            x=0,                  // X
   坐标
10.                 const int            y=0,                  // Y
   坐标
11.                 const int            width=50,              // 宽
   度
12.                 const int            height=18,            // 高
   度
13.                 const color          back_clr=C'236,233,216', // 背景
   色
14.                 const ENUM_BORDER_TYPE border=BORDER_SUNKEN, // 边框
   类型
15.                 const ENUM_BASE_CORNER corner=CORNER_LEFT_UPPER, // 图表
   定位角
16.                 const color          clr=clrRed,            // 平面
   边框颜色 (Flat)
17.                 const ENUM_LINE_STYLE style=STYLE_SOLID,    // 平面
   边框风格
18.                 const int            line_width=1,          // 平面
   边框宽度
19.                 const bool            back=false,           // 在背
   景中
20.                 const bool            selection=false,       // 突出
   移动
21.                 const bool            hidden=true,           // 隐藏
   在对象列表
22.                 const long            z_order=0              // 鼠标
   单击优先
23.                 )

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.10 输出趋势线

```

1.  /*
2.  函    数:输出趋势线
3.  输出参数:true-输出成功
4.  算    法:
5.  注意: 动态趋势线需要先删除对象
6.  */
7.  bool egObj_Trend(const long          chart_ID=0,          // 图表 ID
8.                  const string        name="TrendLine",    // 线的名称
9.                  const int           sub_window=0,         // 子窗口指数
10.                 datetime             time1=0,             // 第一个点的时
    间
11.                 double               price1=0,            // 第一个点的价
    格
12.                 datetime             time2=0,            // 第二个点的时
    间
13.                 double               price2=0,            // 第二个点的价
    格
14.                 const color          clr=clrRed,          // 线的颜色
15.                 const ENUM_LINE_STYLE style=STYLE_SOLID,  // 线的风格
16.                 const int            width=1,             // 线的宽度
17.                 const bool           back=false,          // 在背景中
18.                 const bool           selection=true,       // 突出移动
19.                 const bool           ray_left=false,       // 线延续向左
20.                 const bool           ray_right=false,      // 线延续向右
21.                 const bool           hidden=true,          // 隐藏在对象列
    表
22.                 const long           z_order=0)           // 鼠标单击优先

```

MT5 命令源码在“\Include\egMT45\MT5”中的 egUI5.mqh 文件中。

MT4 命令源码在“\Include\egMT45\MT4”中的 egUI4.mqh 文件中。

命令源码在“\Include\egMT45\”中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回: true 表示执行成功。

16.7.11 输出垂直线

```

1.  /*
2.  函    数:输出垂直线到图表
3.  输出参数:true-输出成功
4.  算    法:

```

```

5.  */
6. bool egObj_VLine(const long chart_ID=0, //主图 ID
7.                  const string name="VLine", //对象名称
8.                  const int sub_window=0, //副图编号
9.                  const datetime time=0, //线时间
10.                 const color clr=clrRed, //线色
11.                 const ENUM_LINE_STYLE style=STYLE_SOLID, //线型
12.                 const int width=1, //线宽
13.                 const bool back=true, //设置为背景
14.                 const bool selection=false, //高亮移动
15.                 const bool hidden=true, //列表中隐藏对
    象名
16.                 const long z_order=0 // priority fo
    r mouse click
17. )

```

命令源码在“\Include\egMT45\”中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.12 点击鼠标获取对象名

```

1.  /*
2.  函 数:点击鼠标获取对象名
3.  输出参数:对象名
4.  算 法:
5.  */
6. string egUI_GetNameByMouseClicked(long lparam, //x 坐标
7.                                     double dparam, //y 坐标
8.                                     string sparam
9. )

```

命令源码在“\Include\egMT45\”中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.13 图表环境初始化

```

1.  /*
2.  函 数:图表环境初始化
3.  输出参数:无

```

```

4. 算 法:
5. */
6. void egUI_EventInit()

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

无。

返回：无。

这条命令用在主程序 OnInit()模块中，执行对象创建于显示、鼠标事件等图表初始化任务。

16.7.14 移动鼠标获取对象名

```

1. /*
2. 函 数:移动鼠标获取对象名
3. 输出参数:对象名
4. 算 法:
5. */
6. string egUI_GetNameByMouseMove(long lparam, //x 坐标
7.                                double dparam //y 坐标
8.                                )

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.15 鼠标移动到对象显示备注

```

1. /*
2. 函 数:鼠标移动到对象显示备注
3. 输出参数:true=成功显示
4. 算 法:
5. 分为两种显示方式
6. note_type=0, 显示文字, 在 obj_comment 输入文字。
7. note_type=1, 显示图片, 要求 BMP 格式, 尺寸 180*180。在 obj_comment 输入图片路径和名称, 范例: \\Images\\EasyGo\\contact.bmp。
8. */
9. bool egUI_ObjectEjectNoteByMouse(string obj_name, //对象名
10.                                  string obj_comment, //文字说明, 如果是, 路径格式
11.                                  int note_type, //显示类型 0=文字, 1=图片
12.                                  long lparam, //x 坐标

```

```

13.                                     double dparam //y 坐标
14.                                     )

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

其中显示方式有两种：显示文字和显示文字。

显示图片，要求 BMP 格式，尺寸 180*180。在 obj_comment 输入图片路径和名称，范例：
\\Images\\EasyGo\\contact.bmp。

显示文字，在 obj_comment 输入文字。

返回：true 表示执行成功。

16.7.16 鼠标移动对象

```

1.  /*
2.  函    数:鼠标移动对象
3.  输出参数:true=成功移动
4.  算    法:
5.  */
6.  bool egUI_ObjectDragByMouse(ObjectAttributeStruct &mydrag, //拖动对象
7.                               string drag_name, //指定对象名,"*"=任意对象
8.                               long lparam, //x 坐标
9.                               double dparam, //y 坐标
10.                               string sparam
11.                               )

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.17 对象跟随坐标移动

```

1.  /*
2.  函    数:对象跟随坐标移动
3.  输出参数:true=成功移动
4.  算    法:
5.  */
6.  bool egUI_ObjectMoveByXY(string obj_name, //指定对象名
7.                           int x, //x 坐标
8.                           int y //y 坐标
9.                           )

```


命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.18 刷新对象显示内容

```

1.  /*
2.  函    数:刷新对象显示内容
3.  输出参数:ture=成功显示
4.  算    法:
5.  */
6.  bool egUI_ObjectTextRefresh(string object_name, //对象名称
7.                                string object_text //显示内容
8.                                )

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.19 删除关键字对象

```

1.  /*
2.  函    数:删除关键字对象
3.  输出参数:true-成功删除
4.  算    法:
5.  */
6.  bool egUI_ObjectsDeleteByKeyword(string keyword) //关键字

```

命令源码在 “\Include\egMT45\” 中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回：true 表示执行成功。

16.7.20 组合对象:步进取值

```

1.  /*
2.  函    数:组合对象:步进取值
3.  输出参数:可编辑框数值
4.  算    法:共 3 个对象:步进减按钮+可编辑框数值+步进加按钮。其中:
5.  myobj1:=步进加按钮

```

```

6. myobj2:=可编辑框数值
7. myobj3:=步进减按钮
8. */
9. string egUI_ObjGrp_StepBox(ObjectAttributeStruct &myobj1, //步进加按钮
10.                               ObjectAttributeStruct &myobj2, //可编辑框数值
11.                               ObjectAttributeStruct &myobj3, //步进减按钮
12.                               int width, //宽度
13.                               int height, //高度
14.                               int x, //x 坐标
15.                               int y, //y 坐标
16.                               double step
17.                               )

```

命令源码在“\Include\egMT45\”中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回: true 表示执行成功。

范例:

step1 预定义图形对象数组。

```
1. ObjectAttributeStruct obj_array[1000]; //对象数组，对应面板所有对象
```

step2

给步进取值组合三个图形对象赋值。

```

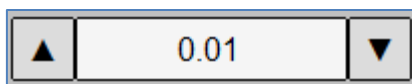
1. obj_array[82].name=windows_name+"_0_ObjGrp_1_1"; //步进加按钮名称
2. obj_array[83].name=windows_name+"_0_ObjGrp_1_2"; //可编辑框数值名称
3. obj_array[84].name=windows_name+"_0_ObjGrp_1_3"; //步进减按钮名称
4. obj_array[82].x=200;
5. obj_array[82].y=200;

```

step3 在主图画出步进取值图形。

```
1. egUI_ObjGrp_StepBox(obj_array[82],obj_array[83],obj_array[84],200,32,300,300,0.01);
```

执行代码，在主图像素坐标[300,300]位置显示一个长 200 像素，高 32 像素的步进取值图形。



16.7.21 组合对象:单选

```

1.  /*
2.  函    数:组合对象:单选
3.  输出参数:选中/未选
4.  算    法:共 2 个对象: 选中按钮+可编辑框数值。其中:
5.  myobj1:=选中按钮
6.  myobj2:=可编辑框
7.  */
8.  bool egUI_ObjGrp_CheckBox(ObjectAttributeStruct &myobj1, //选中按钮
9.                               ObjectAttributeStruct &myobj2, //可编辑框
10.                               int width, //宽度
11.                               int height, //高度
12.                               int x, //x 坐标
13.                               int y //y 坐标
14.                               )

```

命令源码在“\Include\egMT45\”中的 egUI.mqh 文件中，MT4/5 通用。

【参数说明】

见上述函数注释。

返回: true 表示执行成功。

范例:

step1 预定义[图形对象](#)数组。

```

1. ObjectAttributeStruct obj_array[1000]; //对象数组，对应面板所有对象

```

step2

给单选组合两个图形对象赋值。

```

1. //单选组合
2. obj_array[91].name=windows_name+"_0_ObjGrp_2_1"; //按钮名称，外加 ObjGrp_2_1 关
   键字，区分其他对象，摆放在第 1 层
3. obj_array[92].name=windows_name+"_0_ObjGrp_2_2"; //文字框名称，外加 ObjGrp_2_2
   关键字，区分其他对象，摆放在第 1 层
4. obj_array[92].text="浮盈平仓金额$";
5. obj_array[92].x=200;
6. obj_array[92].y=200;

```

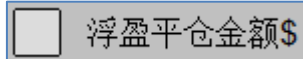
step3 在主图画上单选图形。

```

1. egUI_ObjGrp_CheckBox(obj_array[91],obj_array[92],180,24,obj_array[92].x,obj_
   array[92].y);

```

执行代码，在主图像素坐标[200,200]位置显示一个长 180 像素，高 24 像素的单选框。



16.7.22 关于面板制作

函数名以“egUI_”开头的都是为编制图形化控制面板准备的，融合编程 1.9 版在使用过程中发现控制面板在 MT5 环境中十分卡顿，我用系统自带的标准程序库（图形库）尝试后，解决了这个问题。新的更显得专业和顺滑的控制面板如下：



这套解决方案将另行发布。